UNREAL NETWORKING • 项目复盘 • 游戏客户端秋招版

UE 双人联机射击 Demo：面试复盘与正确实现

一个从单机 FPS 改成双人合作打僵尸的 UE 5.5 蓝图 Demo：先把它的 28 个蓝图逐节点读出来，找出它真实的运行方式和问题；再按 UE 联机的正确方式，把“开火到击中僵尸”“角色、控制器、武器怎么绑定”“弱网卡顿怎么优化”“子弹同步与换弹”这几道面试题讲透，每个结论都有源码证据或实测数据。

原工程审计：19 个 RPC · 11 个复制变量 · 23 条问题

7 组网络行为实测（MSVC /O2）

逐类蓝图事件图表 40+ 张（每张标出执行端 / 复制变量 / RPC / 组播）· 2 个交互实验

12 条追问链 + 56 道面试题 + 速记卡

CHAPTER 00

先背这个：四道题的 30 秒面试版答案

面试官拿着简历上的这个 Demo 往下问，先用 30 秒给出“正确做法”的骨架，再主动说出原实现哪里不对、后来怎么改的——这比假装原实现没问题加分得多。

问 ①：从按下开火到击中僵尸，中间发生了什么？

本机：Enhanced Input 触发武器的 `StartFire`；本地先检查本地记录的弹药和状态，**立刻**播开火动画、枪口火焰、声音，并预测扣一发（这些都是表现，不是结果）。然后把这一发的信息发给服务器：`ServerFire(序号, 射线起点, 方向, 客户端时间)`。

服务器：校验（活着、没在换弹、射速、弹药、起点和方向是否合理）→ 扣弹 → 做射线检测（PvP 要按客户端时间回溯目标，PvE 打僵尸可以放宽成“客户端判定 + 服务器容差校验”）→ 命中后 `ApplyPointDamage`，Instigator 填开火者的 Controller、DamageCauser 填武器。僵尸的 `TakeDamage` **只在服务器**改血量；血量归零就进入死亡状态、停 AI、给开火者的 PlayerState 加分。

回到各端：弹药按 `COND_OwnerOnly` 复制回开火者，用序号对账，UI 不回跳；开火计数 `BurstCounter` 按 `COND_SkipOwner` 复制给另一端，让对方播开火表现；僵尸的血量 / 死亡标志通过 OnRep 让每一端播受击和倒地。本机感觉零延迟，另一端大约晚“半个 RTT + 一个网络帧”看到，**伤害和死亡只以服务器为准。**

问 ②：控制的角色、控制器、武器应该怎么绑定？

GameMode 在服务器为每个登录的 `PlayerController` 生成 Pawn 并 `Possess`；Possess 会把 Pawn 的 Owner 设成这个 PC。PC 只存在于服务器和它自己的客户端，所以 Owner 链“武器 → Pawn → PC → 网络连接”决定了：**谁能在这个 Actor 上调用 Server RPC**、`COND_OwnerOnly` 发给谁、Client RPC 去哪。

武器在**服务器**生成，生成参数里直接给 `Owner = Pawn`、`Instigator = Pawn`（不要先生成再 SetOwner，BeginPlay 会早于 SetOwner）；挂到第三人称模型的手部 Socket，另一份挂到只有

自己看得见的第一人称手臂；`CurrentWeapon` 复制并在 OnRep 里做挂接和动画层切换。输入映射在“成为本地控制者”时添加（`NotifyControllerChanged` / `PawnClientRestart`），不要在 BeginPlay 里用 `GetPlayerController(0)`。分数、是否准备这类跟人走的数据放 `PlayerState`。

问 ③：弱网造成卡顿，应该怎么优化？

先测再改：PIE 打开网络模拟（100~200 ms、5~10% 丢包），用 `stat net`、`p.NetShowCorrections 1`、Networking Insights 判断是哪一类问题。

① **操作反馈慢** → 客户端预测：开火表现、弹药、换弹动画本地先播，服务器确认后对账；② **移动被拉回** → 所有影响移动速度的状态（冲刺、瞄准、下蹲）必须进 CharacterMovement 的 SavedMove 一起预测，不能另走 RPC，更不能两端各算各的（实测：冲刺只在服务器生效时每秒纠正 9.2 次）；③ **明明瞄准了却没打中** → 服务器按开火时刻回溯目标，或客户端判定 + 服务器容差（实测：目标 4.8 m/s、RTT 200 ms 时不补偿命中率只有 10.9%）；④ **消息堆积、带宽高** → 逐发 Reliable 改为计数器 / 带序号的 Unreliable，表现类消息一律不用 Reliable，降低 NetUpdateFrequency、用休眠和相关性，抛射物用“假子弹”。远端角色靠插值平滑，服务器 Tick 和带宽上限按需要调整。

问 ④：子弹同步、换弹是怎么实现的？

弹药：服务器权威，`COND_OwnerOnly` 只复制给持枪者；本地预测扣弹，显示值 = 服务器弹数 − 还没被服务器确认的发数，避免 OnRep 把 UI 拉回去（实测：直接覆盖每次连射平均回跳 1 次，序号对账 0 次）。**别人看到的子弹**：命中扫描武器只同步“开了几枪 + 打在哪”，各端本地画曳光和弹孔；真抛射物由服务器权威生成，开火者本地先放一颗不复制的假子弹，不要每发生成一个复制 Actor。

换弹是服务器上的一个小状态机：`ServerStartReload` → 检查“备弹 > 0 且弹匣未满且不在换弹” → `bPendingReload = true`（SkipOwner 复制给别人播动画）→ 服务器计时器到点后**一次性**计算 `take = min(弹匣空位, 备弹)` 再加减 → 状态复位。换弹期间开火被拒；切枪或死亡要取消计时器且不改弹药。本机立即播换弹蒙太奇，不等服务器。

面试注意：这个 Demo 原来的实现，面试官最可能抓住的五处

- **伤害在每一端都算了一遍**：子弹是复制 Actor，它的 `ReceiveHit` 在服务器和每个客户端都会触发，里面直接 `ApplyDamage`，客户端会本地扣僵尸血、本地触发死亡（第 1、4 章）。
- **开火表现要等服务器多播回来**：按下后约一个 RTT 才看到枪口动画；多播还排在弹药检查之前，打空了也会“空放”动画（E1）。
- **换弹会吞子弹**：备弹不够补满时，“剩余子弹计算”函数的 false 分支没有连到返回节点，返回 0，弹匣被清空（E4：换弹前 3/7，换完 0/0）。
- **冲刺 / 瞄准两端不一致**：冲刺标志只在服务器、瞄准标志只在本机，MaxWalkSpeed 两端不同，持续被服务器拉回（E6）。
- **大量“本地当全局”**：`GetPlayerController(0)`、UI 里的倒计时、客户端的 `SetGamePaused`、`OpenLevel` 重开——单机写法在联机下全部会错到别人身上（第 9、13 章）。

面试时怎么介绍这个 Demo

推荐口径（约 1 分钟）

“这是我早期的一个 UE 5.5 蓝图 Demo，先做成单机打僵尸，后来改成双人 Listen Server 合作。当时对 UE 网络不熟，基本是‘哪里不同步就补一个 Multicast’，结果出现了开火延迟、伤害在客户端被重复计算、换弹吞子弹、冲刺被拉回这些问题。后来我把它系统复盘了一遍：明确了服务器权威和 Owner 链，开火改成客户端预测 + 服务器校验 + 计数器复制，换弹改成服务器状态机，冲刺放进 CharacterMovement 的预测里，并在 200 ms / 10% 丢包下做了对比测试。”

要点：**先承认问题、再讲原理、最后给数据。**面试官问“你的 Demo”时真正想考的是你能不能看出问题，而不是 Demo 本身做得多完整。

本书怎么读

第一部分 • 复盘原 Demo

第 1 章：28 个蓝图逐节点读出来后的真实运行方式与 23 条问题；第 2 章：只讲本 Demo 用得到的 UE 网络基础。

第二部分 • 面试四问

第 3 章绑定；第 4 章开火到击中；第 5 章子弹同步与换弹；第 6 章弱网优化。每章：30 秒答案 → 原

理 → 原实现对照 → 正确实现 → 实测 → 坑位 → 自测。

第三部分 • 重做：合格的联机 Demo

第 7 章 Gameplay 框架（World / Controller / Player BP 的 OOP 关系、生命周期、启动时序）；第 8 章合格标准、类清单、复制总表与接收矩阵、RPC 与组播总表、执行端速查；第 9~13 章逐类蓝图事件图表（框架层、玩家层、武器、僵尸 AI、UI）；第 14 章测试、调试与演示。

第四部分 • 面试防守

第 15 章 12 条追问链与危险说法对照；第 16 章 56 道面试题与速记卡。

附录 A • 问题全清单与审计方法

每条问题对应的蓝图节点编号、后果和修复；没有 UE 编辑器时如何把 .uasset 读成伪代码。

关于证据等级与实测

原工程：`github.com/dxHWH/-demo`，快照 commit `d7e5c75`，UE 5.5，纯蓝图（没有 Source 目录）。仓库里大量美术资产与逻辑无关，只拉取了 `Content/Blueprints`、`Content/UI`、主地图与 `Config`。蓝图用自写的解析脚本 `tools/bpdump.py` 从 .uasset 读出节点、引脚连线、RPC 标志和复制设置，导出在 `audit/*.txt`，书中节点编号（如 `#150`）都能在里面查到。

书中用三种标签区分可信度：`源码事实` 蓝图里确实这么连的；`推断` 根据 UE 运行规则推出的运行时后果（没有在编辑器里实跑，附录 A 给出验证方法）；`实测 Ex` 来自 `demos/coop_net_demo.cpp` 的离散事件模拟——它不跑 UE，而是按 UE 的默认参数（服务器网络 Tick 30 Hz、Reliable 有序重传、`MAXPOSITIONERRORSQUARED=3` 等）复现与本 Demo 相关的网络行为，用来比较方案的**相对**好坏，不代表某台机器上的绝对值。

PART ONE

第一部分　复盘原 Demo

先弄清它实际上是怎么跑的，再谈怎么改。面试里最有说服力的是“我读过自己的代码，知道哪一行错”。

CHAPTER 01

原 Demo 审计：它实际是怎么跑起来的

28 个蓝图、19 个 RPC、11 个复制变量。把每张图按执行线读成伪代码之后，单机改联机的典型错误几乎全在这里。

本章目标

- 说清原 Demo 的对象结构、开局流程、开火 / 换弹 / 伤害链路分别在哪一端执行。
- 按严重程度列出问题，每条都能指到具体蓝图节点。
- 记住单机改联机最常见的四类错误：**本地当全局、表现等服务器、结果在客户端算、两端各算各的。**

Fact Sheet：工程到底有什么

项目	事实	证据
引擎 / 语言	UE 5.5，纯蓝图，没有 C++ 模块	`FPS_Start.uproject`：EngineAssociation 5.5，无 Modules 段
联机方式	只在 PIE 里以 Listen Server 跑过；没有创建 / 查找会话的节点	全工程搜不到 Session 节点；`GM_MyGame` 里有两个 `servertravel CoffeeShopA_P` 节点，但都没有连到任何事件
默认地图 / GameMode	`CoffeeShopA_P`，WorldSettings 覆盖为 `GM_MyGame`（GameModeBase）	DefaultPawn = `BP_Player`，GameState = `BP_MyGameState`；没有自定义 PlayerController / PlayerState
角色	`BP_Player`（Character），唯一的网格是第一人称手臂 `SK_Custom_Arms`	CDO：`bUseControllerRotationPitch = True`，相机挂在手臂上的 SpringArm（臂长 0）
武器 / 子弹	`BP_Weapon`（Actor，bReplicates）每人一把；`BP_Bullet`（Actor，bReplicates + bReplicateMovement，ProjectileMovement 20000 cm/s，寿命 1 s）	弹匣 10、备弹 30、射速 0.12 s（循环计时器）
僵尸	`BP_Zombie`（Character，AutoPossessAI），血量 100；爆头 100、身体 40	开局在 4 个 TargetPoint 附近刷 10 只，死一只补一只
RPC	19 个：BP_Player 7、BP_Weapon 6、BP_Zombie 3、GameMode 1、关卡蓝图 2	CustomEvent 的 FunctionFlags（`audit/*.txt` “Custom events / RPC” 段）
复制变量	11 个：玩家 HP / WeaponRef / MouseY；武器 5 个弹药相关；僵尸 血量 / 正在攻击；GameState ReadyPlayersList	除 `OnRep_WeaponRef` 外，所有 OnRep 函数都是空的

实际的对象与调用关系

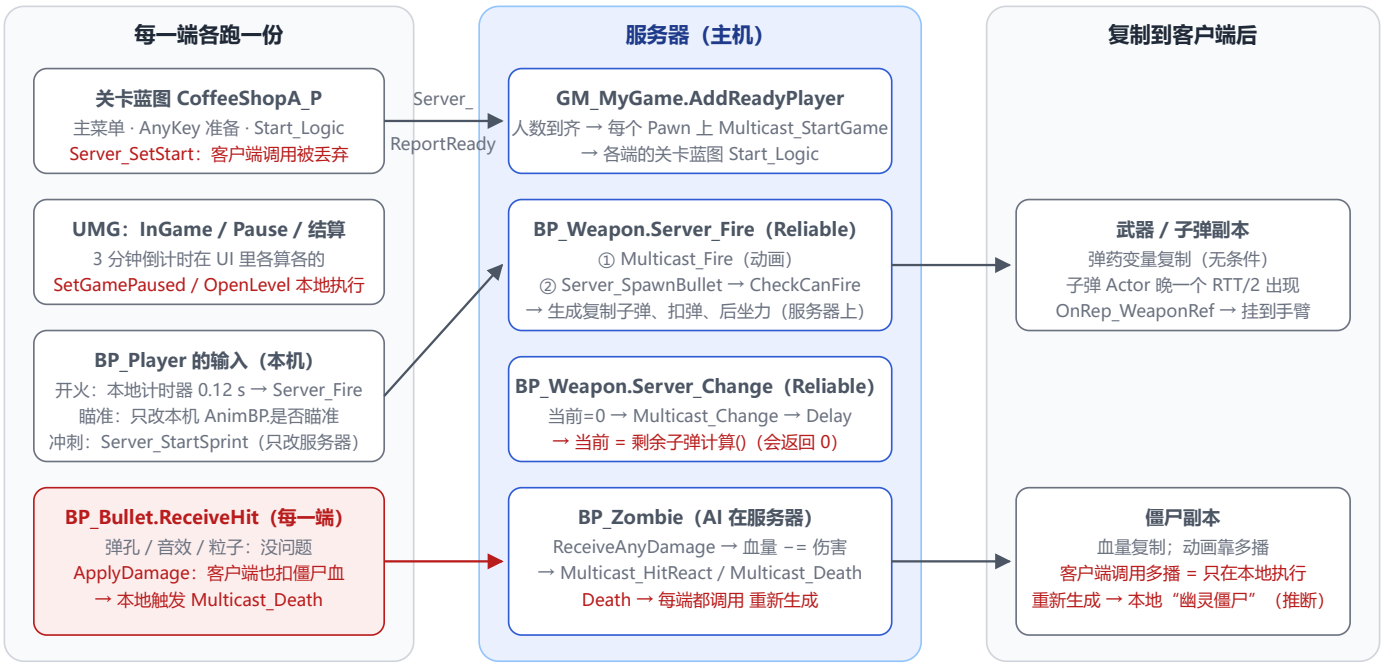


图 1-1 原 Demo 的实际运行结构（红色 = 联机下会出错的地方）。左列每台机器各执行一份，中列只有服务器执行，右列是复制到客户端后的副本

三段关键蓝图，读成伪代码

下面是用 `tools/bpdump.py` 从 .uasset 读出的执行线（`#数字` 是节点在包里的导出序号，方便回查）。注意每段开头的事件是在**哪一端**触发的。

BP_Weapon · 射击（本机计时器 → 服务器）

```
CustomEvent 开始射击()                          // 本机：IA_Fire Started → WeaponRef.开始射击
  Branch(正在换弹) [false]:
    SetTimer("射击中", 0.12, 循环) ; 射击中()
CustomEvent 射击中()                            // 本机，每 0.12 s 一次
  Branch(当前子弹数量 ≤ 0)
    [true]:  PlaySound2D(空枪声) ; ClearTimer("射击中")
    [false]: Server_Fire()                       // Reliable Server RPC，一发一条
CustomEvent Server_Fire [Server, Reliable]     // 服务器
  Multicast_Fire()                               // ① 先多播开火动画（Unreliable）
  Server_SpawnBullet()                           // ② 再判断能不能开火
CustomEvent Server_SpawnBullet [Server]
  Branch(CheckCanFire()) [true]:                 // CheckCanFire = 当前子弹数量 > 0
    SpawnActor(BP_Bullet, 枪口插槽, Owner=∅, Instigator=∅)   #150
    当前子弹数量 -= 1
    后坐力计算()                                  // Timeline → GetOwner().Instigator.AddControll
```

BP_Bullet · ReceiveHit（每一端都会触发）

```
Event ReceiveHit(Other, OtherComp, Hit)                  // 复制子弹在服务器和每个客户端各模拟一份
  SpawnDecalAttached(弹孔 或 血迹) ; SpawnSound / SpawnEmitter
  给丧尸伤害(Hit)
    Branch(Hit.BoneName == "head")
      [true]:  ApplyDamage(Hit.Actor, 100, Instigator=∅, Causer=∅)   #25
      [false]: ApplyDamage(Hit.Actor,  40, Instigator=∅, Causer=∅)   #24
  K2_DestroyActor()                                      // 客户端上对复制 Actor 无效，由服务器销
```

BP_Weapon · 换弹（服务器）与 剩余子弹计算

```
CustomEvent Server_Change [Server, Reliable]
  Branch(总子弹数 > 0) → Branch(弹夹最大容量 == 当前子弹数量)[false] → Branch(正在换弹)[false]:
    子弹保存 = 当前子弹数量 ; 当前子弹数量 = 0 ; 正在换弹 = true
    Multicast_Change()                                    // 播手臂 + 枪的换弹蒙太奇
    Delay(蒙太奇时长)
    正在换弹 = false
    当前子弹数量 = 剩余子弹计算()                          #192
Function 剩余子弹计算() → 子弹保存
  SwitchHasAuthority [Authority]:
    need = 弹夹最大容量 - 子弹保存
    Branch(总子弹数 - need > 0)
      [true]:  总子弹数 -= need ; 子弹保存 = 弹夹最大容量 ; Return(子弹保存)   #123
      [false]: 子弹保存 += 总子弹数 ; 总子弹数 = 0                          // ← 没有连 Return 节点
```

为什么 false 分支会返回 0

蓝图函数的输出参数在函数入口被初始化为类型默认值（int 是 0），只有执行线走到 **Return Node** 时才会把当前值写进去。false 分支执行完 `总子弹数 = 0` 就结束了，没有经过 Return，于是函数返回 0；调用方 `当前子弹数量 = 剩余子弹计算()` 把弹匣清零，算好的 `子弹保存` 就这么丢了。这和网络无关，单机下也会出现——只是联机后弹药 UI 本来就“跳”，掩盖了这个 bug。第 5 章用 E4 逐节点移植验证。

问题清单（按严重程度）

P0 = 联机下功能必错；P1 = 弱网或特定时序下出错；P2 = 设计不合理 / 浪费；P3 = 代码质量。完整 23 条及修复见附录 A，这里列面试最常被追问的 12 条。

#	级别	问题	证据	后果
1	P0	换弹在“备弹不够补满”时清空弹匣	源码事实 剩余子弹计算 false 分支无 Return（#185~#186）	实测 E4 3/7 换完变 0/0；整局打空再换只能打出 30/40 发
2	P0	伤害在每一端都算	源码事实 BP_Bullet.ReceiveHit → ApplyDamage 无权限判断（#24/#25）	推断 客户端本地扣僵尸血、本地播受击 / 死亡，和服务器不一致
3	P0	僵尸死亡时每一端都“补刷一只”	源码事实 Multicast_Death → 重新生成；重新生成直接跳进 SwitchHasAuthority 之后的 Array_Random（#15 → #9）	推断 客户端本地生成只有自己看得见的僵尸
4	P0	玩家死亡逻辑用 GetPlayerController(0)	源码事实 BP_Player.ReceiveAnyDamage（只在服务器触发）→ DisableInput(GetPlayerController(0))（#118）	推断 客户端玩家死后：目标是自己的 Pawn、传入的是主机 PC，APawn::DisableInput 只打错误日志、不生效；服务器上关输入本来也管不到客户端——死者仍能移动、开火；惨叫在主机响
5	P0	冲刺 / 瞄准状态两端不一致	源码事实 IsRunning 只由 Server RPC 设置且不复制；是否瞄准 只在本机 AnimBP 设置；Tick 里两端各自算 MaxWalkSpeed（速度计算 #233）	实测 E6 冲刺时每秒纠正 9.2 次，平均回拉 21.4 cm
6	P0	开局、计时、胜负、暂停、重开都在本地做	源码事实 UWB_InGame 计时器；SetGamePaused；OpenLevel("CoffeeShopA_P")	推断 客户端暂停无效、主机暂停全员；任意一方点重开就断开联机
7	P1	开火表现等服务器多播	源码事实 本机不播任何表现，全部在 Multicast_Fire 里	实测 E1 RTT 200 ms 时按下后 237 ms 才看到动画

8	P1	多播排在弹药检查之前	源码事实 Server_Fire：Multicast_Fire 先于 Server_SpawnBullet	实测 E1 RTT ≥ 100 ms 时每次打空多 1 次“空放”；还会打断换弹蒙太奇
9	P1	逐发 Reliable RPC	源码事实 Server_Fire 为 Reliable，射速 8.3 发/秒	实测 E2 RTT 200 / 丢包 10%：P99 处理延迟 554 ms
10	P1	每发子弹一个复制 Actor	源码事实 BP_Bullet bReplicates + bReplicateMovement	实测 E1/E7 远端出现时已飞出约 3.3 m；估算约 3767 B/s，推荐方案约 333 B/s
11	P1	客户端玩家没有后坐力、瞄准不减散布	源码事实 后坐力 Timeline 在服务器上调用 AddControllerYawInput；散布读服务器 AnimBP 的 是否瞄准	推断 AddControllerYawInput 只对本地控制器生效，客户端玩家手感与主机不同
12	P2	对方看到的是“漂浮的手臂”	源码事实 角色唯一 Mesh 是第一人称手臂，武器挂在手臂 Socket	需要 1P / 3P 两套网格（第 3 章）

面试注意：别把问题说成“UE 网络不稳定”

这些问题全部是**代码在错误的一端执行**造成的，不是引擎 bug，也不是“网络不好”。面试时把它们归成四类，显得你看懂了本质：

- **本地当全局**：GetPlayerController(0)、UI 里算倒计时、客户端 SetGamePaused、OpenLevel。
- **表现等服务器**：开火动画、后坐力都在服务器触发，本机要等一个来回。
- **结果在客户端算**：复制子弹在每一端都 ApplyDamage。
- **两端各算各的**：MaxWalkSpeed 依赖只在一端存在的标志；多播里修改复制变量（正在攻击 = false）。

它也有做对的地方

- **武器在服务器生成**（SwitchHasAuthority 之后 SpawnActor），`WeaponRef` 复制并在 `OnRep_WeaponRef` 里挂接——因为武器没开 bReplicateMovement，挂接信息（AttachmentReplication）本来就不会复制，这个 OnRep 是必要的。
- **僵尸 AI 和攻击判定只在服务器**：寻路、攻击范围检测、攻击伤害检测都套了 SwitchHasAuthority。
- **受击反馈用 Client RPC**：`Client_PlayHitFX` 只发给被打的玩家播镜头抖动、受击音效和后处理，这是 Client RPC 的标准用法。
- `for_server` 函数说明作者发现了“武器 BeginPlay 早于 SetOwner”——只是修补的方式不对（第 3 章）。

本章速记

- 原 Demo：纯蓝图、PIE Listen Server、19 个 RPC、11 个复制变量，只有 1 个 OnRep 有内容。
- 最致命的 3 条：**子弹 ReceiveHit 在每端都 ApplyDamage**、**换弹函数漏连 Return 吞子弹**、**冲刺 / 瞄准两端不一致导致拉回**。
- 四类根因：本地当全局、表现等服务器、结果在客户端算、两端各算各的。
- 回答“你的 Demo 有什么问题”时：先分类，再各举一个节点级的例子，最后说改法。

▶ **自测：复制子弹的 ReceiveHit 为什么会在客户端触发？客户端上 K2_DestroyActor 又为什么没效果？**

▶ **自测：在客户端调用一个 Multicast 事件会发生什么？**

CHAPTER 02

只讲用得到的 UE 网络基础

四个问题：这个对象在哪几端存在？这段代码在哪一端执行？谁能调用这个 RPC？这个变量会发给谁？答清楚这四个，原 Demo 的问题就都能解释。

本章目标

- 能画出 Listen Server 双人局里 GameMode / GameState / PlayerState / PC / Pawn / 武器 / UI 各在哪一端存在。
- 分清 Role（权限）和 Owner（所有权），背出 RPC 可达性表。
- 知道属性复制的四个性质：单向、只发最新值、按网络帧发、可以按条件发。

服务器权威，以及 Listen Server 的“主机也是玩家”

UE 的联机是**服务器权威**：世界的真实状态只在服务器上；客户端看到的是服务器复制过来的近似值，客户端想改变世界只能通过 Server RPC **请求**服务器去改。双人合作最常用 **Listen Server**：主机进程既是服务器又有一个本地玩家；Dedicated Server 则没有本地玩家、不渲染。

面试注意：主机玩家同时满足“有权限”和“本地控制”

很多人写 `if (HasAuthority()) { 服务器逻辑 } else { 本地表现 }`，在 Listen Server 上主机玩家永远走不到 else，于是主机看不到自己的开火表现、UI 不刷新。**“要不要播本地表现”看 `IsLocallyControlled()`，“要不要改游戏状态”看 `HasAuthority()`**，两个判断互相独立。原 Demo 用了大量 SwitchHasAuthority，而几乎没用 IsLocallyControlled。

每个对象在哪几端存在

对象	主机（服务器 + 玩家 1）	客户端（玩家 2）	放什么
GameMode	**有**	**没有**	规则：刷怪、开局条件、胜负判定、重生。客户端拿到的是 nullptr
GameState	**有**	**有（复制）**	所有人都该知道的全局状态：比赛阶段、剩余时间、波次、PlayerArray
PlayerState ×2	**两份**	**两份（复制）**	跟“人”走的数据：名字、分数、是否准备、击杀数；Pawn 死了也不丢
PlayerController	**两份**	**只有自己那份**	输入、相机、UI、Client RPC 的落点；**别人的 PC 在客户端不存在**
Pawn / Character ×2	**两份**	**两份（复制）**	身体：移动、血量、当前武器
武器 ×2	**有**	**有（复制）**	弹药、开火状态；Owner = 持有者的 Pawn
AIController / 行为树	**有**	**没有**	僵尸的决策只在服务器
HUD / UMG / 音效 / 粒子	**自己那份**	**自己那份**	纯本地表现，从复制下来的数据里读
GameInstance	**自己那份**	**自己那份**	跨关卡的本地数据，不复制

Listen Server 双人局的对象分布。原 Demo 的 ReadyPlayersList 把 PlayerController 引用复制到 GameState——客户端上别人的 PC 不存在，复制下来就是空引用

Role：这一端对这个 Actor 有多大权限

Actor	在主机上的 LocalRole	在客户端上的 LocalRole
主机玩家的角色	Authority（同时 IsLocallyControlled）	SimulatedProxy
客户端玩家的角色	Authority	**AutonomousProxy**（本地控制、做移动预测）
客户端玩家的武器	Authority	SimulatedProxy（但 Owner 链属于这个客户端）
僵尸	Authority	SimulatedProxy
客户端自己 Spawn 的 Actor	不存在	Authority（只在这台机器上）

最后一行就是原 Demo“幽灵僵尸”的来源：客户端执行到 SpawnActor，生成的 Actor 只存在于本地，而且在本地是 Authority，后面的 SwitchHasAuthority 也挡不住它（第 12 章）。

Owner：谁能调用 Server RPC

每个 Actor 可以有一个 Owner，Owner 的 Owner……一路往上，如果最终是某个 `PlayerController`，这个 Actor 就“属于”那个 PC 背后的**网络连接**。Possess 时 Pawn 的 Owner 被设成 PC；武器的 Owner 设成 Pawn，就挂在同一条链上。

RPC 类型	从服务器调用	从拥有该 Actor 的客户端调用	从其他客户端调用
Server	在服务器本地执行	**发到服务器执行**	**丢弃（日志：No owning connection）**
Client	发到拥有者客户端执行；没有拥有者时在服务器执行	在本地执行	在本地执行
NetMulticast	服务器 + 所有相关的客户端执行	**只在本地执行**	**只在本地执行**

RPC 可达性（与 Role 无关，只看调用端和 Owner 连接）。原 Demo：关卡蓝图的 Server_SetStart 由客户端调用 → 关卡脚本 Actor 不属于任何客户端连接 → 被丢弃；客户端本地扣血后调用 Multicast_Death → 只在本地执行

Reliable 与 Unreliable

- **Reliable**：保证到达、按顺序执行。丢包时后面的 Reliable 要等前面的重传成功（队头阻塞）；发得太快会把可靠缓冲区塞满，引擎会直接断开连接。适合低频且必须到达的事：开始 / 停止开火、换弹请求、准备就绪。
- **Unreliable**：可能丢、可能被合并，Multicast 还会受相关性影响（不相关的客户端收不到）。适合高频且“丢一个无所谓”的事：开火特效、脚步声。
- 两者共同的问题：**RPC 只对当时在线且相关的客户端生效，晚加入的人收不到。**持续存在的状态（是否死亡、是否在换弹、比赛阶段）必须用复制属性表示。

属性复制的四个性质

1. **单向**：只从服务器到客户端。客户端改复制变量只改了自己的副本；服务器不会因此重发，直到服务器端的值再次变化（原 Demo 在客户端写 `MouseY`、在多播里写 `正在攻击 = false`，都是这种“只改了本地”）。
2. **只发最新值**：服务器在一个网络帧里把值从 10 改成 9 再改成 8，客户端只会收到 8。所以“状态”适合复制，“事件”不适合——要用复制属性表达事件，就用一个只增不减的计数器（如 BurstCounter），客户端在 OnRep 里看增量。
3. **按网络帧发**：服务器网络 Tick 默认 30 Hz（`NetServerMaxTickRate`），每个 Actor 再按 `NetUpdateFrequency` 决定多久检查一次。本机改了变量，最快也要等到下一个网络帧才发出去（E1 中 RTT=0 时仍有约 20 ms 的反馈延迟，就是在等网络帧）。
4. **可以按条件发**：`COND_OwnerOnly`（只发给拥有者，适合弹药）、`COND_SkipOwner`（除了拥有者都发，适合拥有者已经本地预测过的表现计数器）、`COND_InitialOnly`、`COND_SimulatedOnly` 等。

RepNotify：蓝图和 C++ 不一样

C++ 的 `ReplicatedUsing=OnRep_X` 只在**客户端收到新值**时调用，服务器改值不会调用，需要的话手动调一次。蓝图的 RepNotify 变量在服务器上用 Set 节点赋值时，服务器自己也会调用 OnRep 函数。从蓝图迁到 C++ 时常因为这点导致主机玩家的 UI 不刷新。

一条总规则：状态用属性，瞬时事件用 RPC，表现各端本地做

要同步的东西	推荐方式	原 Demo 的做法
血量、是否死亡、是否在换弹、当前武器	复制属性 + OnRep	血量复制但 OnRep 为空；死亡、换弹动画靠 Multicast（晚加入看不到）
开火（另一端要看到）	复制计数器 BurstCounter（SkipOwner）或 Unreliable Multicast	Unreliable Multicast，但排在弹药检查之前
客户端的操作请求	Server RPC（低频用 Reliable）	每发一条 Reliable
只给某个玩家的反馈	Client RPC	Client_PlayHitFX ✓
移动速度相关的状态	放进 CharacterMovement 的预测（SavedMove）	Server RPC 改一个不复制的布尔
弹孔、音效、粒子、UI	各端本地，读复制下来的数据	弹孔 ✓；UI 每帧 Cast GetPlayerPawn(0)

本章速记

- GameMode 只在服务器；别人的 PlayerController 在客户端不存在；跟人走的数据放 PlayerState。
- **Role 决定能不能改状态，Owner 决定能不能调 Server RPC**；能否调 Server RPC 不看 Role。
- 客户端调用 Multicast 只在本地执行；客户端调用不属于自己的 Actor 的 Server RPC 会被丢弃。
- 复制属性：单向、只发最新值、按网络帧发、可按条件发；事件用计数器表达。
- 主机玩家既 HasAuthority 又 IsLocallyControlled，两个判断要分开用。

▶ **自测：客户端玩家的武器在客户端上是 SimulatedProxy，为什么还能在武器上调用 Server RPC?**

▶ **自测：为什么“死亡”不能只用一个 Reliable Multicast 通知?**

PART TWO

第二部分　面试四问

先讲绑定（它决定了谁能发什么 RPC），再讲开火到命中、子弹与换弹，最后讲弱网。每章都对照原 Demo 的实现。

CHAPTER 03

问 ②：角色、控制器、武器应该怎么绑定

绑定不是“把枪挂到手上”这么简单：它决定了 Owner 链、谁能发 Server RPC、COND_OwnerOnly 发给谁、输入在哪台机器上生效。

本章目标

- 讲清 Possess 的完整过程：服务器做什么、客户端什么时候知道“这是我的角色”。
- 武器的生成时机、Owner / Instigator、1P / 3P 挂接、CurrentWeapon 的复制。
- 输入映射、UI、相机这些“只属于本地玩家”的东西应该在哪里初始化。

30 秒版

服务器上 GameMode 为每个 PC 生成 Pawn 并 Possess，Possess 让 Pawn 的 Owner = PC、在拥有者客户端上成为 AutonomousProxy；武器在服务器生成，`SpawnParams.Owner = Pawn`、`Instigator = Pawn`，挂到 3P 模型 Socket，本地再挂一份 1P；`CurrentWeapon` 复制，在 OnRep 里挂接。输入在拥有者客户端“拿到控制权”时绑定；分数和准备状态放 PlayerState。

Possess 到底发生了什么

步骤	在哪一端	发生了什么	适合挂什么逻辑
1. PostLogin / HandleStartingNewPlayer	服务器	新连接有了 PlayerController 和 PlayerState	记录玩家、初始化 PlayerState
2. RestartPlayer → SpawnDefaultPawnFor	服务器	在 PlayerStart 生成 Pawn（DefaultPawnClass）	选出生点
3. `PC→Possess(Pawn)` → `Pawn→PossessedBy(PC)`	服务器	SetOwner(PC)；Pawn 在该连接上成为 AutonomousProxy；Pawn->PlayerState 指向 PC 的 PlayerState	**生成默认武器**、初始化服务器端属性
4. `ClientRestart`（Client RPC）	服务器 → 拥有者	通知客户端“你现在控制这个 Pawn”	—
5. AcknowledgePossession / `PawnClientRestart`	拥有者客户端	建立 InputComponent、调用 `SetupPlayerInputComponent`；相机切到这个 Pawn	**添加输入映射**、创建本地 UI
6. `OnRep_Controller` / `OnRep_PlayerState`	客户端	Controller、PlayerState 引用复制到达	依赖 PlayerState 的显示（名字、队伍色）

关键结论：**BeginPlay 不是“我拿到控制权”的时机。**客户端上 Pawn 的 BeginPlay 可能早于 Controller 复制到达，此时 `IsLocallyControlled()` 还是 false；而且 BeginPlay 会在每一端、每个 Pawn 副本上都执行一次。原 Demo 在 BP_Player 的 BeginPlay 里用 `GetPlayerController(0)` 加输入映射——两个 Pawn 副本各执行一次，加的都是本地玩家的映射，只是恰好没出错。

Owner 链：一张图看懂

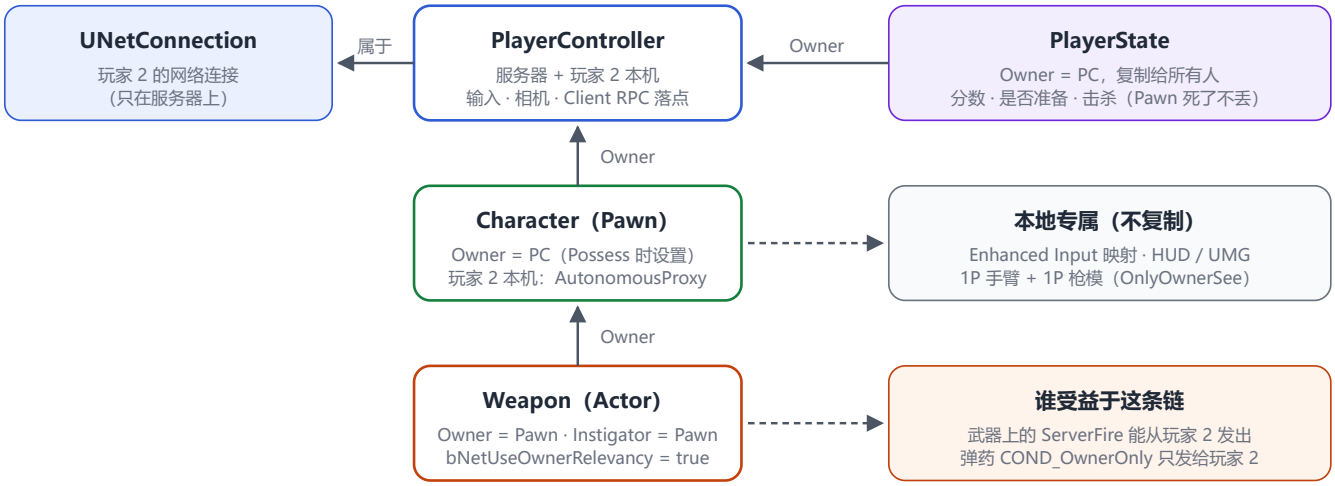


图 3-1　Owner 链：武器 → Pawn → PlayerController → 网络连接。实线箭头指向 Owner；有了这条链，武器上的 Server RPC 才能从玩家 2 发出，OwnerOnly 属性才知道发给谁

武器：在哪生成、怎么挂、怎么复制

1. 在服务器生成，生成时就带上 Owner

原 Demo 是 `SpawnActor(Owner=∅)` → `SetOwner(self)`。问题在于 **SpawnActor 内部就会调用 BeginPlay**，武器 BeginPlay 里 `GetOwner()` 还是空的，于是作者又补了一个 `for_server` 函数再 Cast 一次 Owner。正确做法是把 Owner 放进生成参数；需要在 BeginPlay 前设置更多字段时用 `SpawnActorDeferred` + `FinishSpawning`。

ACoopCharacter：在服务器 PossessedBy 时生成默认武器

```
// CoopCharacter.h
UPROPERTY(ReplicatedUsing = OnRep_CurrentWeapon) TObjectPtr<ACoopWeapon> CurrentWeapon;
UPROPERTY(EditDefaultsOnly) TSubclassOf<ACoopWeapon> DefaultWeaponClass;
UPROPERTY(VisibleAnywhere) TObjectPtr<USkeletalMeshComponent> Mesh1P;   // 第一人称手臂
UFUNCTION() void OnRep_CurrentWeapon(ACoopWeapon* OldWeapon);

// CoopCharacter.cpp
ACoopCharacter::ACoopCharacter() {
    Mesh1P = CreateDefaultSubobject<USkeletalMeshComponent>(TEXT("Mesh1P"));
    Mesh1P->SetupAttachment(FirstPersonCamera);
    Mesh1P->SetOnlyOwnerSee(true);            // 只有自己看得见手臂
    GetMesh()->SetOwnerNoSee(true);           // 自己看不见自己的第三人称身体
    bUseControllerRotationYaw = true;         // 身体只跟随 Yaw
    bUseControllerRotationPitch = false;      // 俯仰交给相机 / 手臂 / AimOffset（原 Demo 是 tru
}

void ACoopCharacter::PossessedBy(AController* NewController) {
    Super::PossessedBy(NewController);        // 服务器：SetOwner(NewController) 在这里完成
    if (!CurrentWeapon && DefaultWeaponClass) {
        FActorSpawnParameters P;
        P.Owner = this;                       // Owner 链：Weapon → Pawn → PC → Connection
        P.Instigator = this;                  // 伤害统计、友伤判断都靠 Instigator
        P.SpawnCollisionHandlingOverride = ESpawnActorCollisionHandlingMethod::AlwaysSpawn;
        ACoopWeapon* W = GetWorld()->SpawnActor<ACoopWeapon>(DefaultWeaponClass, GetActorTrans
        EquipWeapon(W);                       // 服务器上直接装备
    }
}

void ACoopCharacter::EquipWeapon(ACoopWeapon* W) {
    check(HasAuthority());
    ACoopWeapon* Old = CurrentWeapon;
    CurrentWeapon = W;
    OnRep_CurrentWeapon(Old);                 // C++ 的 OnRep 服务器不会自动调用，手动调一次
}

void ACoopCharacter::OnRep_CurrentWeapon(ACoopWeapon* OldWeapon) {
    if (OldWeapon) OldWeapon->OnUnequipped();
    if (CurrentWeapon) CurrentWeapon->OnEquipped(this);   // 内部：3P 枪挂 GetMesh() 的 hand_r
                                                          //       1P 枪挂 Mesh1P 的 GripPoint
}

void ACoopCharacter::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& Out) const {
    Super::GetLifetimeReplicatedProps(Out);
```

```
    DOREPLIFETIME(ACoopCharacter, CurrentWeapon);
}
```

2. 两套模型：自己看手臂，别人看全身

单机 FPS 只需要一双手臂；联机后对方看到的是一个人。标准做法（ShooterGame、Lyra 都是这样）：角色有 1P 手臂（`OnlyOwnerSee`）和 3P 全身（`OwnerNoSee`）两个网格，武器也有 1P / 3P 两个网格分别挂上去；开火、换弹蒙太奇在本机播 1P，在别人那里播 3P。原 Demo 只有手臂，另一端看到的是漂浮的手臂和枪，而且因为 `bUseControllerRotationPitch = true`，整个胶囊体会随低头抬头旋转。远端角色的俯仰用 `GetBaseAimRotation()`（内部读复制的 `RemoteViewPitch`）驱动 AimOffset。

3. 挂接信息靠 OnRep，不靠“挂接复制”

Actor 的挂接（AttachmentReplication）只在开启 `bReplicateMovement` 时随移动一起复制。武器一般不需要复制移动，所以要像原 Demo 的 `OnRep_WeaponRef` 那样，在 `CurrentWeapon` 的 OnRep 里各端自己挂——这一点原 Demo 是对的。武器再设 `bNetUseOwnerRelevancy = true`，让它跟随持有者决定是否相关。

只属于本地玩家的东西：输入、UI、相机

在“成为本地控制者”时添加输入映射（UE 5.x 模板写法）

```
void ACoopCharacter::NotifyControllerChanged() {
    Super::NotifyControllerChanged();
    if (APlayerController* PC = Cast<APlayerController>(Controller)) {
        if (PC->IsLocalController()) {                          // 服务器上的远端 PC 不是本地的，跳
            if (auto* Sub = ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(PC-
                Sub->AddMappingContext(DefaultMappingContext, 0);
        }
    }
}
// SetupPlayerInputComponent 只会在本地控制的 Pawn 上调用，在这里 BindAction 即可。
// HUD / 准星 / 弹药 UI：放在 PlayerController::BeginPlay（IsLocalController 时）或 AHUD 里创建，
// 通过 Pawn / PlayerState 的委托刷新，而不是 UMG 绑定函数每帧 Cast GetPlayerPawn(0)。
```

面试注意：GetPlayerController(0) / GetPlayerCharacter(0) 在联机里是什么

它返回的是**这台机器的第一个本地玩家**的控制器 / 角色。在服务器上执行时，它是主机玩家，而不是“当前这个 Pawn 的控制器”。原 Demo 的玩家死亡逻辑在服务器上对死者调用 `DisableInput(GetPlayerController(0))`：客户端玩家死时，传入的是主机的 PC，`APawn::DisableInput` 发现不是自己的 Controller，只打一条错误日志、什么也不做（第 10 章有完整分析）；同一段里的 `PlaySound2D`（死亡惨叫）在主机响。正确做法：服务器只改状态（bIsDead），并在每个 Server RPC 里校验存活；“关输入、播惨叫”放到拥有者客户端的 OnRep 里做——**输入只能在本地关。**

原实现 vs 正确实现

环节	原 Demo	正确做法
武器生成	BeginPlay + SwitchHasAuthority，Owner=∅ 后 SetOwner，再用 for_server 补取 Owner	PossessedBy（服务器）里生成，SpawnParams 带 Owner / Instigator
挂接	挂到 1P 手臂 R_GunSocket，OnRep_WeaponRef 里再挂一次 ✓	3P 枪挂全身模型，1P 枪挂手臂；OnRep_CurrentWeapon 里各端挂
模型	只有 1P 手臂；胶囊体随 Pitch 旋转	1P（OnlyOwnerSee）+ 3P（OwnerNoSee）；身体只跟 Yaw，AimOffset 处理俯仰
输入映射	BeginPlay 里 GetPlayerController(0)	NotifyControllerChanged / PawnClientRestart，且 IsLocalController
武器拿角色	BeginPlay Cast GetOwner 缓存 As BP Player	OnEquipped(Character) 时传入；或直接 `GetOwner<ACoopCharacter>()`
跟人走的数据	准备列表放 GameState（存 PC 引用）	PlayerState：bIsReady、Score、Kills
死亡	服务器上 DisableInput(GetPlayerController(0))	服务器：改 bIsDead、校验后续请求、通知 GameMode；拥有者客户端：OnRep 里关本地输入

本章速记

- Possess（服务器）→ Owner=PC、AutonomousProxy → ClientRestart → 客户端 PawnClientRestart 绑输入。
- 武器：**服务器生成、生成时带 Owner / Instigator**、CurrentWeapon 复制 + OnRep 挂接、bNetUseOwnerRelevancy。
- 1P 手臂 OnlyOwnerSee + 3P 全身 OwnerNoSee；身体跟 Yaw，俯仰靠 RemoteViewPitch + AimOffset。
- GetPlayerController(0) = 本机第一个本地玩家，服务器上写它就是在操作主机。

▶ **自测：武器的 Owner 不设（为空），会出现哪些现象?**

▶ **自测：Possess 之后客户端上 IsLocallyControlled 什么时候变成 true?**

CHAPTER 04

问 ①：从开火到击中僵尸，发生了什么

把一发子弹拆成“本机表现、服务器裁决、各端表现”三段：本机不等服务器，结果只认服务器，别人看到的是服务器复制的结果。

本章目标

- 按时间顺序讲清一发子弹在三台“视角”（本机 / 服务器 / 另一端）上各发生了什么。
- 对比原 Demo 的时序：反馈延迟、空放、子弹突现从哪来（E1）。
- 写出服务器校验、伤害、僵尸受击与死亡的正确实现。

原 Demo 的一次连射，在时间轴上长什么样

下面是 E1 的一次模拟：RTT 200 ms、按住开火 1.5 s、弹匣 10 发。按原蓝图的逻辑：本机计时器每 0.12 s 发一条 `Server_Fire`，服务器先多播开火动画，再检查弹药、生成复制子弹；弹药变量在下一个网络帧复制回来。

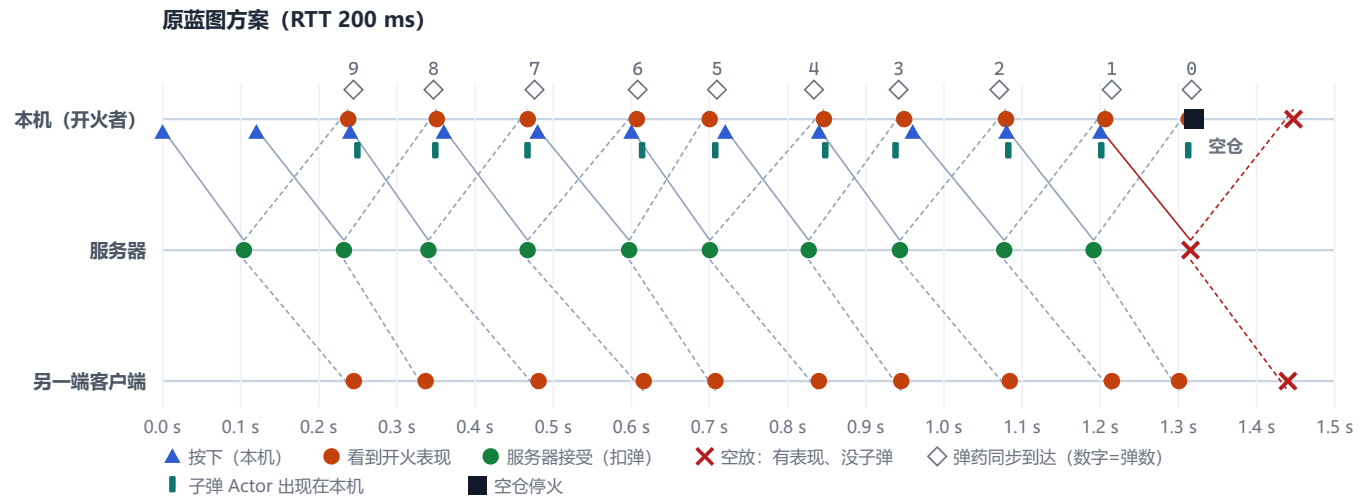


图 4-1　原 Demo 的开火时序（E1，RTT 200 ms）。本机按下（蓝三角）后要等约一个 RTT 才看到自己的开火动画（橙点）；本机的弹药显示（菱形）也晚一个 RTT，于是打空后还多发了一次请求，服务器照样多播了一次“空放”（红叉）

三个现象都能从这张图读出来：

- **反馈延迟**：本机看到自己开火的时间 = 上行半个 RTT + 等服务器网络帧 + 下行半个 RTT。RTT 200 ms 时平均 237 ms；就算 RTT 为 0，也有约 20 ms（等 30 Hz 网络帧）。
- **空放**：本机判断“还有子弹”用的是复制下来的旧值，最后一发之后还会再发一次请求；服务器的 `Server_Fire` 先调 `Multicast_Fire` 再检查弹药，于是两端都播了一次没有子弹的开火动画。这次多播还带着 `bStopAllMontages = true`，如果玩家此时在换弹，换弹动画会被打断。
- **子弹突现**：子弹是服务器生成的复制 Actor，到达本机时它已经飞了“一个网络帧 + 下行时间”，平均约 3.3 m——玩家看到子弹凭空出现在枪口前几米处。

方案	RTT ms	本机看到自己开火 ms	另一端看到 ms	本机空放次数	另一端空放	子弹突现距离 m
原蓝图方案	0	20.0	20.0	0.0	0.0	4.00
原蓝图方案	50	86.5	86.5	0.0	0.0	3.24
原蓝图方案	100	136.2	136.2	1.0	1.0	3.17
原蓝图方案	200	236.9	236.9	1.0	1.0	3.30
预测+直接覆盖	0	0.0	20.0	0.0	0.0	0.00
预测+直接覆盖	50	0.0	84.7	0.0	0.0	0.00
预测+直接覆盖	100	0.0	134.4	0.8	0.0	0.00
预测+直接覆盖	200	0.0	235.0	1.0	0.0	0.00

预测 + 序号对账	0	0.0	20.0	0.0	0.0	0.00
预测 + 序号对账	50	0.0	84.7	0.0	0.0	0.00
预测 + 序号对账	100	0.0	134.4	0.0	0.0	0.00
预测 + 序号对账	200	0.0	235.0	0.0	0.0	0.00

表 4-1　E1：一次按住 1.5 s 的连射（弹匣 10 发、0.12 s/发，抖动 20 ms，5 个随机种子平均）。“预测”两行的另一端延迟和原方案相同——**预测只能消除自己的延迟，消除不了别人看到你的延迟**

正确的流程：三段式

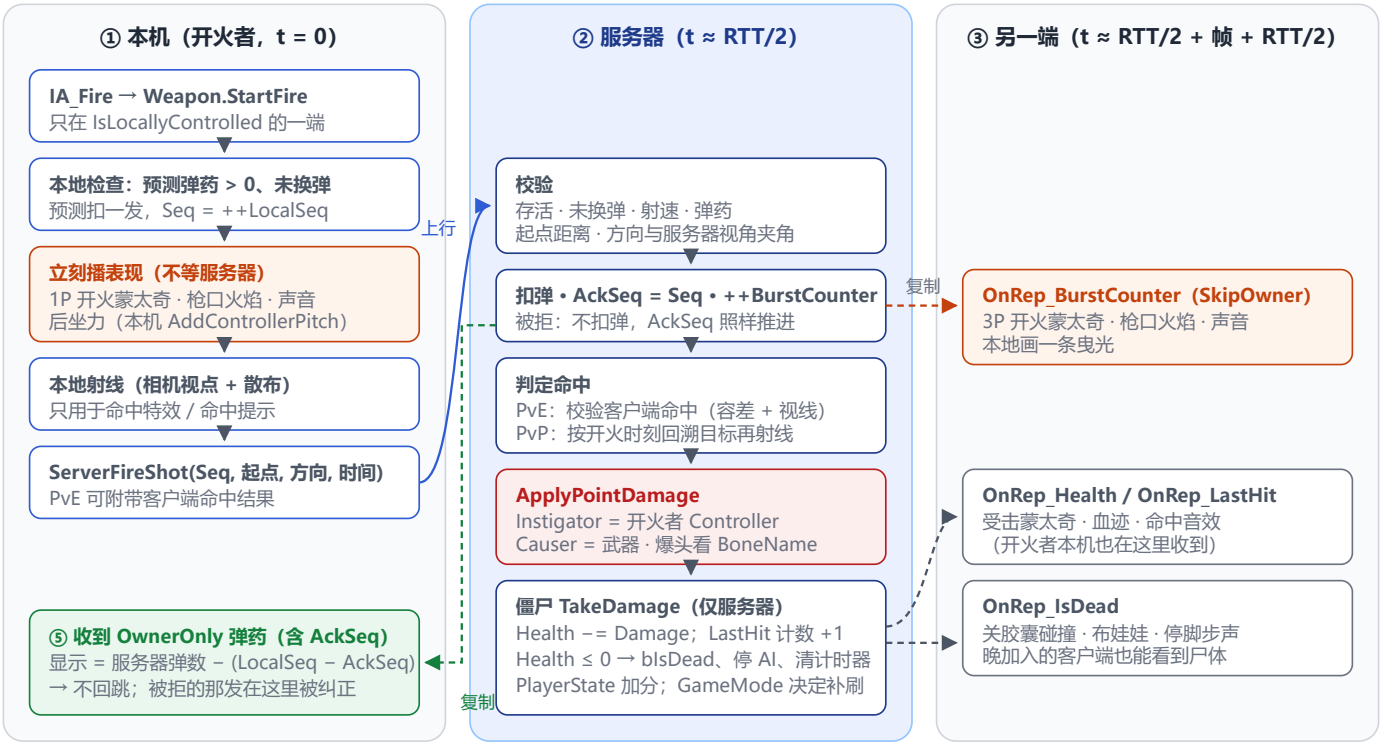


图 4-2 推荐的开火流程。橙色是表现（各端本地播放），红色是结果（只在服务器发生）；虚线是属性复制，在服务器下一个网络帧发出

第一段：本机（只做表现和预测）

ACoopWeapon：本机开火（拥有者客户端 + 主机玩家都走这里）

```
void ACoopWeapon::StartFire() {                          // 由本地输入调用，只在 IsLocallyControlle
    if (bWantsToFire) return;
    bWantsToFire = true;
    FireShot();
    GetWorldTimerManager().SetTimer(RefireTimer, this, &ACoopWeapon::FireShot, FireInterval, t
}

void ACoopWeapon::FireShot() {
    if (!bWantsToFire || IsReloadingLocal()) return;
    if (GetDisplayedAmmo() ≤ 0) { StopFire(); PlayDryFireLocal(); TryAutoReload(); return; }

    const uint16 Seq = ++LocalShotSeq;                   // 预测：显示弹数 = 服务器弹数 - (LocalSho
    FVector Start, Dir;
    GetAim(Start, Dir, /*SpreadSeed*/ Seq);              // 相机视点 + 散布；用 Seq 做种子，服务器能
    PlayFireCosmetics(/*bFirstPerson*/ true);            // 1P 蒙太奇、枪口火焰、声音
    ApplyRecoilLocal();                                  // 后坐力必须在本地控制者上做（原 Demo 在服

    FHitResult Hit;
    const bool bHit = TraceShot(Start, Dir, Hit);        // 本地射线：只用于命中特效 / 准星命中提示
    if (bHit) PlayImpactCosmetics(Hit);

    FShotRequest R;
    R.Seq = Seq; R.Start = Start; R.Dir = Dir;
    R.ClientTime = GetWorld()->GetGameState()->GetServerWorldTimeSeconds();
    R.HitActor = bHit ? Hit.GetActor() : nullptr;        // PvE：把客户端命中结果一起发上去，服务器做
    R.HitBone = bHit ? Hit.BoneName : NAME_None;
    if (HasAuthority()) ProcessShot(R);                  // 主机玩家：直接在服务器处理
    else ServerFireShot(R);                              // Reliable：低频（8 发/秒）且会造成伤害，
}
```

第二段：服务器（校验、扣弹、判定、伤害）

ACoopWeapon：服务器处理一发

```
void ACoopWeapon::ServerFireShot_Implementation(const FShotRequest& R) { ProcessShot(R); }

void ACoopWeapon::ProcessShot(const FShotRequest& R) {
    check(HasAuthority());
    Ammo.AckSeq = R.Seq;                                    // 无论接不接受都推进 AckSeq，本机据此纠正
    ACoopCharacter* Shooter = GetOwner<ACoopCharacter>();
    const float Now = GetWorld()->GetTimeSeconds();
    const bool bOk = Shooter && Shooter->IsAlive() && !bPendingReload && Ammo.InClip > 0
        && FVector::Dist(R.Start, Shooter->GetPawnViewLocation()) < MaxOriginError  // 起点：不
        && FVector::DotProduct(R.Dir, Shooter->GetBaseAimRotation().Vector()) > 0.8f // 方向：
        && ConsumeShotBudget(Now);   // 射速：令牌桶，放最后（有副作用：只有其他条件都过了才扣令牌）
    if (!bOk) return;                                       // 被拒：弹数不变、AckSeq 已推进，本机显示会

    --Ammo.InClip;
    ++BurstCounter;                                         // COND_SkipOwner：另一端据此播开火表现
    if (!Shooter->IsLocallyControlled() && GetNetMode() == NM_ListenServer)
        PlayFireCosmetics(/*bFirstPerson*/ false);    // 主机看客户端开火：SkipOwner 的 OnRep 不

    FHitResult Hit;
    if (ResolveHit(R, Hit))                                 // PvE：校验 R.HitActor（包围盒 + 容差 +
        UGameplayStatics::ApplyPointDamage(Hit.GetActor(), BaseDamage, R.Dir, Hit,
            Shooter->GetController(), this, DamageTypeClass);   // Instigator / Causer 都要填
}
```

为什么这里用 Reliable，而不是原 Demo 的“逐发 Reliable 有问题”？

原 Demo 的问题不在于 Server_Fire 是 Reliable，而在于**本机的表现要等它往返一圈**，并且多播排在弹药检查之前。改成本机预测后，Reliable 的排队（E2：RTT 200 / 丢包 10% 时 P99 处理延迟 554 ms）只会推迟伤害生效，不会影响手感；而换成 Unreliable 则会在同样条件下丢掉约 10% 的命中——“明明打中了没扣血”比“晚 0.3 秒扣血”更难接受。射速很高（如 20 发/秒以上）时，再把同一帧内的多发合并成一个数组发送。

第三段：僵尸受击与死亡（只在服务器改状态，各端从 OnRep 播表现）

ACoopZombie：伤害与死亡

```
UPROPERTY(ReplicatedUsing = OnRep_Health) float Health = 100.f;
UPROPERTY(ReplicatedUsing = OnRep_LastHit) FZombieHitInfo LastHit;   // {uint8 Counter; FVecto
UPROPERTY(ReplicatedUsing = OnRep_IsDead) bool bIsDead = false;

float ACoopZombie::TakeDamage(float Damage, const FDamageEvent& Ev, AController* InstigatorPC,
    if (!HasAuthority() || bIsDead) return 0.f;                          // 原 Demo 缺的就是这一行
    bool bHead = false;
    if (Ev.IsOfType(FPointDamageEvent::ClassID)) {
        const auto& PE = static_cast<const FPointDamageEvent&>(Ev);
        bHead = PE.HitInfo.BoneName == HeadBoneName;
        LastHit.Point = PE.HitInfo.ImpactPoint;
    }
    Damage = bHead ? HeadshotDamage : Damage;                            // 原 Demo：爆头 100，身体
    Health = FMath::Max(0.f, Health - Damage);
    LastHit.bHeadshot = bHead; ++LastHit.Counter;                        // 计数器：同一网络帧多次命
    if (Health ≤ 0.f) Die(InstigatorPC);
    return Damage;
}

void ACoopZombie::Die(AController* Killer) {
    bIsDead = true;
    OnRep_IsDead();                                                      // 服务器（主机玩家）自己也
    if (AAIController* AI = Cast<AAIController>(GetController()))
        if (UBrainComponent* Brain = AI->GetBrainComponent()) Brain->StopLogic(TEXT("Dead"));
    GetWorldTimerManager().ClearAllTimersForObject(this);               // 原 Demo 清错了计时器名，
    if (Killer) if (auto* PS = Killer->GetPlayerState<ACoopPlayerState>()) PS->AddKill();
    if (auto* GM = GetWorld()->GetAuthGameMode<ACoopGameMode>()) GM->OnZombieKilled(this, Kill
    SetLifeSpan(5.f);                                                    // 服务器销毁，客户端自动跟
}

void ACoopZombie::OnRep_IsDead() {
    GetCapsuleComponent()->SetCollisionEnabled(ECollisionEnabled::NoCollision);
    GetCharacterMovement()->DisableMovement();
    GetMesh()->SetCollisionProfileName(TEXT("Ragdoll"));
    GetMesh()->SetSimulatePhysics(true);                                 // 布娃娃各端各算，属于表现
}
```

两个容易被追问的细节

- **爆头判定用的是哪套骨骼？**服务器用自己的骨骼姿势做射线。Listen Server 的主机会渲染，骨骼每帧更新；Dedicated Server 默认不刷新不可见网格的骨骼，需要把僵尸网格的 `VisibilityBasedAnimTickOption` 设为 `AlwaysTickPoseAndRefreshBones`，或者干脆用胶囊体 + 头部球体这类简化碰撞体判定。
- **受击反馈用 Multicast 还是复制属性？**两者都可以。用复制属性（如 `LastHit` 计数器）能和血量在同一次更新里到达，不会出现“血先掉、动画后播”；Unreliable Multicast 更简单，丢了也只是少播一次受击动作。**绝对不要在客户端调用它。**

改完之后的时间轴

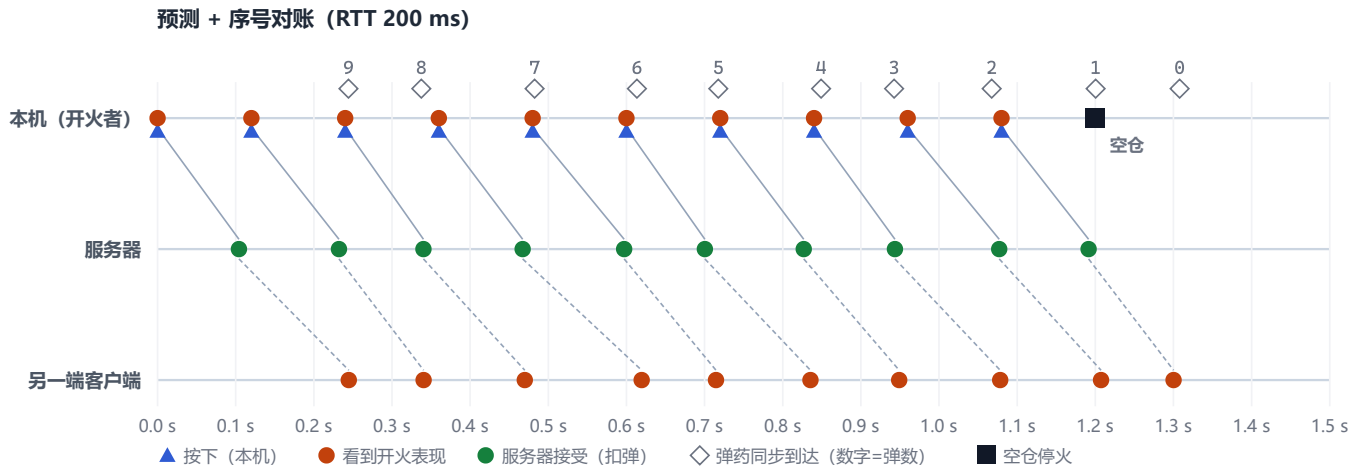
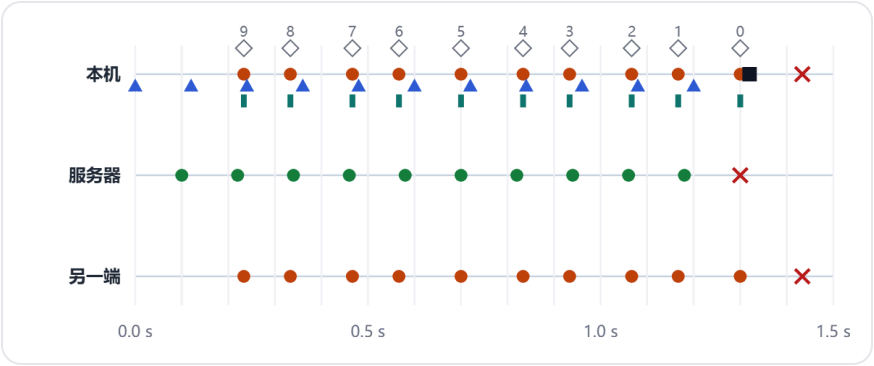


图 4-3 改造后的开火时序（E1，RTT 200 ms）。本机按下即播表现（橙点与蓝三角重合），服务器接受 10 发、没有空放；另一端靠 BurstCounter 播开火，仍然晚约一个 RTT——这部分延迟只能靠第 6 章的手段缓解，不能消除

交互实验 • 开火时序：本机 / 服务器 / 另一端



本机看到自己开火	220 ms
另一端看到	220 ms
有效子弹	10 发
空放 / 被拒	1 次
UI 回跳	0 次
子弹突现距离	4.0 m

切换方案、拖动 RTT：看蓝三角（按下）到橙点（看到开火）的距离、红叉（空放 / 被拒）和菱形（弹药复制到达）。这里不加抖动，数字与表 4-1 略有差异。

命中扫描还是抛射物？

	命中扫描（Hitscan）	服务器抛射物 + 本机假子弹	原 Demo：复制抛射物
适合	步枪、手枪（子弹飞行时间 < 一帧到几帧）	火箭筒、榴弹、慢速弓箭	—
本机手感	立即	立即（假子弹不复制，只在本机）	晚一个 RTT 才看到子弹
别人看到	BurstCounter + 本地曳光	服务器抛射物复制下来（可按 RTT/2 向前外推）	突然出现在几米外
网络开销	每发几个字节	每发一个 Actor 通道	每发一个 Actor 通道（E7）
伤害在哪算	服务器	服务器抛射物的命中	每一端都算（错误）

原 Demo 的子弹 20000 cm/s，打 20 m 外的僵尸只要 0.1 s，本质上就是命中扫描，没必要做成复制 Actor。如果一定要保留“看得见的子弹”，就在每一端本地生成不复制的曳光弹，伤害由服务器射线决定。

面试注意：这道题的高频坑

- 说“客户端做射线，打中了就调 Server RPC 扣血”——要补上**服务器校验**，否则外挂可以随便上报命中。PvE 合作游戏可以放宽，但射速、弹药、起点、方向这些必须校验。
- 说“服务器多播开火，所有人一起播动画”——开火者自己会晚一个 RTT。开火者本机预测播，别人用 SkipOwner 的计数器或多播。
- 忘了 **Listen Server 主机**：服务器上看客户端开火时，SkipOwner 属性的 OnRep 不会在服务器触发，主机需要在 ProcessShot 里自己播远端表现。
- 伤害不填 Instigator：后续的击杀统计、友伤规则、仇恨（僵尸追击打它的人）都做不了。

本章速记

- **本机**：检查预测弹药 → 扣一发、Seq+1 → 立即播 1P 表现和后坐力 → 本地射线播命中特效 → ServerFireShot(Seq, 起点, 方向, 时间)。
- **服务器**：校验（存活 / 换弹 / 射速 / 弹药 / 起点 / 方向）→ 扣弹、AckSeq、BurstCounter → 判定命中 → ApplyPointDamage(Instigator, Causer)。
- **僵尸**：TakeDamage 仅服务器；Health / LastHit / bIsDead 复制，OnRep 播受击和布娃娃；死亡时停 AI、清计时器、加分、由 GameMode 补刷。
- E1：原方案本机反馈 ≈ RTT + 网络帧；预测后本机为 0，**另一端的延迟不变。**

▶ **自测：为什么服务器拒绝一发之后，仍然要把 AckSeq 推进到这一发的序号?**

▶ **自测：原 Demo 中客户端玩家为什么感觉不到后坐力?**

CHAPTER 05

问 ④：子弹同步与换弹

“子弹同步”其实是三件事：弹药数怎么同步、别人怎么看到你开枪、子弹（命中）怎么同步。换弹则是一个必须由服务器驱动的小状态机。

本章目标

- 弹药的复制条件、预测与对账：为什么直接用 OnRep 覆盖会让 UI 回跳（E5）。
- 逐节点复现原 Demo 的换弹 bug，并给出守恒的写法（E4）。
- 换弹状态机：本机预测、服务器计时、别人看到动画、打断与取消。

30 秒版

弹药服务器权威，`COND_OwnerOnly` 只发给持枪者；本机预测扣弹，显示 = 服务器弹数 − 未确认发数，被拒的发数会自动纠正。别人看到开枪靠 `COND_SkipOwner` 的 BurstCounter，命中靠服务器的伤害结果复制。换弹：本机立刻播动画并预测状态 → `ServerStartReload` → 服务器校验后 `bPendingReload=true`（SkipOwner 给别人播动画）→ 计时器到点一次性 `take = min(空位, 备弹)` → 复位；期间开火被拒，切枪 / 死亡取消计时器且不改弹药。

原 Demo 的换弹：逐节点移植后跑一遍

把第 1 章的 `Server_Change` 与 `剩余子弹计算` 按节点一一翻译成 C++（包括“false 分支没有 Return 时返回 0”的蓝图语义），与修正版对比：

E4：原蓝图逻辑（逐节点移植）与修正版

```
// 原蓝图 BP_Weapon.剩余子弹计算（#120~#187）
int origRemain(Gun& g) {
    int need = g.maxMag - g.saved;
    if (g.total - need > 0) { g.total -= need; g.saved = g.maxMag; return g.saved; }   // #123
    g.saved = g.saved + g.total; g.total = 0;
    return 0;                    // 没经过 Return 节点：输出参数保持默认值 0
}
// 原蓝图 BP_Weapon.Server_Change（#105）
bool origReload(Gun& g) {
    if (!(g.total > 0) || g.maxMag == g.cur || g.reloading) return false;
    g.saved = g.cur; g.cur = 0; g.reloading = true;     // 一开始就把弹匣清零
    /* Multicast_Change + Delay(蒙太奇时长) */
    g.reloading = false; g.cur = origRemain(g);
    return true;
}
// 修正版：一次性计算转移量，总数守恒
bool fixedReload(Gun& g) {
    if (g.total ≤ 0 || g.cur ≥ g.maxMag || g.reloading) return false;
    int take = std::min(g.maxMag - g.cur, g.total);
    g.cur += take; g.total -= take;
    return true;
}
```

换弹前 弹匣 / 备弹	原蓝图 → 弹匣 / 备弹	原蓝图丢失	修正版 → 弹匣 / 备弹
0 / 30	10 / 20	**0**	10 / 20
3 / 30	10 / 23	**0**	10 / 23
3 / 7	0 / 0	**10**	10 / 0
3 / 5	0 / 0	**8**	8 / 0
0 / 10	0 / 0	**10**	10 / 0
0 / 4	0 / 0	**4**	4 / 0
9 / 1	0 / 0	**10**	10 / 0
10 / 20	10 / 20	**0**	10 / 20

表 5-1　E4：弹匣容量 10。只要“备弹 ≤ 需要补的数量”，原蓝图就把弹匣清零；最后一行弹匣已满，两者都不换

打法	版本	整局打出	换弹次数
打空再换	原蓝图	30 / 40 发	3
打空再换	修正版	40 / 40 发	3
剩 3 发就换	原蓝图	35 / 40 发	5
剩 3 发就换	修正版	40 / 40 发	5

表 5-2　E4：一整局（初始 10 / 30）。原蓝图在最后一次换弹时吞掉弹匣——打空再换少打 10 发，剩 3 发就换少打 5 发

面试注意：这是逻辑 bug，不是网络 bug

被问到“换弹怎么实现”时，如果拿原 Demo 举例，要主动说出这一点：它在单机里也会发生，只是联机后 UI 本来就会跳，把它掩盖了。顺带还有两个设计问题：① 换弹一开始就把 `当前子弹数量` 置 0，换弹被打断（死亡）时子弹凭空消失；② 换弹中客户端仍可能发出 `Server_Fire`（它读的 `正在换弹` 是晚一个 RTT 的复制值），服务器先多播开火动画（`bStopAllMontages = true`）把换弹动画打断，然后才发现没子弹。

弹药怎么复制：条件、预测、对账

复制条件

原 Demo 的 5 个弹药变量全部无条件复制给所有人，但另一个玩家根本不需要知道你还剩几发。弹药只对持枪者有意义：`COND_OwnerOnly`。需要给别人看的只有“他在开枪”和“他在换弹”，分别用 `COND_SkipOwner` 的 BurstCounter 和 bPendingReload（拥有者自己已经本地预测过了）。

ACoopWeapon：复制设置

```
USTRUCT() struct FAmmoState {
    GENERATED_BODY()
    UPROPERTY() int16  InClip = 10;
    UPROPERTY() int16  Reserve = 30;
    UPROPERTY() uint16 AckSeq = 0;          // 服务器已处理到的开火序号（接受或拒绝都会推进）
};
UPROPERTY(ReplicatedUsing = OnRep_Ammo) FAmmoState Ammo;              // 一个结构体：三者同一次
UPROPERTY(ReplicatedUsing = OnRep_BurstCounter) uint8 BurstCounter;  // 只增不减，OnRep 里看增量
UPROPERTY(ReplicatedUsing = OnRep_PendingReload) bool bPendingReload;

void ACoopWeapon::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& Out) const {
    Super::GetLifetimeReplicatedProps(Out);
    DOREPLIFETIME_CONDITION(ACoopWeapon, Ammo,           COND_OwnerOnly);
    DOREPLIFETIME_CONDITION(ACoopWeapon, BurstCounter,   COND_SkipOwner);
    DOREPLIFETIME_CONDITION(ACoopWeapon, bPendingReload, COND_SkipOwner);
}

int32 ACoopWeapon::GetDisplayedAmmo() const {          // UI 和本机开火判断都用它
    const uint16 Unacked = LocalShotSeq - Ammo.AckSeq;   // uint16 回绕也正确
    return FMath::Max(0, Ammo.InClip - (int32)Unacked);
}
```

为什么不能让 OnRep 直接覆盖本地预测值

本机按下就扣一发（预测），但服务器的弹数要一个 RTT 后才回来，回来的是“服务器处理完**某一发**时”的值。如果 OnRep 直接 `LocalAmmo = 服务器值`，此时本机往往已经又多打了一两发，显示会先往回跳一下再继续减：

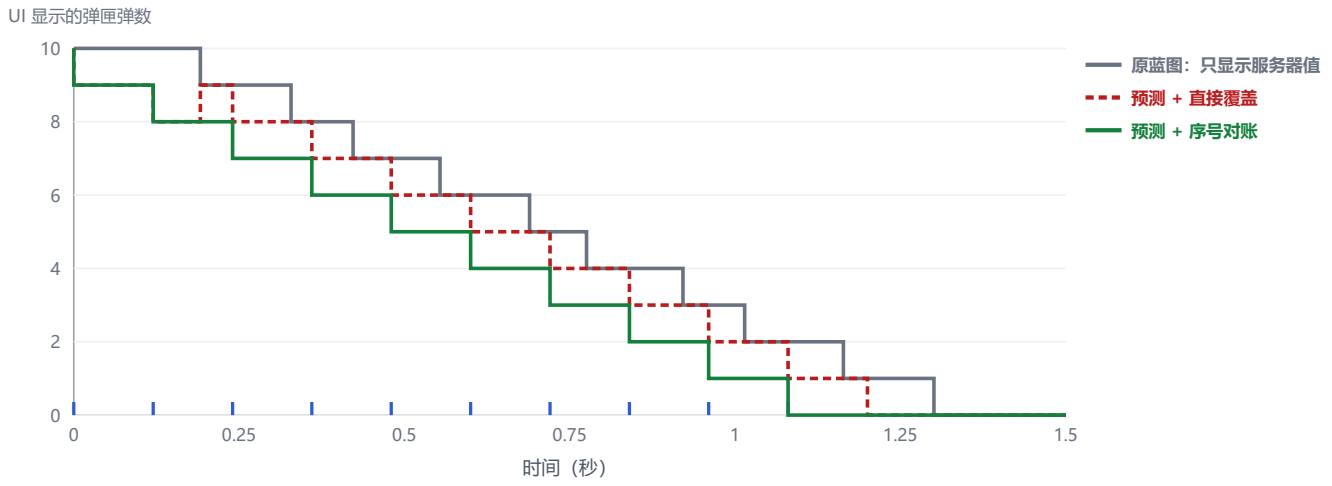


图 5-1　E5：RTT 150 ms、抖动 30 ms，一次连射打空时 UI 上的弹数（底部蓝色刻度 = 每次按下开火）。灰线整体晚约一个 RTT；红色虚线在第一次复制到达时回跳 +1；绿线始终单调递减

显示方式	每次连射平均回跳次数（20 个种子）	最大回跳	本机空放
原蓝图：只显示服务器值	0.00	0	1.00
预测 + 直接覆盖	**1.00**	1	1.00
预测 + 序号对账	**0.00**	0	0.00

表 5-3　E5 汇总。原蓝图“不回跳”是因为它根本不预测——代价是整条曲线晚一个 RTT，而且打空后还会多发一次请求

别人怎么看到你开枪：BurstCounter

另一端需要的不是弹数，而是“他又开了一枪”。复制属性只发最新值，所以不能用布尔 `bFiring`（同一网络帧里开两枪会被合并成一次），而是用一个只增不减的计数器：

OnRep_BurstCounter：另一端播开火表现

```
void ACoopWeapon::OnRep_BurstCounter(uint8 OldValue) {
    const uint8 Shots = BurstCounter - OldValue;          // 一个网络帧内可能有多发（uint8 回绕也
    for (uint8 i = 0; i < FMath::Min<uint8>(Shots, 3); ++i)
        PlayFireCosmetics(/*bFirstPerson*/ false);        // 3P 蒙太奇、枪口火焰、声音、沿枪口方
}
// 晚加入的客户端第一次收到 BurstCounter 时 OldValue = 0，会误播几次：
// 用 COND_SkipOwner + “首次复制不播放”（记录一个 bHasReceivedFirst 标志）即可。
```

ShooterGame 就是这么做的；Lyra 走 GAS，开火是一个 Gameplay Ability，表现用 Gameplay Cue 在各端播放。原 Demo 的 `Multicast_Fire`（Unreliable）也能达到“另一端看到”的效果，问题只在于它排在弹药检查之前，而且开火者自己也要等它。

换弹：服务器状态机 + 本机预测

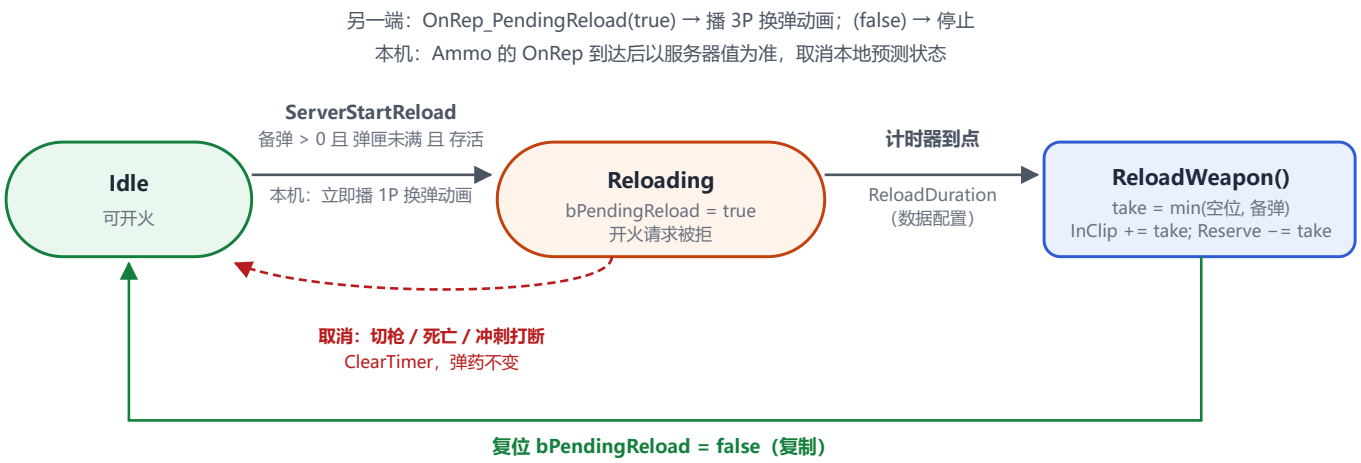


图 5-2 换弹状态机。状态由服务器驱动，本机只预测“动画和不能开火”；弹药转移只在计时器到点时发生一次，被取消就什么都不改

ACoopWeapon：换弹

```
void ACoopWeapon::StartReload() {                          // 本地输入（拥有者）
    if (!CanReloadLocal()) return;                         // 用显示值判断：备弹 > 0 且显示弹数 <
    bLocalReloading = true;
    PlayReloadCosmetics(/*bFirstPerson*/ true);            // 立即播 1P 换弹蒙太奇，不等服务器
    if (HasAuthority()) ServerStartReload_Implementation(); else ServerStartReload();    // Rel
}

void ACoopWeapon::ServerStartReload_Implementation() {
    if (bPendingReload || Ammo.Reserve ≤ 0 || Ammo.InClip ≥ ClipSize || !IsOwnerAlive()) {
        ClientCancelReload();                               // 被拒：通知本机停掉预测的动画（Relia
        return;
    }
    bPendingReload = true;                                 // SkipOwner 复制：另一端播 3P 动画
    if (!IsOwnerLocallyControlled() && GetNetMode() == NM_ListenServer) PlayReloadCosmetics(fa
    GetWorldTimerManager().SetTimer(ReloadTimer, this, &ACoopWeapon::ReloadWeapon, ReloadDurat
}

void ACoopWeapon::ReloadWeapon() {                         // 仅服务器，计时器到点
    const int32 Take = FMath::Min<int32>(ClipSize - Ammo.InClip, Ammo.Reserve);
    Ammo.InClip  += Take;
    Ammo.Reserve -= Take;                                  // 总数守恒：InClip + Reserve 不变
    bPendingReload = false;
}

void ACoopWeapon::CancelReload() {                         // 服务器：切枪、死亡时调用
    if (!bPendingReload) return;
    GetWorldTimerManager().ClearTimer(ReloadTimer);        // 弹药一发都不动
    bPendingReload = false;
    ClientCancelReload();
}
// 本机：OnRep_Ammo 到达时 bLocalReloading = false（服务器已经完成或拒绝）；
// 换弹期间的开火在本地就被 IsReloadingLocal() 挡住，不会发出请求。
```

换弹时长用计时器还是蒙太奇里的 AnimNotify?

很多教程用蒙太奇的“弹匣插入”通知来加子弹。在服务器上这有两个风险：Dedicated Server 默认不刷新不可见网格的动画，通知可能不触发或延迟；蒙太奇被其他动作打断时通知也不会来。更稳妥的做法是**服务器用数据配置的时长计时**，动画只负责表现（蒙太奇时长按同一个数据缩放播放速率）；用 GAS 的话，就是 Ability 里 `WaitDelay` 或服务器端的 `WaitGameplayEvent` 加超时。原 Demo 用 `Delay(GetPlayLength(蒙太奇))` 其实是对的思路，只是放在了 RPC 事件里，没有办法取消。

原实现 vs 正确实现

环节	原 Demo	正确做法
弹药复制	5 个 int / bool 无条件复制给所有人	一个结构体，COND_OwnerOnly，带 AckSeq
本机能否开火	读复制的 当前子弹数量（晚一个 RTT）	读 GetDisplayedAmmo()（预测 + 对账）
别人看到开枪	Unreliable Multicast，排在弹药检查前	BurstCounter（SkipOwner），服务器接受后才 +1
换弹计算	先清零，再调用漏连 Return 的函数	到点一次性 take = min(空位, 备弹)，守恒
换弹状态	正在换弹 复制给所有人，但动画靠 Multicast	bPendingReload（SkipOwner）+ OnRep 播动画；本机预测
打断	无法取消（Delay 在 RPC 事件里）	计时器可 ClearTimer；取消时弹药不变
换弹中开火	服务器多播动画打断换弹，再判断无弹	本地挡住；服务器 ProcessShot 校验 !bPendingReload

本章速记

- 弹药：服务器权威、**COND_OwnerOnly**、结构体带 **AckSeq**；显示 = InClip − (LocalSeq − AckSeq)。
- 别人看到开枪：**COND_SkipOwner 的 BurstCounter**，OnRep 看增量；主机看客户端开火要在服务器上手动播。
- 换弹：本机预测动画 → ServerStartReload 校验 → bPendingReload（SkipOwner）→ 计时器到点一次性转移 → 复位；取消不动弹药。
- E4：原蓝图在备弹不够补满时吞弹匣（3/7 → 0/0）；E5：直接覆盖会回跳，序号对账不会。

▶ **自测：为什么 BurstCounter 用 COND_SkipOwner，而弹药用 COND_OwnerOnly?**

▶ **自测：用一个 bool bIsFiring 复制给别人，别人在 OnRep 里每次播一发，会有什么问题?**

CHAPTER 06

问 ③：弱网造成卡顿，应该怎么优化

“卡”有四种：按了没反应、走路被拉回、明明瞄准了打不中、消息堆积一起到。先用工具分清是哪一种，再对症下药。

本章目标

- 会用 UE 自带工具复现弱网、定位问题类型。
- 讲清 CharacterMovement 的预测与纠正，以及为什么冲刺必须进 SavedMove（E6）。
- 讲清延迟补偿的两种做法和代价（E3），Reliable 的队头阻塞（E2），带宽优化手段（E7）。

30 秒版

先复现、再定位：PIE 网络模拟 / `NetEmulation.PktLag`、`PktLoss`，配合 `stat net`、`p.NetShowCorrections 1`、Networking Insights。然后分四类处理：**反馈慢**→本机预测 + 服务器对账；**被拉回**→影响移动的状态全部进 CMC 的 SavedMove，两端用同一套输入算速度；**打不中**→服务器按开火时刻回溯目标，或客户端判定 + 服务器容差，限制最大回溯；**堆积 / 带宽**→表现不用 Reliable，事件用计数器，降低更新频率、用相关性和休眠、子弹不做复制 Actor。远端角色用插值平滑。

第一步：分清是哪一种“卡”

症状	根因	怎么确认	本书对应
画面本身掉帧、卡一下	不是网络：GC、加载、主线程过载	`stat unit`、`stat game`、Unreal Insights（CPU）	—
按开火 / 换弹后过一会儿才有反应	表现等服务器往返	RTT 越大越明显；本机 = 主机时正常	E1 / 第 4~5 章
自己走路被拉回、原地抖	两端移动模拟结果不一致，服务器纠正	`p.NetShowCorrections 1` 画出纠正位置；日志 LogNetPlayerMovement	E6
别人 / 僵尸移动一顿一顿	更新间隔大、丢包、平滑参数不合适	看该 Actor 的 NetUpdateFrequency 和到达间隔（Networking Insights）	本章第 5 节
明明瞄准了没打中	看到的目标位置和服务器判定时的位置不同	移动目标 + 高延迟时明显，静止目标正常	E3
丢包后几条消息一起生效	Reliable 队头阻塞	Insights 里看 RPC 到达时间成簇	E2
人一多就全体延迟变大	带宽 / 服务器网络帧超预算	`stat net` 的 Out Rate、饱和标志；Insights 按 Actor 统计字节	E7

弱网复现：PIE 设置与控制台命令

```
编辑器偏好设置 → 关卡编辑器 → 运行 → 多人选项 → 启用网络模拟（Network Emulation）
    Emulation Target：Everyone        Profile：Bad / Custom（Average、Bad 为内置预设）
控制台（非 Shipping 包）：
    NetEmulation.PktLag 100           // 单向额外延迟 ms
    NetEmulation.PktLagVariance 20    // 抖动
    NetEmulation.PktLoss 5            // 出包丢包率 %
    NetEmulation.PktIncomingLoss 5    // 入包丢包率 %
    NetEmulation.Off
诊断：
    stat net                          // Ping、In/Out Rate、丢包、通道数
    p.NetShowCorrections 1            // 画出 CharacterMovement 的纠正（红 = 客户端、绿 = 服务器）
    启动参数 -NetTrace=1 -trace=net   // 录 Networking Insights：每个 Actor / 属性 / RPC 的字节数
Epic 官方建议的极端测试条件：500 ms 往返、10% 以上丢包
```

1. 按了没反应：预测（第 4、5 章回顾）

原 Demo 的开火、换弹、后坐力全部在服务器触发，本机等一个往返（E1：RTT 200 ms 时 237 ms）。改成本机预测后自己的延迟为 0；被服务器拒绝的操作用序号对账纠正。**预测只适用于“错了能纠正、纠正代价小”的东西**：开火表现、弹药数、换弹动画可以预测；击杀、掉落、分数不能预测（或只能做“未确认”的提示）。

2. 走路被拉回：CharacterMovement 的预测与纠正

CMC 的移动是 UE 自带的客户端预测：拥有者客户端每帧本地移动，把这一帧的输入（加速度、时间、少量标志位）存成一个 `FSavedMove` 并通过 `ServerMove` 发给服务器；服务器用**同样的输入、自己的状态**重新模拟一遍，误差超过 `MAXPOSITIONERRORSQUARED`（默认 3，即约 1.73 cm）就回一个 `ClientAdjustPosition`；客户端收到后回到服务器给的位置，再把之后还没确认的 SavedMove 重放一遍。

这套机制成立的前提是：**两端用同样的输入能算出同样的结果**。原 Demo 违反了它：

- **冲刺**：`IA_Run` 调用 `Server_StartSprint`，只在服务器把 `IsRunning` 设为 true，这个变量不复制，本机永远是 false——服务器按 500 算，本机按 150 算。
- **瞄准**：`是否瞄准` 只在本机的 AnimBP 上设置，服务器那份永远是 false——本机按 120 算，服务器按 150 算。
- 两端都在 Tick 里用 `FInterpTo(当前速度, 目标速度)` 改 `MaxWalkSpeed`，结果还依赖各自的帧率。

情形	RTT ms	丢包	纠正次 / 秒	平均回拉 cm	最大回拉 cm	5 秒内回拉总距离 cm
原方案：冲刺只在服务器生效	100	0%	**9.20**	21.43	23.33	985.8
原方案：冲刺只在服务器生效	200	0%	**9.20**	21.05	23.33	968.3
原方案：瞄准只在本机生效	100	0%	**9.60**	2.99	3.00	143.5
原方案：瞄准只在本机生效	200	0%	**9.60**	2.98	3.00	143.0
半修复：本地切换 + Server RPC	100	0%	**0.00**	0.00	0.00	0.0
半修复：本地切换 + Server RPC	100	10%	**0.35**	9.61	23.33	17.0
半修复：本地切换 + Server RPC	200	10%	**0.56**	10.87	23.33	30.7
正确：冲刺标志随移动包预测	100	10%	**0.00**	0.00	0.00	0.0
正确：冲刺标志随移动包预测	200	10%	**0.00**	0.00	0.00	0.0

表 6-1　E6：持续前进 5 s，其中 1~3 s、4~5 s 按住冲刺（丢包行取 200 局平均）。纠正发送受 NetworkMinTimeBetweenClientAdjustments（0.1 s，大误差时 0.05 s）限频

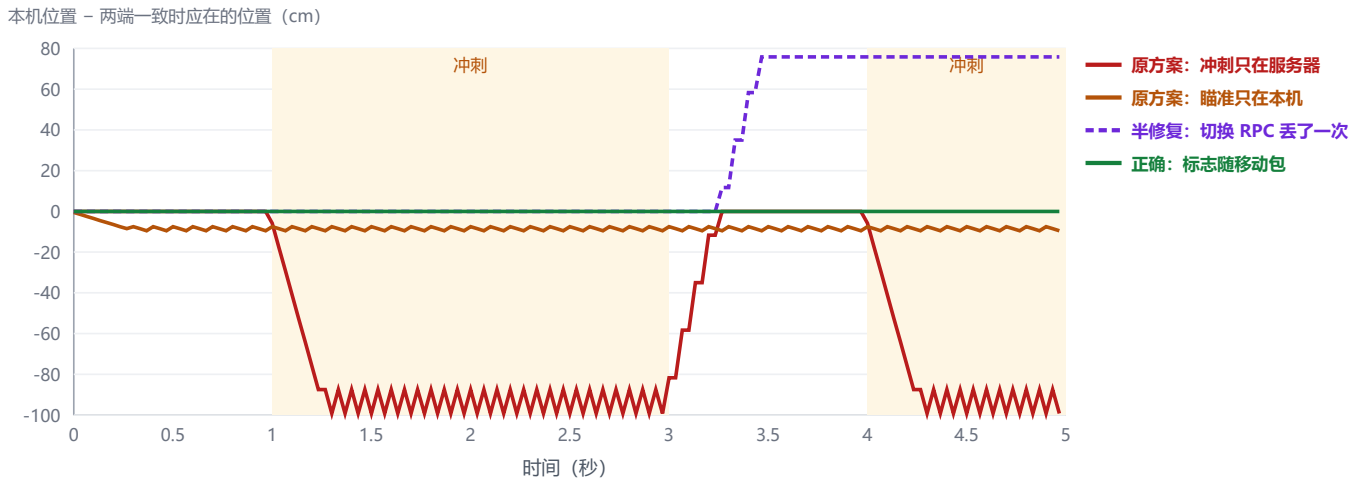


图 6-1　E6：RTT 200 ms 时本机位置的偏差。红线在冲刺期间被不断往前拽（锯齿就是一次次纠正）；紫色虚线是“本地切换 + Server RPC”方案恰好丢了一次切换包：服务器多跑了一个 RTT，本机被往前拉了几十厘米；绿线始终为 0

“半修复”（本机也切换速度，同时发 Server RPC）在不丢包时看起来完全正常，所以很容易被当成正确答案；但 RPC 和移动包走的是不同的路径，一旦切换消息晚到，两端就有一段时间用不同的速度模拟（表 6-1：10% 丢包时每秒仍有 0.56 次纠正）。**正确做法是把冲刺标志放进 SavedMove，和这一帧的输入一起发给服务器**，服务器在重放这一帧时用的就是同一个标志：

把冲刺标志放进 CharacterMovement 的预测（经典 CompressedFlags 写法）

```
// UCoopMovementComponent : UCharacterMovementComponent
float UCoopMovementComponent::GetMaxSpeed() const {
    const float Base = Super::GetMaxSpeed();
    if (bWantsToAim)    return Base * AimSpeedMultiplier;    // 两端用同一个规则算速度，不在 Tic
    if (bWantsToSprint) return Base * SprintSpeedMultiplier;
    return Base;
}

class FSavedMove_Coop : public FSavedMove_Character {
public:
    uint8 bSavedSprint : 1; uint8 bSavedAim : 1;
    virtual void Clear() override { Super::Clear(); bSavedSprint = bSavedAim = 0; }
    virtual void SetMoveFor(ACharacter* C, float Dt, const FVector& Accel, FNetworkPredictionD
        Super::SetMoveFor(C, Dt, Accel, D);
        auto* M = Cast<UCoopMovementComponent>(C->GetCharacterMovement());
        bSavedSprint = M->bWantsToSprint; bSavedAim = M->bWantsToAim;
    }
    virtual uint8 GetCompressedFlags() const override {     // 标志随 ServerMove 一起上行
        uint8 F = Super::GetCompressedFlags();
        if (bSavedSprint) F |= FLAG_Custom_0;
        if (bSavedAim)    F |= FLAG_Custom_1;
        return F;
    }
    virtual bool CanCombineWith(const FSavedMovePtr& New, ACharacter* C, float MaxDelta) const
        auto* N = static_cast<FSavedMove_Coop*>(New.Get());
        return bSavedSprint == N->bSavedSprint && bSavedAim == N->bSavedAim && Super::CanCombi
    }
    virtual void PrepMoveFor(ACharacter* C) override {      // 纠正后重放时恢复当时的标志
        Super::PrepMoveFor(C);
        auto* M = Cast<UCoopMovementComponent>(C->GetCharacterMovement());
        M->bWantsToSprint = bSavedSprint; M->bWantsToAim = bSavedAim;
    }
};

void UCoopMovementComponent::UpdateFromCompressedFlags(uint8 Flags) {   // 服务器：每个移动包解
    Super::UpdateFromCompressedFlags(Flags);
    bWantsToSprint = (Flags & FLAG_Custom_0) != 0;
    bWantsToAim    = (Flags & FLAG_Custom_1) != 0;
}
// 另需：FNetworkPredictionData_Client_Coop::AllocateNewMove() 返回 FSavedMove_Coop，
//       GetPredictionData_Client() 返回它。输入事件里只改 bWantsToSprint / bWantsToAim，不发任何
// UE 5.x 也可以用 FCharacterNetworkMoveData 子类传更多自定义数据，原理相同。
```

别人的角色和僵尸：模拟代理的平滑

模拟代理（SimulatedProxy）不预测，只接收服务器的位置并做平滑：CMC 默认 `NetworkSmoothingMode = Exponential`，网格从旧位置向新位置追赶，胶囊体直接到位。远端一顿一顿时先看两件事：该 Actor 的 `NetUpdateFrequency` 是否太低（10 只僵尸可以 10~20 Hz，玩家保持默认）、动画是否用复制下来的速度驱动（原 Demo 的 AnimBP 用 `GetVelocity()`，这是对的）。

3. 明明瞄准了却打不中：延迟补偿

客户端看到的别人 / 僵尸，是服务器“过去某一时刻”的位置再加上外推。开火请求到达服务器时，目标已经又走了半个 RTT。目标横移越快、延迟越大，服务器在“现在”做射线就越容易打空：

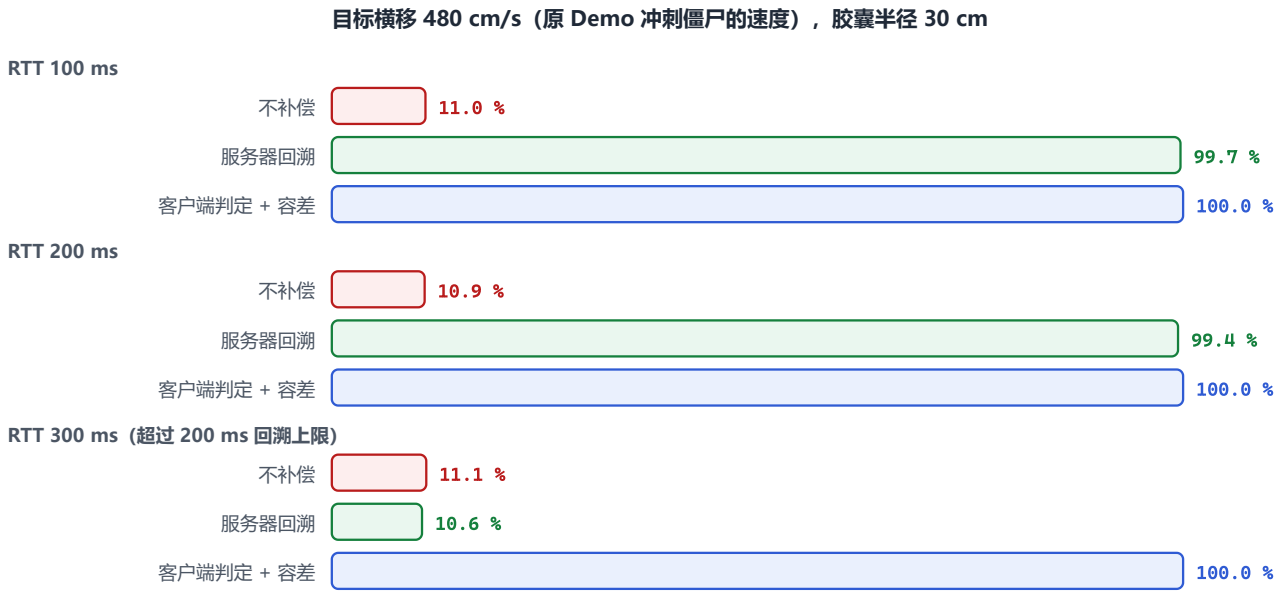
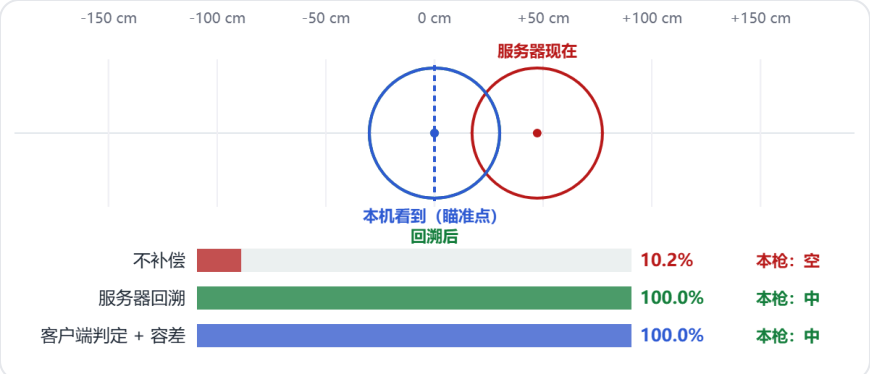


图 6-2 E3：瞄准本机看到的目标中心开枪，服务器判定为命中的比例。回溯在延迟超过上限后失效；容差方案（ShooterGame 的 ClientSideHitLeeway = 200 cm）几乎总是通过，但它信任客户端

目标速度 cm/s	RTT ms	不补偿 %	服务器回溯 %	容差 %	本机显示误差 均值 cm	受害者位移 cm
200	0	**100.0**	100.0	100.0	0.2	0.0
200	50	**100.0**	100.0	100.0	13.1	12.9
200	100	**99.0**	100.0	100.0	21.6	21.4
200	150	**25.7**	100.0	100.0	29.0	28.8
200	200	**25.9**	100.0	100.0	35.8	33.1
200	300	**25.2**	98.7	100.0	46.5	33.0
480	0	**100.0**	100.0	100.0	0.5	0.1
480	50	**33.0**	99.4	100.0	31.4	30.9
480	100	**11.0**	99.7	100.0	51.8	51.2
480	150	**10.9**	99.4	100.0	69.7	69.1
480	200	**10.9**	99.4	100.0	85.9	79.4
480	300	**11.1**	10.6	100.0	111.7	95.7

表 6-2　E3 全表。“显示误差”是本机看到的目标与服务器判定时刻真实位置的距离；“受害者位移”是回溯命中时，目标在服务器当前时刻已经离开命中点多远（被打的一方会觉得“我都躲到墙后了还被打中”）

交互实验 • 延迟补偿：本机看到的目标 vs 服务器判定时的目标



本机显示误差　47.3 cm
回溯后误差　0.0 cm
受害者位移　47.3 cm
（目标在服务器“现在”已离开命中点多远）

拖动“第几枪”看单次判定：蓝 = 本机看到并瞄准的位置，红 = 请求到达服务器时目标的真实位置，绿 = 服务器回溯到开火者看到的时刻（上限 200 ms）。圆的半径就是胶囊半径 30 cm。

两种做法

	服务器回溯（Server-Side Rewind）	客户端判定 + 服务器校验（ShooterGame）
做法	服务器每个网络帧记录目标的位置 / 碰撞体，收到开火时按“客户端看到的时刻”插值回去再射线	客户端射线打中后上报 (目标, 命中点)；服务器检查目标包围盒扩大容差后是否包含命中点、视线是否被墙挡、方向是否合理
优点	结果由服务器算，防作弊	实现简单，开火者手感最好
代价	每帧存历史（内存 + CPU）；要限制最大回溯时间；高延迟玩家的命中会让别人“躲到墙后还挨打”	信任客户端，容差就是外挂的空间
适合	PvP 竞技	**PvE 合作（本 Demo）**：僵尸不会抱怨不公平

服务器回溯：每个网络帧记录一帧历史，按时刻插值

```
struct FHitboxFrame { float Time; FVector Location; FQuat Rotation; };   // 简化：只存胶囊；骨骼
TArray<FHitboxFrame> History;                                            // 环形缓冲，保留 MaxRew

void ACoopZombie::RecordFrame() {                                        // 服务器：在网络帧末尾调
    History.Add({GetWorld()->GetTimeSeconds(), GetActorLocation(), GetActorQuat()});
    while (History.Num() > 2 && History[0].Time < GetWorld()->GetTimeSeconds() - MaxRewind) Hi
}

FHitboxFrame ACoopZombie::SampleAt(float T) const {                      // T = 服务器时间轴上“客户
    T = FMath::Max(T, GetWorld()->GetTimeSeconds() - MaxRewind);         // 限制最大回溯（E3：超过
    for (int32 i = History.Num() - 1; i > 0; --i)
        if (History[i - 1].Time ≤ T) {
            const float A = (T - History[i - 1].Time) / FMath::Max(KINDA_SMALL_NUMBER, History
            return {T, FMath::Lerp(History[i - 1].Location, History[i].Location, A),
                    FQuat::Slerp(History[i - 1].Rotation, History[i].Rotation, A)};
        }
    return History.Num() ? History[0] : FHitboxFrame{T, GetActorLocation(), GetActorQuat()};
}
// 开火时客户端上报 ClientTime（GameState::GetServerWorldTimeSeconds() 估计的服务器时间，再减去插值
// 服务器把候选目标临时移到 SampleAt(ClientTime) 的位置做射线，然后移回。
```

4. 消息堆积：Reliable 的队头阻塞

Reliable RPC 保证顺序：第 n 条丢了，第 n+1 条即使先到也要等第 n 条重传成功才执行。原 Demo 每 0.12 s 一条 Reliable 的 `Server_Fire`，丢包时会出现“卡一下然后几发一起打出去”：

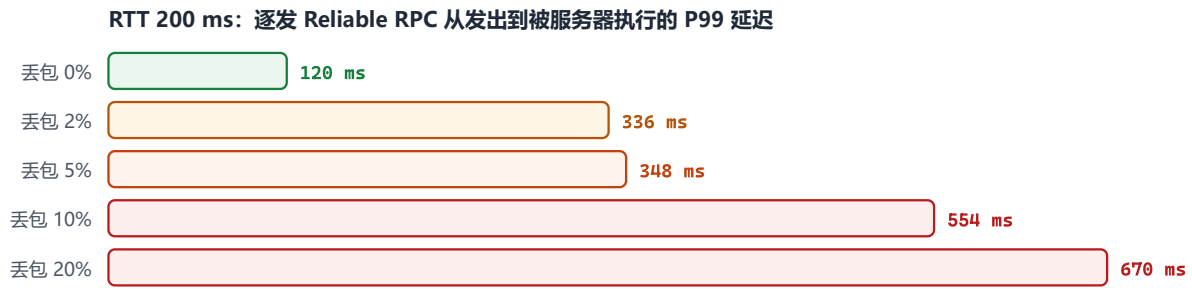


图 6-3 E2：同样 2000 发，丢包从 0% 到 10%，P99 延迟从约半个 RTT 涨到 554 ms，25.8% 的请求和别的请求挤在同一个服务器帧里执行

RTT ms	丢包	平均 ms	P95 ms	P99 ms	最大 ms	同帧扎堆 %	若用 Unreliable 会丢 %
100	0%	60.1	69.0	69.7	70.0	0.0	0.0
100	2%	62.6	69.5	185.1	204.0	3.3	1.9
100	5%	67.3	175.5	193.6	315.2	9.2	4.8
100	10%	74.8	187.7	203.0	449.9	16.7	8.9
100	20%	101.4	295.5	443.1	579.8	35.9	18.8
200	0%	109.9	119.0	119.7	120.0	0.0	0.0
200	2%	117.3	119.9	335.7	566.3	5.4	2.4
200	5%	126.8	310.3	347.5	571.5	11.9	4.6
200	10%	154.4	341.5	554.2	1027.9	25.8	10.2
200	20%	208.1	547.3	670.4	1022.0	44.5	19.8

表 6-3　E2 全表。Reliable 不丢但会排队，Unreliable 不排队但会丢——所以按“这条消息丢了怎么办”来选，而不是一律 Reliable

面试注意：Reliable 的三条规则

- **只给低频、必须到达的消息用**：开始 / 停止开火、换弹、切枪、准备就绪、命中上报。
- **表现类一律 Unreliable 或计数器**：开火特效、受击特效、脚步声。原 Demo 的 Multicast 都是 Unreliable，这点是对的。
- **不要每帧发 Reliable**：可靠缓冲区被塞满时引擎会直接断开这个连接。Tick 里发 Reliable RPC 是面试必扣分项。

5. 带宽与服务器负载

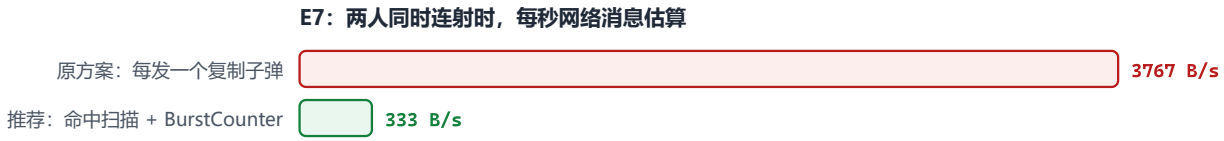


图 6-4 E7：原方案每秒新开 16.7 个子弹 Actor 通道、发 125 次移动更新。字节数是量级假设（见 run_output.txt），真实值请用 Networking Insights 测；结论看数量级

手段	做什么	用在本 Demo
不复制短命 Actor	子弹、弹壳、特效只在各端本地生成	BP_Bullet 改成命中扫描 + 本地曳光
复制条件	OwnerOnly / SkipOwner / InitialOnly	弹药 OwnerOnly；BurstCounter、bPendingReload SkipOwner
NetUpdateFrequency	Actor 默认 100 Hz 检查一次，按需降低	僵尸 10~20 Hz；武器很少变化，可配合休眠
休眠（Dormancy）	不变的 Actor 不参与复制比较；变化前 `FlushNetDormancy()`	拾取物、门、刷怪点
相关性	`NetCullDistanceSquared`、`bOnlyRelevantToOwner`、`bNetUseOwnerRelevancy`	武器跟随持有者；双人小地图可以都相关
量化	`FVector_NetQuantize`、`FVector_NetQuantizeNormal`，int16 代替 int32	命中点、方向
Push Model / Iris	属性改动时主动标脏，省掉逐帧比较；Iris 是 UE5 的新复制系统（需手动启用）	两人局收益不大，面试知道即可
服务器网络帧	`NetServerMaxTickRate`（默认 30）、带宽上限配置	射击游戏常提到 60；代价是带宽和服务器 CPU

本章速记

- **先复现再定位**：NetEmulation + stat net + p.NetShowCorrections + Networking Insights。
- 拉回的本质是两端模拟不一致：**冲刺 / 瞄准进 SavedMove，GetMaxSpeed 统一算**，不另发 RPC、不在 Tick 写 MaxWalkSpeed（E6：原方案每秒纠正 9.2 次 → 0）。
- 打不中：**服务器回溯**（PvP，限最大回溯）或**客户端判定 + 容差**（PvE）（E3：480 cm/s、RTT 200 不补偿 11% → 回溯 99%）。
- Reliable 有序 → 丢包时队头阻塞（E2）；按“丢了怎么办”选 Reliable / Unreliable。

- 带宽：不复制短命 Actor、复制条件、降频、休眠、相关性、量化。

▶ **自测：为什么“本机切换速度 + Server RPC 通知服务器”在 PIE 里测不出问题?**

▶ **自测：服务器回溯为什么要设上限？上限设多少合适?**

PART THREE

第三部分　重做：一个合格的联机 Demo

先用 OOP 的眼光把 UE Gameplay 框架重新梳理一遍（谁是谁、谁包含谁、谁活多久、谁在哪端），再定义“合格”的标准，然后逐个类画出蓝图事件图表，最后是测试与演示。

CHAPTER 07

UE Gameplay 框架：从 World 到 Player BP（OOP 视角）

面试官问“讲讲 GameMode 和 GameState 的区别”“PlayerController 和 Pawn 是什么关系”，考的不是背定义，而是你脑子里有没有这张图：对象怎么组合、怎么继承、活多久、在哪一端。

本章目标

- 画出一个进程里的运行时对象结构：GameInstance → World → Level → Actor → Component。
- 分清继承（is-a）、组合（has-a）和 UE 特有的 Outer / Owner / AttachParent / Instigator / Possess 五种“关系”。
- 说出每个框架类的生命周期、存在于哪一端、客户端上“拿”它会得到什么。
- 按时间顺序讲清主机开局、客户端加入时框架类被创建与调用的顺序。

30 秒版：UE 的 Gameplay 框架是什么

它是一套按职责切开的类：**GameMode 定规则**（只在服务器，负责登录、生成、胜负）；**GameState 广播全局状态**（所有端都有，比赛阶段、剩余时间、玩家列表）；**PlayerState 记“人”的数据**（分数、准备，Pawn 死了也还在）；**PlayerController 是“灵魂”**（输入、相机、UI，只在服务器和它自己的客户端）；**Pawn / Character 是“身体”**（被 Possess 后接受控制，所有端都有副本）；它们都是 `AActor`，活在某个 `UWorld` 的 `ULevel` 里；`UGameInstance` 在 World 之上，整个进程只有一份，切地图也不销毁。功能用 `UActorComponent` 组合（移动、血量、相机），界面是不复制的 `UUserWidget`。

1. 运行时对象结构：一个进程里到底有什么

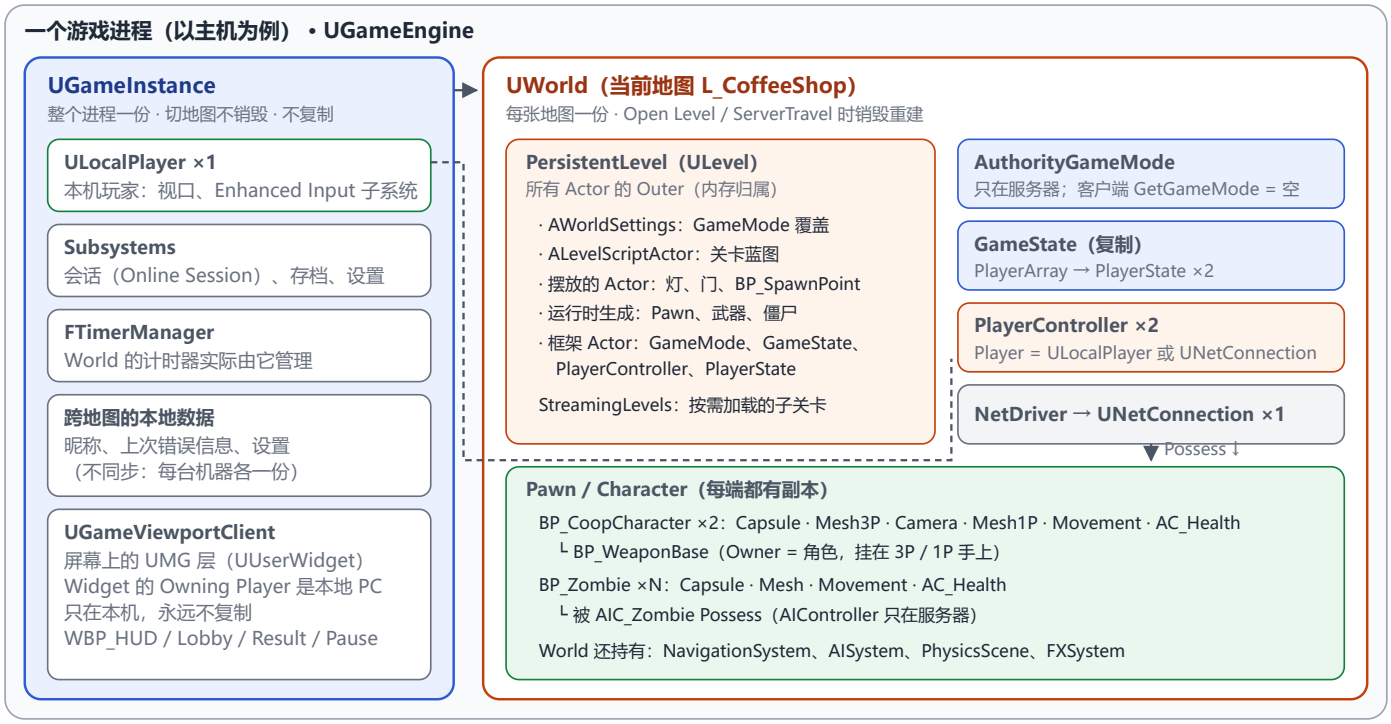


图 7-1 运行时对象结构（主机进程）。GameInstance 在 World 之上；切地图时 World 及其中所有 Actor（包括 GameMode、GameState、PlayerController）都会重建，GameInstance 与 LocalPlayer 留下。虚线：本地 PC 的 Player 指向 ULocalPlayer，远端 PC 的 Player 指向 UNetConnection

一个常被忽略的 OOP 细节：UNetConnection 也是一个 UPlayer

`UNetConnection` 继承自 `UPlayer`，和 `ULocalPlayer` 是兄弟类。`APlayerController::Player` 指向谁，决定了这个 PC 是“本地玩家”还是“远端玩家在服务器上的代理”。`IsLocalController()` 本质上就是在问：这个 PC 背后是不是一个 `ULocalPlayer`。理解这一点，就知道为什么服务器上可以有“不是本地”的 PlayerController，而客户端上永远只有自己那一个。

2. 继承树（is-a）：每个类站在哪一层

类（红 = 本 Demo 新建的子类）	一句话职责	存在于（Listen Server）
UObject	反射、垃圾回收、序列化的基类	—
UGameInstance	进程级：会话、跨地图本地数据	每个进程各一份，不复制
BP_CoopGameInstance	主持 / 加入 / 离开、网络错误处理	每个进程各一份
UPlayer	“一个玩家”的抽象	—
ULocalPlayer	本机玩家：视口、输入子系统	各进程本地
UNetConnection	远端玩家 = 一条网络连接	服务器上每个远端一条
UActorComponent	挂到 Actor 上的功能块（组合）	随所属 Actor
USceneComponent	有变换、可挂接	随所属 Actor
USkeletalMeshComponent	骨骼网格：3P 身体、1P 手臂、枪模	随所属 Actor
UCameraComponent	第一人称相机	随所属 Actor
UCharacterMovementComp	角色移动 + 客户端预测 / 服务器纠正	随 Character
UCoopMovementComponent	冲刺 / 瞄准进 SavedMove（C++）	随 Character
AC_Health	血量与死亡，玩家和僵尸共用	随 Actor（组件复制）
UUserWidget	UMG 界面	仅本机
WBP_HUD / Lobby / Result	只读显示 + 通过 PC 发请求	仅本机
AActor	能放进 World 的对象	看 bReplicates
AInfo	没有外形的“管理者”基类	—
AGameModeBase	规则：登录、生成、胜负	仅服务器
GM_Coop	开局、刷怪、结算、重开	仅服务器
AGameStateBase	全局状态 + PlayerArray	所有端（复制）
GS_Coop	Phase、结束时刻、僵尸数	所有端
APlayerState	跟“人”走的数据	所有端（复制）
PS_Coop	准备、击杀、是否存活	所有端
AController	“灵魂”：Possess 一个 Pawn	—
APlayerController	输入、相机、UI、Client RPC 落点	服务器 + 自己的客户端
PC_Coop	建 UI、准备 / 重开请求、观战	服务器 + 自己的客户端
AAIController	AI 的“灵魂”：行为树、感知	仅服务器
AIC_Zombie	运行 BT_Zombie	仅服务器
APawn	“身体”：可被控制	所有端（复制）
ACharacter	胶囊 + 骨骼网格 + 移动组件	所有端（复制）
BP_CoopCharacter	玩家：1P/3P、武器、输入	所有端
BP_Zombie	僵尸：动画变体、攻击、死亡	所有端
BP_WeaponBase	武器：开火 / 换弹（Owner = 角色）	所有端（复制）
BP_SpawnPoint	刷怪点标记，只给服务器读	关卡摆放，各端都加载

图 7-2 继承树：灰 / 蓝 / 橙 / 绿 / 紫 / 青是引擎类，红色是重做后本 Demo 新建的子类。框架类大多是 AInfo（没有外形）；“身体”是 APawn，“灵魂”是 AController；功能靠 UActorComponent 组合

怎么讲继承树才不像背书

按“三问”讲：① **它是不是 Actor**（能不能放进 World、能不能复制）；② **它在哪一端**（GameMode 只在服务器、PC 只在服务器 + 自己、其余大多所有端）；③ **它活多久**（GameInstance 进程级、GameMode 地图级、Pawn 一条命）。本 Demo 新建的每个子类都只做一件事，红色框的“一句话职责”就是面试时的答案。

3. 五种“关系”：继承和组合之外，UE 还有 Outer / Owner / AttachParent / Instigator / Possess

关系	是什么	谁来设置	影响什么	本 Demo 的武器
继承 is-a	类层级	类定义	能重写哪些事件、能调用哪些函数	BP_Rifle → BP_WeaponBase → AActor
组合 has-a	Actor 由 Component 拼成	组件面板 / 构造函数	功能复用：AC_Health 同时给玩家和僵尸	Mesh1P、Mesh3P、枪口点
Outer	UObject 的“容器”	创建时（NewObject / SpawnActor）	对象路径、GC 归属、随谁一起卸载	PersistentLevel
Owner	网络上的归属	SpawnParams.Owner / SetOwner / Possess	Server RPC 能否发出、OwnerOnly 发给谁、相关性	持有它的角色
AttachParent	变换层级	AttachToComponent	跟着谁移动、旋转	3P 身体的 hand_r 插槽
Instigator	“这件事是谁造成的”	SpawnParams.Instigator	伤害统计、友伤判断、仇恨	持有它的角色
Possess	灵魂 ↔ 身体	Controller.Possess（服务器）	谁的输入驱动它、Owner 链、PlayerState 关联	—（角色被 PC Possess）

面试注意：这五个最容易被混为一谈

“武器的 Owner 是角色”和“武器挂在角色手上”是两件独立的事：前者决定网络（能不能从这个客户端发 ServerFire），后者决定画面（枪跟着手动）。原 Demo 先挂接、再 SetOwner，作者以为是一件事，所以写出了 `for_server` 这种补丁。被问到时用一句话区分：**Outer 管内存，Owner 管网络，AttachParent 管变换，Instigator 管责任，Possess 管控制。**

4. 每个类在哪一端，在客户端上“拿”它会得到什么

类	主机进程	客户端进程	在客户端上调用 Get... 得到	典型错误
GameInstance	自己的一份	自己的一份	本机的 GameInstance	以为改它会同步给别人
GameMode	**有**	**没有**	`Get Game Mode` = None	UI 里 Cast GameMode 读规则
GameState	**有**	**有（复制）**	复制下来的那份；PlayerArray 也有	在 GameState 上存 PlayerController 引用
PlayerState ×2	**两份**	**两份**	可能晚于 Pawn 到达	Pawn BeginPlay 里直接读 PlayerState
PlayerController	**两份**	**只有自己**	`Get Player Controller(0)` = 本机玩家	在服务器上用 index 0 当“当前玩家”
别人的 Pawn	**有**	**有（SimulatedProxy）**	`Get Controller` = None	在别人的 Pawn 上 Cast Controller
僵尸 + AIController	**都有**	**只有僵尸**	僵尸的 `Get Controller` = None	客户端里取 AIController / 黑板
Widget	自己的	自己的	只有本机创建的	服务器“给客户端创建界面”

5. 谁活多久：生命周期



图 7-3 生命周期。* 非无缝漫游（本 Demo 的 servertravel）会断开重连式地重建 PC 与 PlayerState；无缝漫游（bUseSeamlessTravel）下 PC 保留、PlayerState 通过 CopyProperties 拷贝。要跨地图保留的本地数据放 GameInstance

为什么分数放 PlayerState，不放 Character

Character 是“一条命”：死亡、重生会换一个新的 Actor，变量全部丢失；PlayerState 是“这个人”，整局都在，而且**总是复制给所有人**（GameState 的 PlayerArray 在每一端都维护），记分板、队友名字、准备状态直接读它就行。原 Demo 把准备列表放在 GameState、存的是 PlayerController 引用——客户端上别人的 PC 不存在，复制下来是空。

6. 一局是怎么启动的：两张时序图

主机开局（Listen Server）

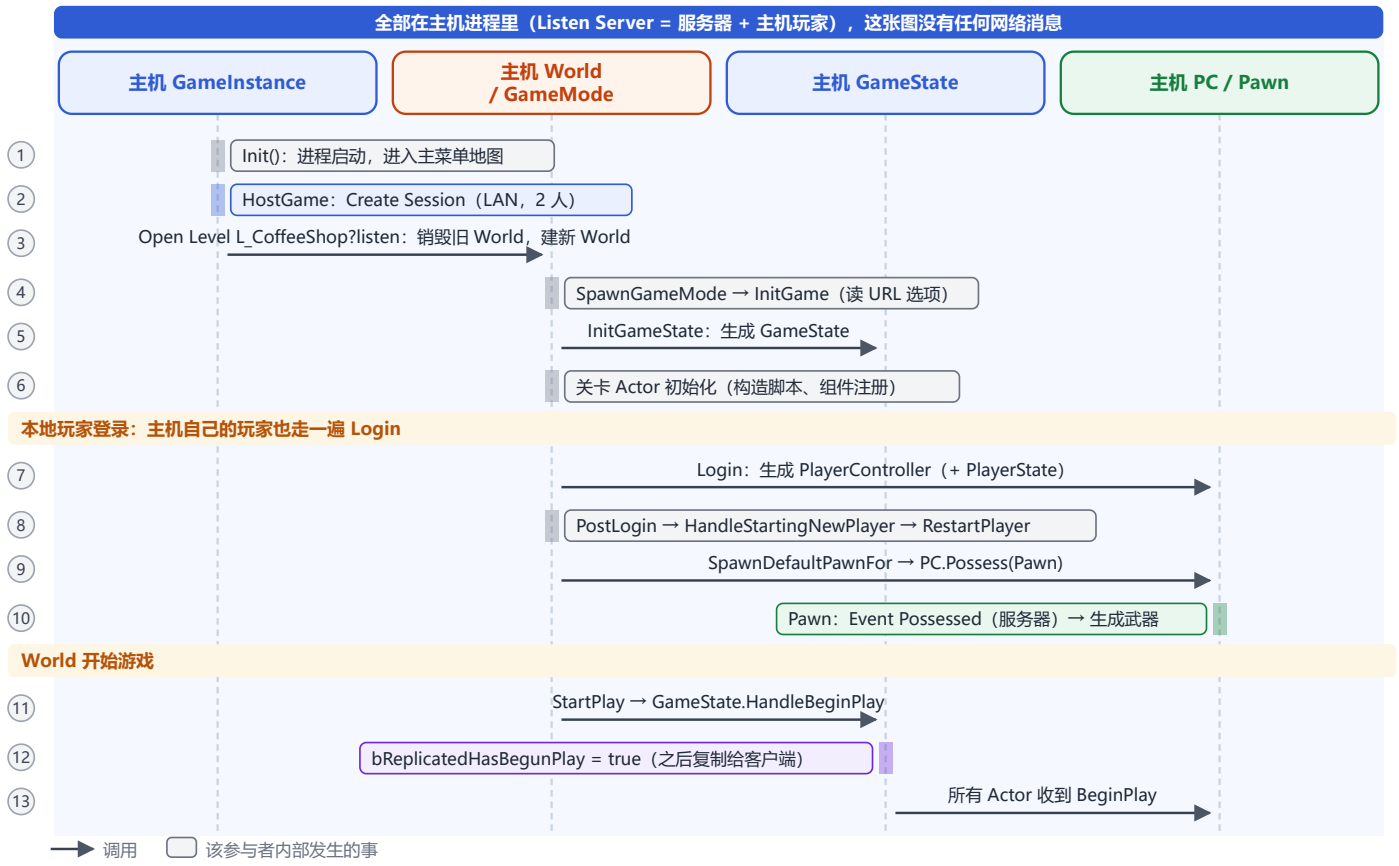


图 7-4　主机开局（GameModeBase 默认流程）。第 7~10 步发生在第 11~13 步之前：主机自己的 Pawn 和武器在 World 开始游戏之前就已生成并被 Possess，所以服务器上这个 Pawn 的 Event Possessed 早于它的 BeginPlay

客户端加入

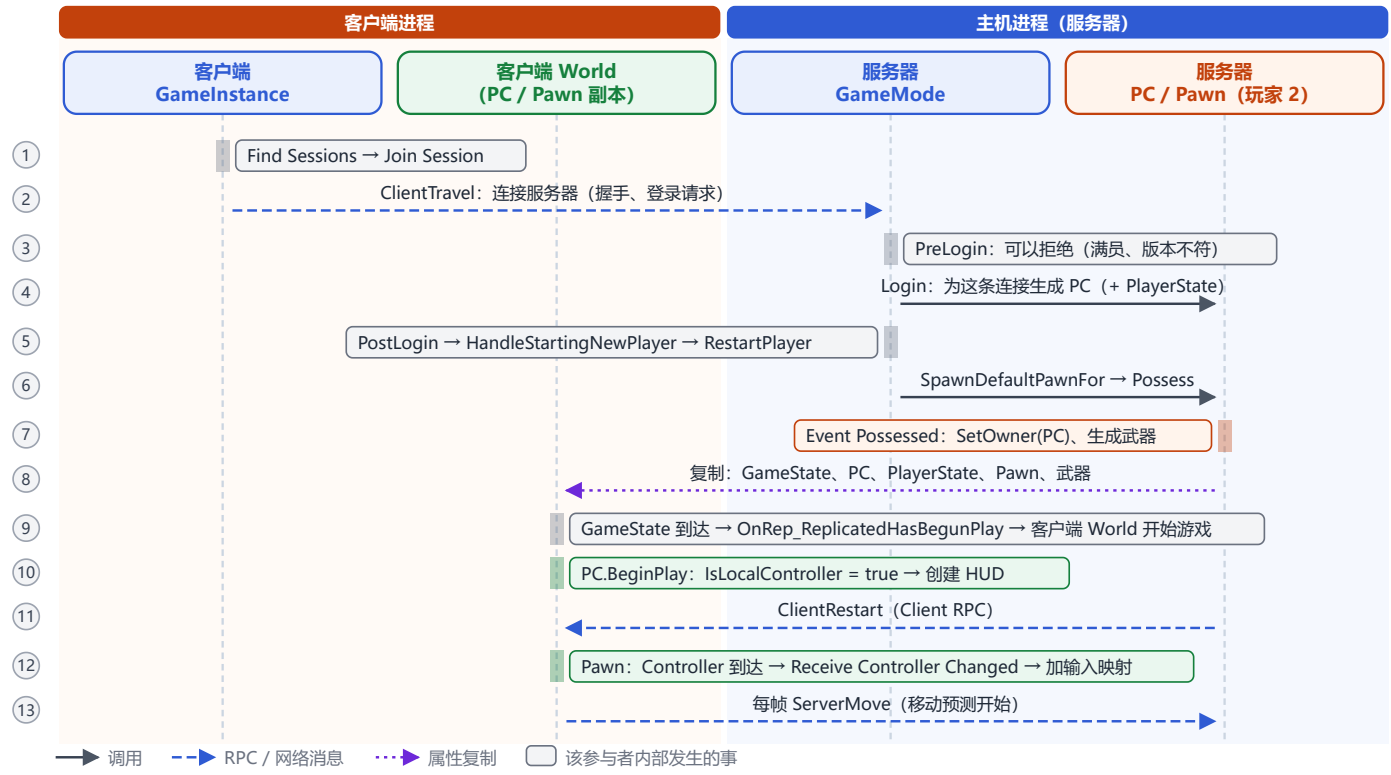


图 7-5 客户端加入。客户端上各 Actor 的到达顺序不保证（PlayerState 可能晚于 Pawn），所以“本地玩家专属”的初始化放在 Receive Controller Changed / PC BeginPlay，依赖 PlayerState 的东西要能处理“暂时为空”。跨过两个色块的箭头才是真正的网络消息：蓝虚线 = RPC / 连接握手，紫点线 = 属性复制

7. 单个 Actor 的生命周期：服务器和客户端各触发哪些事件

阶段	蓝图事件	服务器	拥有者客户端	其他客户端	注意
构造	Construction Script	✓	✓	✓	编辑器里拖动也会跑，不要写游戏逻辑
开始游戏	Event BeginPlay	✓	✓	✓	客户端此时 Controller / PlayerState 可能还没到
被控制	Event Possessed / Unpossessed	✓	✗	✗	只在服务器：生成武器、初始化服务器状态
控制者变化	Event Receive Controller Changed	✓	✓	✗	加输入映射、绑 HUD 的好时机（判断 IsLocallyControlled）
属性到达	OnRep_X（RepNotify 函数）	**蓝图：Set 时也执行**	✓	✓	C++ 的 OnRep 服务器不执行；蓝图会
每帧	Event Tick	✓	✓	✓	能用事件就别用 Tick
结束	Event End Play（Reason）	✓	✓	✓	Destroyed / LevelTransition / Quit 等原因
销毁	Event Destroyed	✓	✓	✓	客户端是跟随服务器销毁，自己调 Destroy 无效

客户端的 BeginPlay 是谁触发的

服务器上，GameMode 的 `StartPlay` 让 GameState 调 `HandleBeginPlay`，所有 Actor 收到 BeginPlay，同时把 `bReplicatedHasBegunPlay` 置为 true；客户端收到这个复制属性，在 `OnRep_ReplicatedHasBegunPlay` 里让自己的 World 开始游戏。所以：**客户端 World 开始游戏时 GameState 一定已经到了**；而之后才复制过来的 Actor，到达时就直接 BeginPlay。这解释了“BeginPlay 里 GameState 一般能拿到、PlayerState 和 Controller 却不一定”。

8. 把 OOP 原则落到这几个类上

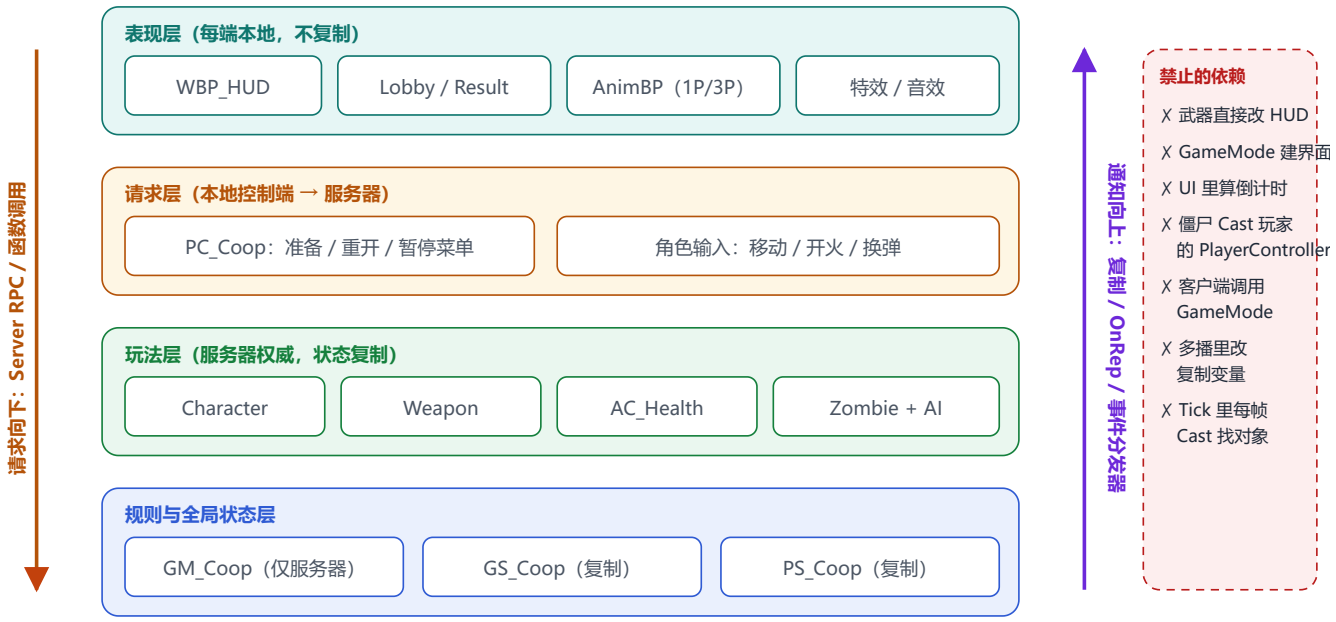


图 7-6 依赖方向。上层只能“读”下层和“发请求”，下层通过复制和事件分发器“通知”上层，从不直接引用上层；右侧是原 Demo 里出现过的反方向依赖

单一职责

每个类一句话说得清：GameMode 管规则、GameState 管全局状态、Weapon 管开火与弹药、AC_Health 管血量。原 Demo 的关卡蓝图同时管菜单、开局、相机、BGM、刷怪，是典型的“上帝对象”。

组合优于继承

血量做成 AC_Health 组件，玩家和僵尸都挂一个，伤害管线只写一遍；而不是给 Character 写一个“可受伤的基类”再让僵尸继承玩家。

依赖倒置 / 事件驱动

下层不认识上层：AC_Health 只广播 OnHealthChanged / OnDeath，谁关心谁订阅（HUD、Character、GameMode）。UI 不再每帧 Cast GetPlayerPawn(0)。

数据驱动

武器参数（伤害、射速、弹匣、换弹时长、散布）放 DA_Weapon 数据资产，加一把枪不改代码；僵尸变体同理。

面向接口

伤害走引擎的 ApplyPointDamage → OnTakeAnyDamage，武器不需要知道打的是僵尸

权威边界清晰

每个事件只在一处决定结果：伤害在服务器、表现在各端、请求从拥有者发出。图里每个节点都标了“在哪端执行”。

还是油桶；需要交互的东西（门、拾取物）用蓝图接口 BPI_Interactable。

面试注意：框架类的三个高频问法

- **“GameMode 和 GameState 有什么区别？”**——不要只答“一个服务器一个客户端”。答：GameMode 是规则（只在服务器，决定能不能进、在哪出生、什么时候结束），GameState 是规则运行的结果（复制给所有人看）；客户端想知道比赛状态只能读 GameState，想改只能通过自己拥有的 Actor 发 Server RPC，再由服务器上的 GameMode 决定。
- **“PlayerController 和 Pawn 为什么要分开？”**——灵魂与身体分离：身体可以死、可以换（载具、观战），灵魂一直在；输入、相机、UI 跟着“人”走，而不是跟着某一具身体。AI 也复用同一套 Possess（AIController）。
- **“GameInstance 能用来做什么？”**——跨地图的本地数据、会话管理、网络错误回调；它不复制，**不能**用来在玩家之间同步任何东西。

本章速记

- 结构：**GameInstance（进程）→ World（地图）→ Level → Actor → Component**；UI 挂在 ViewportClient，只在本机。
- 职责：GameMode 规则（服务器）· GameState 全局状态（复制）· PlayerState 人（复制）· PC 灵魂（服务器 + 自己）· Pawn 身体（复制）。
- 五种关系：Outer 管内存、Owner 管网络、AttachParent 管变换、Instigator 管责任、Possess 管控制。
- 主机开局：本地玩家的 Login → PostLogin → RestartPlayer → Possess 早于 World BeginPlay；客户端 World 开始游戏由 GameState 的复制触发。
- OOP：单一职责、组合（AC_Health）、事件驱动（分发器）、数据驱动（DataAsset）、依赖只向下。

▶ **自测：服务器上一个 PlayerController 的 IsLocalController 为 false，说明了什么？**

▶ **自测：为什么服务器上 Pawn 的 Event Possessed 可能比 BeginPlay 先触发？**

CHAPTER 08

一个合格的联机 Demo 应该是什么样子

“合格”不是“两个人能一起打僵尸”，而是：在 Bad 网络、Dedicated Server、晚加入、谁先死这些条件下，规则依然只有一份、结果依然只认服务器、操作依然跟手。本章把它写成可验收的标准，并给出类清单、复制总表、RPC 总表和工程结构。

本章目标

- 能用 5 个维度、20 条可勾选的标准定义“合格”，并说出每条怎么验证。
- 给出重做后的类清单：父类、职责、在哪端、关键事件、关键变量。
- 给出全部复制变量和 RPC 的总表——面试时被问“你同步了哪些东西”能逐条答出。
- 说清每个复制变量在双人局里**发给了谁**、每个事件**在哪端执行**、原 Demo 的 8 个组播**各自换成了什么**。
- 说清纯蓝图和“蓝图 + 少量 C++”两条路线各自能做到哪一步。

30 秒版：我心目中合格的联机 Demo

① **权威清晰**：伤害、死亡、弹药、比赛阶段、刷怪只在服务器决定，客户端只发请求、只播表现；② **手感跟手**：开火、换弹、冲刺本机零等待，200 ms / 10% 丢包下 UI 不回跳、角色不被拉回；③ **状态完整**：所有持续状态都是复制属性，晚加入的人看到正确的阶段、尸体和弹药；④ **职责单一**：GameMode / GameState / PlayerState / PC / Character / Weapon / 组件各司其职，没有关卡蓝图里的上帝逻辑，没有 `GetPlayerController(0)`；⑤ **可验证**：有测试清单、有弱网录屏、有 `stat net` 数据，并且在 Dedicated Server 模式下也能跑。

1. 验收标准（Definition of Done）

维度	#	标准	怎么验证	原 Demo
权威	1	伤害、死亡、弹药变化只在服务器发生	在客户端打印僵尸血量变化来源；全工程搜 ApplyDamage 都在权限分支后	**✗ 子弹每端 ApplyDamage**
	2	比赛阶段、剩余时间、胜负只有一份（GameState）	网络模拟下两端同时结算；客户端 GetGameMode 为空也不影响 UI	**✗ UI 各算各的**
	3	刷怪、AI 决策只在服务器	客户端世界大纲里僵尸数 = 服务器僵尸数	**✗ 补刷在每端执行**
	4	没有 GetPlayerController(0) / GetPlayerCharacter(0) 出现在可能于服务器执行的逻辑里	全局搜索；只允许出现在 UI / 本地相机代码	**✗ 死亡逻辑里使用**
	5	所有 Server RPC 都在服务器端校验参数和状态	Code Review：每个 Server 事件第一个节点是校验分支	**✗ 无校验**
手感	6	开火 / 换弹表现本机零等待	Bad 网络下按下即有枪口火焰和换弹动画（E1）	**✗ 等一个 RTT**
	7	弹药 UI 单调变化，不回跳	Bad 网络连射录屏逐帧看数字（E5）	**不回跳但整体晚 RTT**
	8	冲刺 / 瞄准不触发移动纠正	`p.NetShowCorrections 1` 下冲刺无红绿胶囊（E6）	**✗ 每秒 9 次纠正**
	9	移动目标在 RTT 200 ms 下命中率不明显下降	对横移僵尸连射统计命中（E3）	**✗ 无补偿**
状态完整	10	持续状态（死亡、换弹、阶段）都是复制属性 + OnRep	Allow Late Joining 下中途加入，看到尸体、阶段、对方在换弹	**✗ 靠多播**
	11	随机结果由服务器决定一次并复制（僵尸外观、攻击动作）	两端并排看同一只僵尸的动作	**✗ 各端各自随机**
	12	换弹守恒；换弹中开火被拒；被打断时弹药不变	表驱动测试（E4）+ PIE 手测	**✗ 吞子弹**

	13	玩家死亡后另一人可继续，死者观战；全灭才失败	两种死亡顺序各测一遍	✗ **一人死全员失败**
职责	14	每个类一句话说清职责；关卡蓝图只做场景演出	类清单（本章第 2 节）	✗ **关卡蓝图管开局**
	15	UI 只读数据、通过委托刷新，不每帧 Cast	UMG 里没有属性绑定函数	✗ **每帧 Cast**
	16	武器 / 僵尸参数数据驱动（DataAsset）	加一把枪不改蓝图逻辑	✗ **写死**
	17	血量是组件，玩家和僵尸共用一套伤害管线	AC_Health 被两个类引用	✗ **两套**
可验证	18	Listen Server、Dedicated Server、独立进程三种模式都能完整打一局	测试清单（第 15 章）	✗ **只测过 PIE**
	19	有 Bad 网络下的演示录屏和 stat net 截图	README 里附视频	✗
	20	日志里没有 No owning connection / Accessed None / 大量纠正	LogNet、LogNetPlayerMovement、LogScript 过滤	✗

面试注意：说“合格”要说可验证的东西

“我的 Demo 同步做得很好”没有信息量。要说“我在 200 ms、10% 丢包下测过：开火零延迟、弹药不回跳、冲刺不被拉回、两个人死亡顺序都测过、Dedicated Server 模式也能跑”——每一条后面都能接一个具体做法。面试官往下追问时，你就有路可走。

2. 重做后的类清单

类	父类	存在于	职责（一句话）	关键事件
BP_CoopGameInstance	GameInstance	各进程	主持 / 加入 / 离开会话，处理网络错误	Init、NetworkError、TravelError
GM_Coop	GameModeBase	仅服务器	登录、出生、开局、刷怪、胜负、重开	OnPostLogin、OnLogout、StartMatch、EndMatch
GS_Coop	GameStateBase	所有端	把比赛状态广播给所有人	OnRep_Phase → OnPhaseChanged
PS_Coop	PlayerState	所有端	“这个人”的状态：准备、击杀、存活	OnRep_Ready、OnRep_Kills
PC_Coop	PlayerController	服务器 + 自己	本地 UI、暂停菜单；准备 / 重开请求；观战	BeginPlay、OnPossessedPawnChanged
BP_CoopCharacter	Character	所有端	玩家身体：输入、武器、死亡表现	Possessed、ReceiveControllerChanged、OnRep_CurrentWeapon
BP_WeaponBase	Actor	所有端	开火、换弹、弹药、开火表现	StartFire、Server_FireShot、Server_StartReload、OnRep_*
AC_Health	ActorComponent	随 Actor	血量、受伤、死亡，广播事件	HandleDamage、RecordHit、OnRep_Health / LastHit / bIsDead
BP_Zombie	Character	所有端	僵尸身体：外观变体、攻击表现、死亡	BeginPlay、OnRep_VariantSeed、StartAttack、HandleDeath

AIC_Zombie	AIController	仅服务器	运行行为树	OnPossess
BT_Zombie / BB_Zombie	BehaviorTree	仅服务器	选目标、追击、攻击	Service / Task
BP_SpawnPoint	Actor（不复制）	关卡摆放	刷怪点标记	—
DA_Weapon / DA_Zombie	PrimaryDataAsset	资产	参数：伤害、射速、弹匣、换弹时长、速度、攻击力	—
WBP_HUD / Lobby / Result / Pause	UserWidget	仅本机	只读显示 + 通过 PC 发请求	Construct、委托回调

表 8-1　14 个类。和原 Demo 比：新增 GameInstance / PlayerState / PlayerController / AC_Health / AIController / 行为树 / 数据资产；关卡蓝图里的开局逻辑全部移走

3. 工程结构

Content/Coop/	内容	备注
Coop	本 Demo 的全部逻辑资产	与美术资源分开，方便只拉逻辑
Core	BP_CoopGameInstance · GM_Coop · GS_Coop · PS_Coop · PC_Coop	框架类
Characters	BP_CoopCharacter · ABP_Coop1P · ABP_Coop3P	玩家
Weapons	BP_WeaponBase · BP_Rifle · DA_Rifle · S_AmmoState	武器与数据
Components	AC_Health · S_HitInfo	可复用组件
AI	BP_Zombie · AIC_Zombie · BT_Zombie · BB_Zombie · BTT_Attack · BTS_FindTarget	僵尸
Level	BP_SpawnPoint · L_Menu · L_CoffeeShop	关卡只摆东西、做演出
UI	WBP_MainMenu · WBP_Lobby · WBP_HUD · WBP_Result · WBP_Pause	只读 + 发请求
Input	IMC_Default · IA_Move / Look / Fire / Aim / Reload / Sprint / Pause	Enhanced Input
Source（可选）	UCoopMovementComponent（冲刺 / 瞄准进 SavedMove）	唯一必须 C++ 的部分

图 8-1　工程结构。逻辑资产集中在 Content/Coop，美术包保持原位；面试时打开工程，目录本身就在讲架构

4. 复制总表：同步了哪些东西

所在类	变量	类型	复制条件	RepNotify	谁写
GS_Coop	Phase	E_MatchPhase	无	✓	GM
GS_Coop	MatchEndServerTime	Float	无	—	GM
GS_Coop	AliveZombies	Integer	无	—	GM
PS_Coop	bReady	Bool	无	✓	服务器（经 P 请求）
PS_Coop	Kills	Integer	无	✓	GM
PS_Coop	bIsAlive	Bool	无	—	GM
BP_CoopCharacter	CurrentWeapon	BP_WeaponBase 引用	无	✓	服务器（Possessed
BP_CoopCharacter	bIsAiming	Bool	**Skip Owner**	—	服务器（移动组件解出瞄准标志后写）
BP_WeaponBase	Ammo	S_AmmoState {InClip, Reserve, AckSeq, ReloadCount}	**Owner Only**	✓	服务器
BP_WeaponBase	BurstCounter	Integer	**Skip Owner**	✓	服务器（接受一发 +1）
BP_WeaponBase	bPendingReload	Bool	**Skip Owner**	✓	服务器
AC_Health	Health	Float	无	✓	服务器
AC_Health	LastHit	S_HitInfo {Counter, Point, bHead}	无	✓	服务器

AC_Health	bIsDead	Bool	无	✓	服务器
BP_Zombie	VariantSeed	Integer	**Initial Only**	✓	服务器（生成时）
BP_Zombie	AttackInfo	S_AttackInfo {Counter, Index}	无	✓	服务器（每次攻击 Counter + 1，随机选 Index）

表 8-2　16 个复制变量（原 Demo 11 个，其中 10 个的 OnRep 是空的）。另有引擎自带复制：Character 的移动（ReplicatedMovement + CMC）、Controller、PlayerState 引用、RemoteViewPitch

在蓝图里“打开复制”要勾哪几处

① **Actor 本身**：Class Defaults → Replication → `Replicates` ✓（GameState / PlayerState / PlayerController / Character 父类默认已开；BP_WeaponBase、BP_Zombie 是自己建的，要手动勾）；② **组件**：组件 Details → `Component Replicates` ✓（AC_Health 要勾，否则它的变量不会复制）；③ **变量**：变量 Details → Replication 选 `Replicated` 或 `RepNotify`，再按需要选 Replication Condition。三层缺一层，变量就不会到客户端——第 9~12 章每个类开头的“网络规格”面板就是按这三层列的。

4.1 这些变量在双人局里到底发给了谁

Replication Condition 决定“发给哪些客户端”。Listen Server 双人局只有两个进程：**主机进程**（它就是服务器，同时有一个本地玩家）和**客户端进程**。属于某个玩家的变量（武器、角色上的）两个玩家各有一份，要分开看；GameState、僵尸这类不属于任何玩家的只有一份。

变量	复制设置	这一份属于	主机进程（= 服务器）	客户端进程	用途
Phase	RepNotify · None	—	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**收到 → OnRep**	大厅 / HUD / 结算切换
MatchEndServerTime	Replicated · None	—	权威值（直接读写）	**收到**	倒计时
AliveZombies	Replicated · None	—	权威值（直接读写）	**收到**	HUD 僵尸数
bReady • Kills（PS）	RepNotify · None	—	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**收到 → OnRep**	大厅勾选、记分
bIsAlive（PS）	Replicated · None	—	权威值（直接读写）	**收到**	队友状态、僵尸选目标
CurrentWeapon	RepNotify · None	—	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**收到 → OnRep**	挂接 1P / 3P
bIsAiming	Replicated · Skip Owner	主机玩家的	权威值（直接读写）；主机就是拥有者	**收到**	别人的 3P 开镜姿势

		客户端玩家的	权威值（直接读写）	**不发送（它是拥有者，已本地预测）**	
Ammo	RepNotify · Owner Only	主机玩家的	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**不发送（这份不归它）**	拥有者对账、刷新 HUD
		客户端玩家的	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**收到 → OnRep**	
BurstCounter	RepNotify · Skip Owner	主机玩家的	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行；主机就是拥有者	**收到 → OnRep**	别人播 3P 开火
		客户端玩家的	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**不发送（它是拥有者，已本地预测）**	
bPendingReload	RepNotify · Skip Owner	主机玩家的	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行；主机就是拥有者	**收到 → OnRep**	别人播 / 停 3P 换弹
		客户端玩家的	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**不发送（它是拥有者，已本地预测）**	
Health • LastHit • bIsDead	RepNotify · None	—	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**收到 → OnRep**	血条、受

					击、倒地
VariantSeed	RepNotify · Initial Only	—	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**只在首次复制时收到一次 → OnRep**	僵尸外观、速度
AttackInfo	RepNotify · None	—	权威值（直接读写）；蓝图 Set 时 OnRep 在这里执行	**收到 → OnRep**	僵尸攻击蒙太奇

表 8-4　复制接收矩阵（Listen Server · 双人）。“这一份属于”= 该 Actor 的拥有者是谁；绿 / ✓ = 收到，红 / ✗ = 不发送，黄 / ◐ = 只发一次。主机进程一栏永远是“权威值”——不存在“复制给主机”这回事

怎么读表 8-4：四条规律

- **主机进程只有权威值，没有“收到”**。主机上的 OnRep 不是“收到复制”触发的，而是蓝图 RepNotify 的特性：服务器 `Set w/ Notify` 时**立即在服务器执行一次**（C++ 不会）。所以 Listen Server 的主机能“看到”客户端的开火 / 换弹表现，靠的就是这次服务器端 OnRep。
- **Owner Only**：只有拥有者客户端收到。客户端看不到主机玩家的弹药（不需要），主机也不需要收（它就是服务器）。弹药少发一份，还防止外挂读到别人的弹药。
- **Skip Owner**：拥有者已经本地预测过，只发给“别人”。客户端玩家的 BurstCounter 不会发回给它自己；主机玩家的 BurstCounter 会发给客户端（客户端是“别人”）。坑在服务器端：Set 时 OnRep 在主机执行，对“主机玩家自己的武器”也会执行——所以 OnRep 里要用 `NOT Owner.IsLocallyControlled` 过滤（图 11-4）。
- **Initial Only**：只在 Actor 第一次复制给某个客户端时带上，之后改了也不发。VariantSeed 生成后不变，正合适；晚加入的人也能收到——他的“第一次”就是加入时。

5. RPC 与组播总表：每一条都要能说出“为什么是它”

名称（所在类）	Replicates • Reliable	在哪调用 → 在哪执行	服务器校验 / 为什么
Server_SetReady PC_Coop	Run on Server Reliable	拥有者客户端（大厅按钮 → Get Owning Player） → 服务器上这个玩家的 PC	Phase == Waiting；低频必达
Server_RequestRestart PC_Coop	Run on Server Reliable	拥有者客户端（结算按钮） → 服务器	Phase 为 Won / Lost；只允许主机（服务器上 IsLocalController 的那个 PC）
Client_StartSpectating(Target) PC_Coop	Run on owning Client Reliable	服务器（GM.OnPlayerDied） → 死者的客户端（死者是主机时就在主机本地）	切换观战视角，必须送达
Client_ShowHitMarker PC_Coop	Run on owning Client Unreliable	服务器（Server_FireShot 命中后） → 开火者的客户端	纯表现，丢了无所谓
Client_ShowDamageFeedback PC_Coop	Run on owning Client Unreliable	服务器（玩家受伤后） → 受伤者的客户端	镜头抖动、方向指示（原 Demo 的 Client_PlayHitFX ✓）
Server_FireShot(Seq, Start, Dir, Time, HitActor, HitBone) BP_WeaponBase	Run on Server Reliable	拥有者本机，每发一次 → 服务器	存活 / 未换弹 / 射速令牌 / 弹药 / 起点 / 方向 / 命中容差；会造成伤害，不能丢
Server_StartReload BP_WeaponBase	Run on Server Reliable	拥有者本机 → 服务器	备弹 > 0、未满、未在换弹、存活

Client_CancelReload BP_WeaponBase	Run on owning Client Reliable	服务器（拒绝 / 打断） → 持枪者的客户端	停掉本机预测的换弹动画
Multicast_DeathCry BP_Zombie	Multicast Unreliable	**服务器**（HandleDeath） → 服务器 + 所有与这只僵尸相关的客户端	一次性惨叫；晚加入不该再响
Multicast_Growl BP_Zombie	Multicast Unreliable	**服务器**（BTS_FindTarget 换目标） → 服务器 + 所有相关客户端	一次性低吼；丢了无所谓

表 8-3　10 个网络事件 = 8 个 RPC（4 个 Run on Server、4 个 Run on owning Client）+ 2 个组播。原 Demo 19 个，其中 8 个是组播。删掉的包括：“播动画 / 改状态的组播”（改成复制属性 + OnRep）、冲刺的两个 Server RPC（改进移动预测）、关卡蓝图与 GameMode 上没有意义的 RPC

为什么 RPC 反而变少了

原 Demo 每想同步一件事就加一个组播：开火、换弹、受击、死亡、攻击、开局。这些大多是**状态**（在换弹、死了、开局了），用复制属性表达更对：晚加入能看到、丢包后会被下一次更新纠正、不会被打断。网络事件只剩三类：客户端的**请求**（Run on Server）、只给某个人的**即时反馈**（Run on owning Client）、服务器发起的**一次性世界表现**（Unreliable 组播）。

5.1 组播去哪了：原 Demo 的 8 个 Multicast 逐个替换

原组播（所在类）	原来做什么	出了什么问题	重做后用什么	为什么
Multicast_StartGame BP_Player	人数到齐后 GM 对**每个 Pawn** 调一次 → 触发关卡蓝图 Start_Logic	2 个 Pawn = 每端开局逻辑跑 2 遍；晚加入收不到	**GS_Coop.Phase • RepNotify** → OnRep_Phase → OnPhaseChanged	“比赛开始了”是状态
Multi_SetStrat 关卡蓝图	Server_SetStart → 组播在每端写 GameState 的“游戏是否开始”	客户端不拥有关卡蓝图，它调用的 Server_SetStart 被丢弃；“游戏是否开始”本身不复制，晚加入者永远是 false（附录 A #15）	删除；改由 GM 在服务器写 Phase	状态要放进复制变量，由服务器写一次
Multi_Death BP_Player	任一玩家死亡 → 每端弹“失败”并暂停	活着的人也被判负	**AC_Health.bIsDead • RepNotify**（尸体）+ GM 判全灭 → Phase = Lost；死者收 Client_StartSpectating	死亡是状态，胜负是规则
Multicast_Fire BP_Weapon	服务器收到开火请求后，先组播开火动画	开火者等一个 RTT；排在弹药检查前，打空也播	本机预测 + **BurstCounter • RepNotify • Skip Owner**	高频；拥有者已预测，不该再收
Multicast_Change BP_Weapon	换弹动画	开火者等 RTT；晚加入看不到；打断后无法撤销	本机预测 + **bPendingReload • RepNotify • Skip Owner** + Client_CancelReload	“正在换弹”是持续状态
Multicast_PlayAttackAnim BP_Zombie	各端随机选攻击蒙太奇；On	各端动作不同；客户端写复制变量	**AttackInfo{Counter, Index} • RepNotify**；bAttacking 只在服务器	随机结果由服

	Completed 里写“正在攻击”			务器决定一次
Multicast_HitReact BP_Zombie	受击动画（各端随机）	同上	`AC_Health.LastHit{Counter, Point, bHead}` • RepNotify	Counter 保证连续两次相同命中也触发 OnRep
Multicast_Death BP_Zombie	倒地 + 补刷一只	客户端本地扣血后调用 → 只在本地执行；补刷在每端执行	`bIsDead` • RepNotify（布娃娃）+ GM 在服务器补刷 + `Multicast_DeathCry`（惨叫）	尸体是状态，惨叫是事件

表 8-5　8 个组播的去向：7 个换成 RepNotify，1 个直接删除；新加的 2 个组播只负责一次性音效

面试注意：组播四条件，全满足才用

① **由服务器调用**（客户端调用的组播只在本地执行）；② **一瞬间的**，不是持续状态；③ **晚加入的人不需要**；④ **丢了无所谓**，所以用 Unreliable。再加一条实操：组播没有 Skip Owner，拥有者已经预测过的表现（开火、换弹）不要用组播。被问“为什么不用 Multicast 同步开火”时，就按这四条逐一说。

5.2 每个事件在哪端执行：速查

第 9~13 章每张事件图表的入口节点右上角都有一个执行端标签，图下方有“本图网络摘要”。把全书的事件按执行端归类，就是下面这张表——面试时被问“这段逻辑在哪跑”，先想它属于哪一行。

执行端	本 Demo 里有哪些	靠什么保证只在这里执行
仅服务器	GM_Coop 的全部事件；AIC_Zombie 与行为树；角色 Event Possessed；AC_Health.HandleDamage / RecordHit；角色 / 僵尸 HandleDeath；武器 FinishReload / CancelReload / ServerStopAll；僵尸 StartAttack / DoAttackTrace / EndAttack；PS.AddKill；所有 Server_* 的函数体	对象只在服务器存在（GameMode、AIController）；伤害事件只在 Authority 分支里绑定；由服务器事件调用；Run on Server
仅拥有者客户端	Client_StartSpectating、Client_ShowHitMarker、Client_ShowDamageFeedback、Client_CancelReload	Run on owning Client；拥有者是主机时在主机本地执行
服务器 + 所有客户端	Multicast_DeathCry、Multicast_Growl	Multicast，且必须由服务器调用
服务器（Set 时）+ 收到的客户端	全部 OnRep_*：Phase、bReady、Kills、CurrentWeapon、Ammo、BurstCounter、bPendingReload、Health、LastHit、bIsDead、VariantSeed、AttackInfo，以及在 OnRep 里广播的分发器（OnHealthChanged、OnDeadVisual…）	蓝图 RepNotify 的特性；需要排除某一端时用 IsLocallyControlled、bBegunPlay 过滤
本地控制端	输入事件 IA_*；武器 StartFire / StopFire / FireShot / StartReload；PC 的 IA_Pause	Enhanced Input 只在本地 PC 上触发
所有端（每个副本）	BeginPlay、Tick、EndPlay；AnimBP 更新；Receive Controller Changed（服务器 + 拥有者）	需要分端的逻辑在里面用 Switch Has Authority / Is Local Controller 分流
本机，不联网	全部 Widget 事件；GI 的 HostGame / FindGames / JoinGame / LeaveGame	Widget 与 GameInstance 都不复制

表 8-6　执行端速查。蓝图里没有“这个事件只在服务器执行”的开关（除了 Run on Server），执行端由**对象存在于哪端**和**谁调用**共同决定——这就是第 9 章图 9-1a 那棵判断树

6. 数据资产与碰撞设置

DA_Weapon 字段	Rifle	用途
BaseDamage / HeadshotMultiplier	40 / 2.5	原 Demo：身体 40、爆头 100
FireInterval	0.12 s	本机计时器与服务器射速校验共用
ClipSize / MaxReserve	10 / 30	弹药初值
ReloadDuration	按蒙太奇时长（约 2 s）	服务器计时；动画按此缩放播放速率
Spread（腰射 / 开镜）	2.5° / 0°	Seq 作随机种子，服务器可复现
Range / TraceChannel	10000 cm / Weapon	命中扫描
FireMontage1P / 3P、ReloadMontage1P / 3P、MuzzleFX、ImpactFX、Sounds	—	表现资源全部软引用

通道	类型	默认	谁 Block	原 Demo
Weapon	Trace	Block	僵尸网格（Physics Asset 判头）、墙；玩家胶囊 Ignore（合作无友伤）	Bullet 是 Object 通道，给复制子弹用
ZombieAttack	Trace	Ignore	玩家胶囊	丧尸攻击 ✓（保留）

7. 两条路线：纯蓝图能走多远

需求	纯蓝图	蓝图 + 少量 C++
服务器权威、复制、RPC、OnRep、复制条件	**全部可以**	**同左**
开火预测 + 序号对账、BurstCounter	**可以（OnRep 没有旧值参数，需要本地保存上次值）**	**C++ 的 OnRep 可带 OldValue**
换弹状态机、可取消	**可以（用 Set Timer by Event，不用 Delay）**	**同左**
冲刺 / 瞄准进移动预测	**做不到（SavedMove 只能 C++）**	**UCoopMovementComponent 约 60 行**
服务器回溯（PvP）	**能做但慢且难维护**	**推荐 C++**
会话（主持 / 加入）	**Create / Find / Join Session 节点**	**同左**

纯蓝图下冲刺的“次优解”

不写 C++ 时，至少做到：输入时**本机和服务器同时**切换（本地 Set + Server RPC），两端用同一个函数从同一组变量算速度，并把冲刺状态做成复制变量给模拟代理看。这能消除原 Demo“只有一端生效”的持续纠正（E6 第一行），但丢包时仍有偶发纠正（E6 “半修复”行：每秒 0.56 次）。面试时要主动说出这个局限，并说明 C++ 版本怎么彻底解决——这比假装纯蓝图能做完美更可信。

C++	蓝图里的对应
`UFUNCTION(Server, Reliable)`	自定义事件 → Replicates：Run on Server，勾 Reliable
`UFUNCTION(Client, Reliable)`	Run on owning Client
`DOREPLIFETIME_CONDITION(…, COND_OwnerOnly)`	变量 → Replication：RepNotify，Replication Condition：Owner Only
`HasAuthority()` / `IsLocallyControlled()`	Switch Has Authority / Is Locally Controlled
`FActorSpawnParameters.Owner / Instigator`	Spawn Actor 节点的 Owner / Instigator 引脚
`GetWorld()→ServerTravel( … )`	Execute Console Command “servertravel 地图名”（服务器上）
`SetIsReplicated(true)`（组件）	组件细节面板：Component Replicates

本章速记

- 合格 = **权威清晰、手感跟手、状态完整、职责单一、可验证**，20 条标准每条都能说出验证方法。
- 14 个类；16 个复制变量（Ammo OwnerOnly、BurstCounter / bPendingReload / bIsAiming SkipOwner、VariantSeed InitialOnly）；8 个 RPC（4 个请求、4 个单人反馈）+ 2 个 Unreliable 组播（一次性音效）。
- 打开复制三层：Actor Replicates → 组件 Component Replicates → 变量 Replicated / RepNotify + Condition。主机进程只有权威值，主机上的 OnRep 来自服务器 Set。
- 原 Demo 8 个组播：7 个换成 RepNotify、1 个删除。组播四条件：服务器调用、一瞬间、晚加入不需要、丢了无所谓。
- 纯蓝图能做完除“移动预测扩展”外的全部内容；冲刺要彻底不被拉回需要约 60 行 C++。

CHAPTER 09

框架层事件图表：GameInstance / GameMode / GameState / PlayerState / PlayerController

先把三件事讲清楚：**每个事件在哪台机器上执行、哪些变量勾了复制（以及发给谁）、哪些调用是 RPC / 组播。**然后看一局完整时序，再逐个类打开“网络规格面板 + 事件图表”。

本章目标

- 读懂本书的蓝图风格事件图表：执行端标签、复制标注条、每张图下方的“网络摘要”。
- 给出任意一个事件，能判断它在服务器、拥有者客户端还是所有端执行；给出任意一个 Set，能说出它会不会、发给谁。
- 按时间顺序说出一局里框架类之间的调用与复制。
- 写出 5 个框架类的事件图表，并说出原 Demo 对应逻辑错在哪。

1. 怎么读本书的事件图表

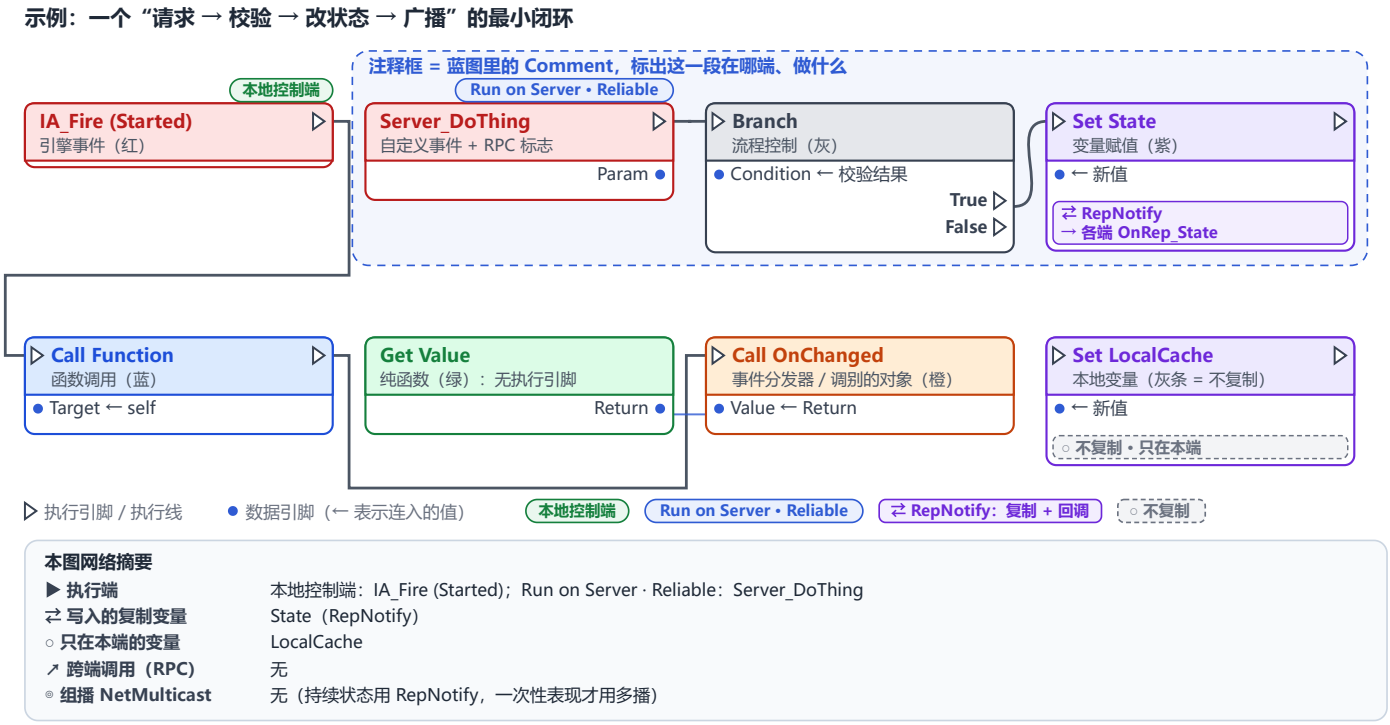


图 9-1　图例。① 节点右上角的小标签 = **执行端**；② Set 节点底部的条 = **复制标注**（紫：RepNotify，蓝：Replicated，灰虚线：不复制），箭头后写“发给谁、触发哪个 OnRep”；③ 图下方的**网络摘要**由图中节点自动汇总：执行端、写了哪些复制变量、有哪些 RPC 和组播

读图四步

① 先看**红色事件节点的执行端标签**：它决定整条执行线在哪台机器上跑（执行线上的普通节点都跟着它）；② 再看**橙色调用节点上有没有 RPC 标签**，执行线到这里“跳”到另一台机器；③ 再看**Set 节点底部的复制标注**，它们是这段逻辑对其他端的“输出”——下一个网络帧发出，对方在 OnRep 里接住；④ 最后对一遍**网络摘要**。

1.1 执行端标签：这个节点在哪台机器上跑

标签	含义（Listen Server 双人局）	蓝图里从哪看出来	本书例子
仅服务器	只在服务器执行；Listen Server 下就是主机进程	GameMode / AIController 的所有事件；Event Possessed；Switch Has Authority 的 Authority 分支之后；只被服务器代码调用的普通自定义事件	GM.StartMatch、HandleDeath、FinishReload
Run on Server • Reliable	拥有者客户端调用 → 送到服务器执行；在服务器上调用 → 直接执行；非拥有者调用 → **被丢弃**	自定义事件 Details → Graph → **Replicates = Run on Server**，Reliable 勾选	Server_FireShot、Server_SetReady
Run on owning Client	在服务器上调用 → 送到这个 Actor 的拥有者客户端执行（拥有者是主机时就在主机本地执行）；没有拥有者时在服务器执行	Replicates = Run on owning Client	Client_StartSpectating、Client_CancelReload
Multicast • Unreliable	**组播**：在服务器上调用 → 服务器自己执行一次 + 每个与该 Actor 相关的客户端各执行一次；在客户端上调用 → 只在本地执行一次	Replicates = Multicast	Multicast_Growl、Multicast_DeathCry（僵尸的一次性音效）
RepNotify • 各端	OnRep_X 函数：客户端收到 X 的新值时执行；**蓝图里服务器 Set X 时也立即执行**	变量 Details → Replication = RepNotify，引擎自动生成 OnRep_X	OnRep_Phase、OnRep_Ammo
所有端	这个 Actor 在几台机器上有副本，就在几台各执行一次	BeginPlay、Tick、EndPlay、Construction Script	Character.BeginPlay
服务器 + 拥有者客户端	只有服务器和拥有者客户端触发，别人的机器不触发	Receive Controller Changed	角色添加输入映射

本地控制端	只在“控制这个 Pawn 的玩家”所在的机器触发	Enhanced Input 事件（IA_*）	IA_Fire、IA_Reload
本机 / 本机 UI	不联网的本地逻辑：谁调用就在谁那里执行，没有网络含义	GameInstance、Widget、纯表现函数	HostGame、HandlePhase、PlayFireCosmetics

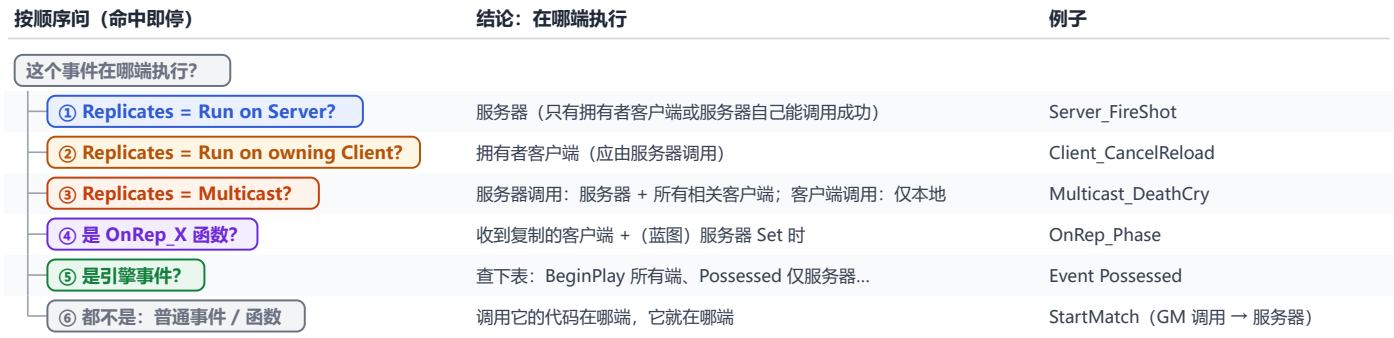


图 9-1a 判断一个事件在哪端执行。普通自定义事件（Replicates = Not Replicated）本身没有“端”的属性，它跟随调用者——这是最容易被忽略的一条

引擎事件 / 节点	服务器	拥有者客户端	其他客户端	备注
BeginPlay / Tick / EndPlay / Construction Script	✓	✓	✓	每个副本各一次
Event Possessed / Unpossessed	✓	✗	✗	Possess 只在服务器发生
Receive Controller Changed	✓	✓	✗	客户端在 Controller 复制到达时触发
Enhanced Input 事件（IA_*）	**仅主机自己的角色**	✓	✗	输入只在本地处理
Event AnyDamage / PointDamage	✓	✗	✗	在调用 ApplyDamage 的那一端触发；本书只在服务器调用
计时器事件（Set Timer by Event）	在**设置计时器的那一端**触发			本地开火循环在本机；换弹计时在服务器
Switch Has Authority → Authority	✓	✗	✗	对服务器生成的复制 Actor 成立；客户端自己生成的 Actor 在本地也是 Authority
Switch Has Authority → Remote	✗	✓	✓	—
Widget 事件（Construct、按钮 OnClicked）	只在创建它的那台机器			UI 永远不复制

表 9-1　常见引擎事件的执行端

1.2 复制标注：这个 Set 会不会、发给谁

复制标注条的读法：“⇄ 复制 • RepNotify • Owner Only → 拥有者 OnRep_Ammo”

- **⇄ 复制 • RepNotify**：这个变量在 Details → Replication 里选了 RepNotify。服务器 Set 之后，引擎在**下一个网络帧**把新值发出去；收到的客户端执行 OnRep_X；**蓝图里服务器这次 Set 本身也会立即执行一次 OnRep_X**（C++ 不会）。
- **⇄ 复制 • Replicated**：只复制、没有回调。适合“UI 每隔一会儿读一次”的值，例如结束时刻、僵尸数。
- ○ **不复制**：Replication = None，只改本端的这一份。服务器上的不复制变量（如 GameMode 的 SpawnPoints）客户端永远看不到；客户端上的（如 LocalShotSeq）服务器永远看不到。
- **Owner Only / Skip Owner / Initial Only**：Replication Condition，决定**发给哪些客户端**（表 8-4 给出了双人局里每个变量的接收方）。
- 三个前提：① 只有**服务器**上的写入会被复制，客户端 Set 复制变量只改了自己的副本；② 所属 Actor 必须勾 **Replicates**（组件还要勾 Component Replicates）；③ 只发最新值，一个网络帧里改多次只发最后一次。

组播在重做版里只剩 2 个

原 Demo 有 8 个 Multicast（开局、开火、换弹、受击、攻击、死亡…），重做后只有僵尸的两个一次性音效还用组播；其余全部换成 RepNotify。原因和逐一对照见第 8 章 5.1“组播去哪了”（表 8-5）。每张图下方的网络摘要都有一行“组播”，没有就写“无”。

2. 一局完整时序

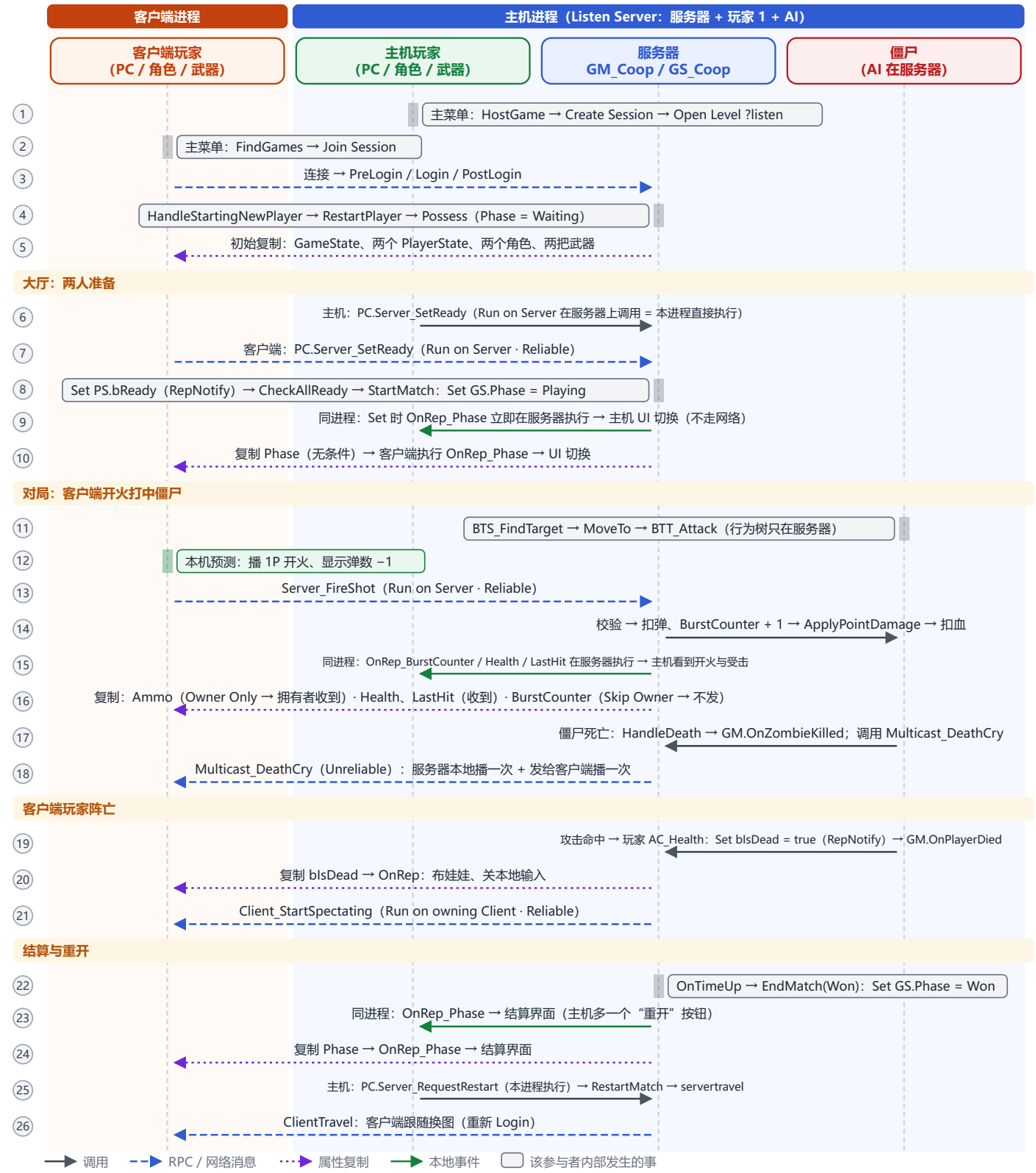


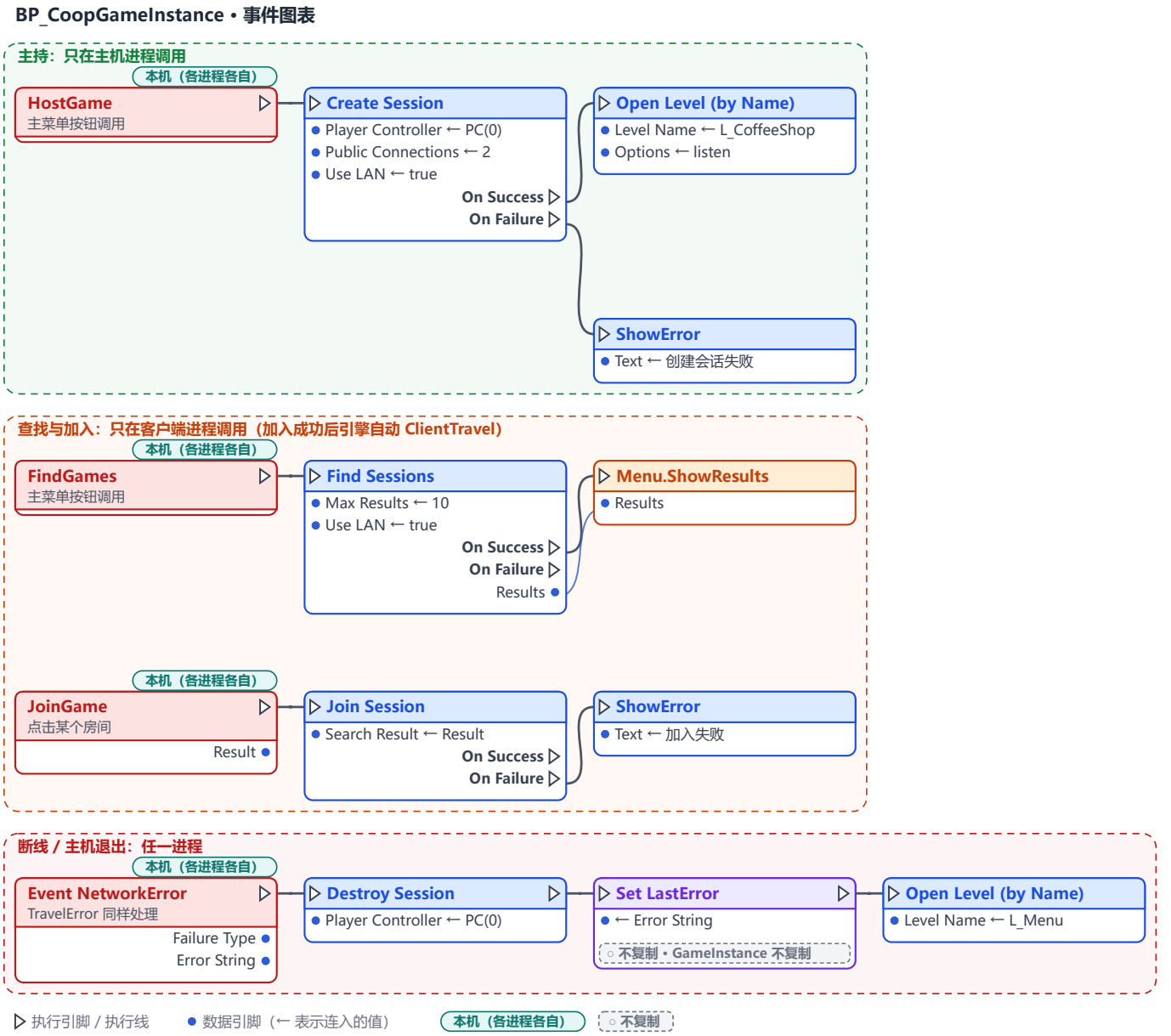
图 9-2 一局完整时序。左边一列是客户端进程，右边三列同属主机进程（顶部色条）。只有**蓝虚线（RPC）和紫点线（属性复制）**真正走网络；主机列收到的都是绿线——同一进程里，服务器 Set 时蓝图 OnRep 直接执行，**没有“复制给主机”这回事**

3. BP_CoopGameInstance：主持、加入、断线

项	内容
父类	GameInstance（项目设置 → Maps & Modes → Game Instance Class）
存在于	每个进程各一份；切地图不销毁；**不复制**
职责	会话：主持 / 查找 / 加入 / 离开；网络错误后回主菜单并提示
关键变量	LastError（Text，本地）——回到主菜单后由菜单读取显示
依赖	Online Subsystem（局域网测试用 Null：DefaultEngine.ini 中 `[OnlineSubsystem] DefaultPlatformService=Null`）



图 9-3a BP_CoopGameInstance 的网络规格：没有任何复制变量和 RPC，所有事件只在本机执行



本图网络摘要

▶ **执行端**	本机（各进程各自）：HostGame、FindGames、JoinGame、Event NetworkError
⇄ **写入的复制变量**	无
○ **只在本端的变量**	LastError
↗ **跨端调用（RPC）**	无
◎ **组播 NetMulticast**	无（持续状态用 RepNotify，一次性表现才用多播）

图 9-3　BP_CoopGameInstance。这里用 GetPlayerController(0) 是**对的**：GameInstance 只服务于本机，0 号就是本机玩家——同一个节点放在服务器逻辑里才是错的

面试注意：原 Demo 根本没有“联机入口”

原工程只在 PIE 里用“Play As Listen Server”跑过，没有任何会话节点，GameMode 里两个 `servertravel` 节点也没接线。打包后两台机器无法互联。合格的 Demo 至少要能“主持 → 局域网查找 → 加入 → 断线回菜单”。会话测试建议用两个 Standalone 进程或打包版本，PIE 单进程下的会话行为与真实情况不完全一致。

4. GM_Coop：规则只在这里

项	内容
父类	GameModeBase（想用自带比赛阶段可换成 GameMode，见下方提示）
存在于	**仅服务器**；客户端上 Get Game Mode 返回 None
类默认值	DefaultPawnClass = BP_CoopCharacter · PlayerControllerClass = PC_Coop · PlayerStateClass = PS_Coop · GameStateClass = GS_Coop · HUD = None
关键变量	MatchDuration = 180 · MaxAlive = 10 · MinPlayers = 2（单人测试改 1）· RespawnDelay = 3 · SpawnPoints（数组）· MatchTimer（句柄）
对外接口	CheckAllReady · OnZombieKilled(Zombie, Killer) · OnPlayerDied(PC) · RestartMatch——都只会被服务器上的对象调用

GM_Coop 父类 GameModeBase · 存在于：仅服务器（客户端上 Get Game Mode = None）

Class Defaults → Replication Replicates：✗ GameMode 从不复制，变量勾了 Replicated 也发不出去

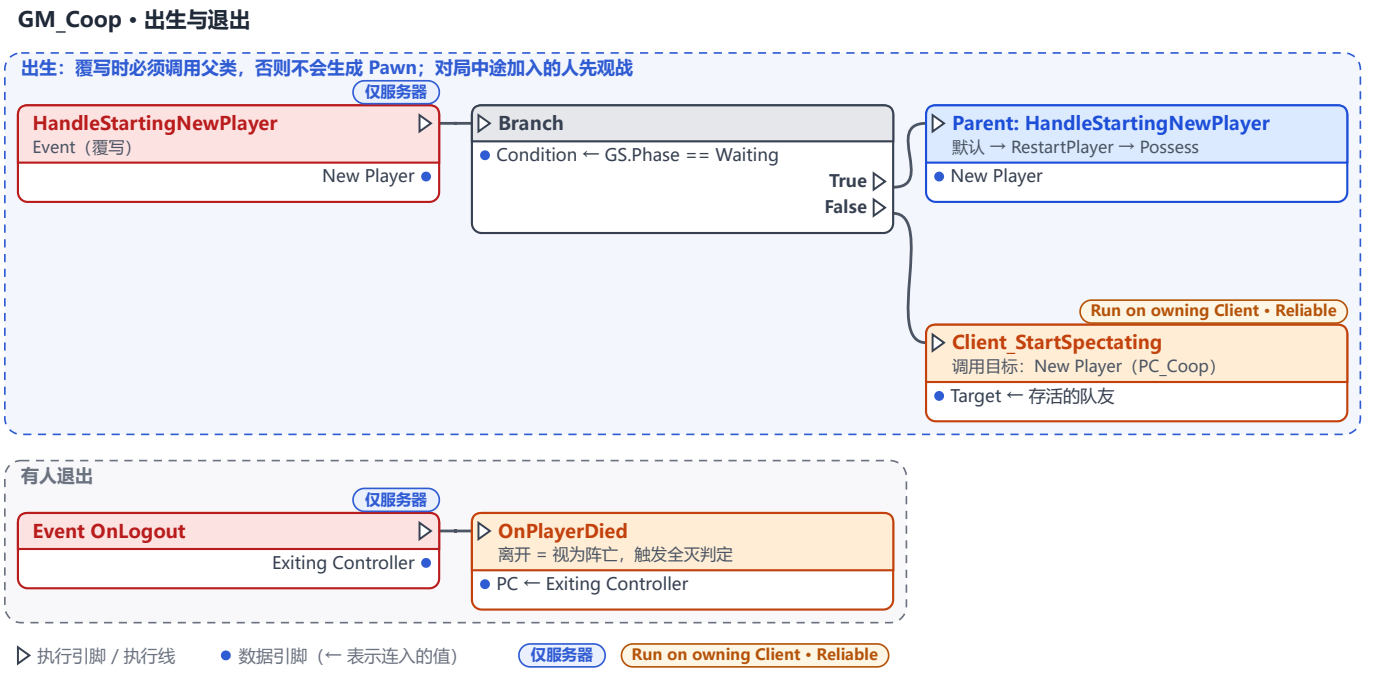
变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
MatchDuration	Float	None 不复制	—	配置	180 s
MaxAlive / MinPlayers	Integer	None 不复制	—	配置	10 / 2
RespawnDelay	Float	None 不复制	—	配置	3 s
SpawnPoints	BP_SpawnPoint[]	None 不复制	—	服务器	StartMatch 时收集一次
MatchTimer / SpawnTimer	Timer Handle	None 不复制	—	服务器	EndMatch 时清除

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
HandleStartingNewPlayer	覆写事件	引擎事件	—	仅服务器	引擎：玩家登录后
Event OnLogout	引擎事件	引擎事件	—	仅服务器	引擎：玩家断开
CheckAllReady	函数	函数	—	仅服务器	PC.Server_SetReady
StartMatch / EndMatch	自定义事件	Not Replicated	—	仅服务器	CheckAllReady / OnTimeUp
SpawnZombie	自定义事件	Not Replicated	—	仅服务器	StartMatch、计时器
OnZombieKilled	自定义事件	Not Replicated	—	仅服务器	BP_Zombie.HandleDeath
OnPlayerDied	自定义事件	Not Replicated	—	仅服务器	角色 HandleDeath、OnLogout
OnTimeUp	自定义事件	Not Replicated	—	仅服务器	MatchTimer
RestartMatch	自定义事件	Not Replicated	—	仅服务器	PC.Server_RequestRestart

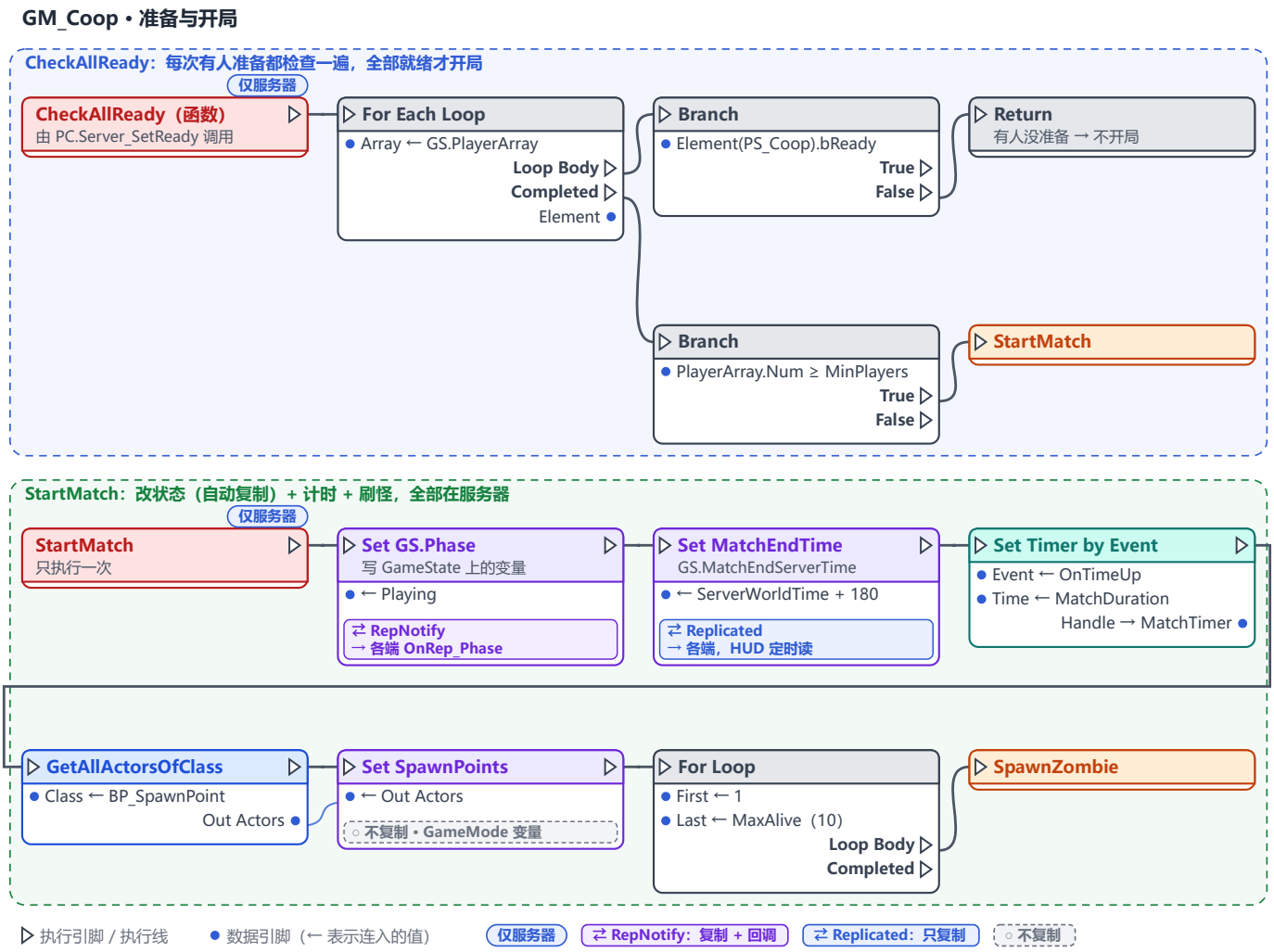
图 9-4a GM_Coop 的网络规格：全部变量不复制、全部事件只在服务器——它对外的唯一“输出”是去写 GameState / PlayerState 上的复制变量



本图网络摘要

▶ **执行端**	仅服务器：HandleStartingNewPlayer、Event OnLogout
⇄ **写入的复制变量**	无
○ **只在本端的变量**	无
↗ **跨端调用（RPC）**	Client_StartSpectating（Run on owning Client · Reliable）
◎ **组播 NetMulticast**	无（持续状态用 RepNotify，一次性表现才用多播）

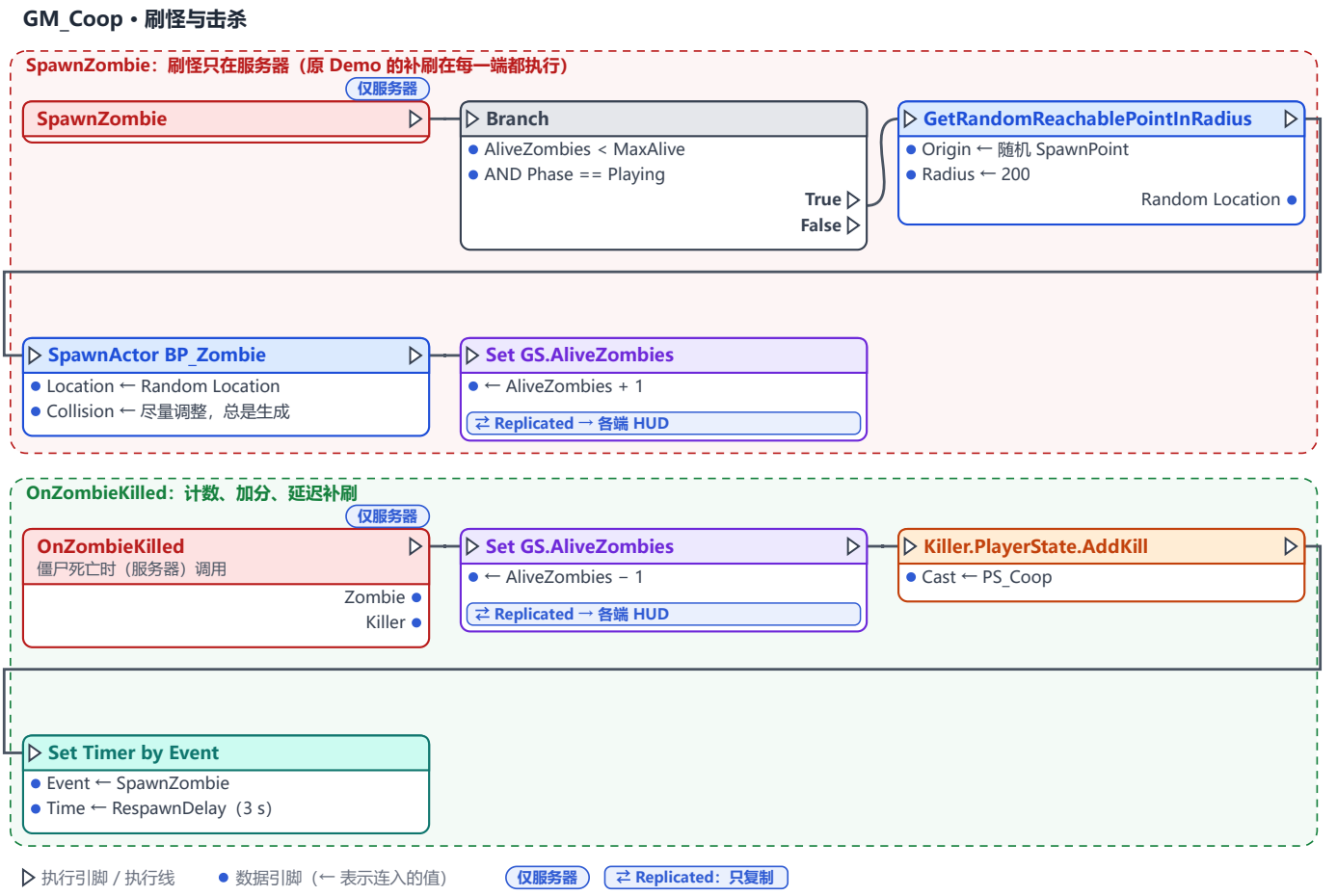
图 9-4 GM_Coop：出生与退出。HandleStartingNewPlayer 是 BlueprintNativeEvent，覆写后默认实现不再执行，需要显式调用 Parent 节点



本图网络摘要

▶ 执行端	仅服务器：CheckAllReady（函数）、StartMatch
⇄ 写入的复制变量	GS.Phase（RepNotify）、GS.MatchEndServerTime（Replicated）
○ 只在本端的变量	SpawnPoints
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 9-5 GM_Coop：准备与开局。只写 GameState 上的复制变量，不直接碰任何 UI——各端的界面会因为 OnRep_Phase 自己切换



本图网络摘要

▶ **执行端**	仅服务器：SpawnZombie、OnZombieKilled
⇄ **写入的复制变量**	GS.AliveZombies（Replicated）
○ **只在本端的变量**	无
↗ **跨端调用（RPC）**	无
◎ **组播 NetMulticast**	无（持续状态用 RepNotify，一次性表现才用多播）

图 9-6 GM_Coop：刷怪与击杀。僵尸不再自己调用“重新生成”，只把“我死了、谁杀的”报告给 GameMode，补不补刷由规则决定

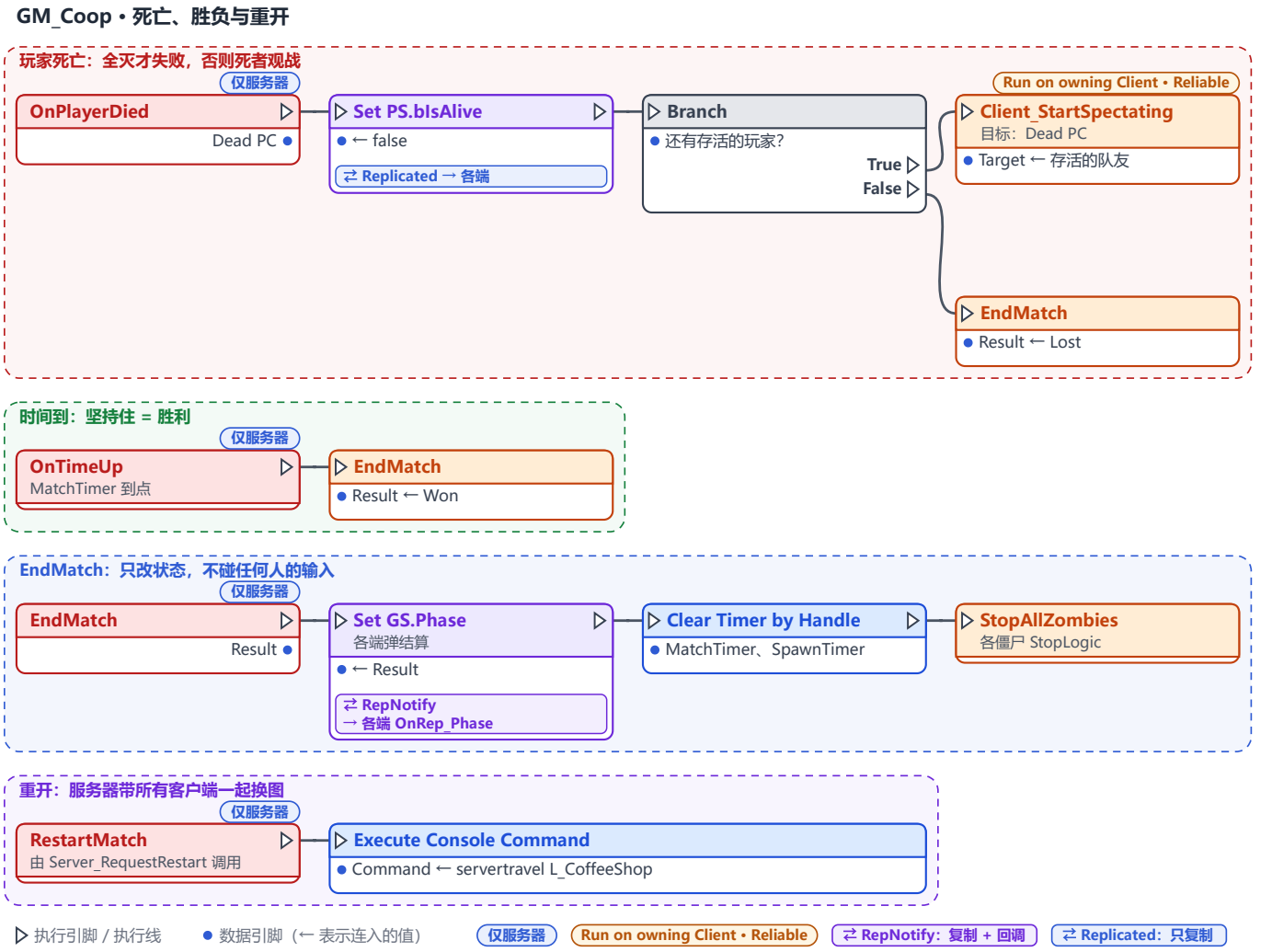


图 9-7　GM_Coop：死亡、胜负与重开。注意 EndMatch 里没有 DisableInput——服务器上禁用远端玩家的输入是无效的（输入在客户端处理）；“能不能再开火”由服务器在 Server_FireShot 里校验 Phase 决定，客户端界面由 OnRep_Phase 切换

GameModeBase 还是 GameMode?

`AGameMode`（不是 Base）自带一套比赛阶段 MatchState（WaitingToStart → InProgress → WaitingPostMatch），配合 `AGameState` 复制，有 `ReadyToStartMatch`、`StartMatch`、`EndMatch` 可以覆写，还提供断线玩家的 PlayerState 暂存（InactivePlayerArray）。本 Demo 用 GameModeBase + 自定义 Phase，是为了把流程画得一目了然；面试时提一句“正式项目可以直接用 AGameMode 的 MatchState”是加分项。

5. GS_Coop 与 PS_Coop：只存状态、只广播

项	GS_Coop	PS_Coop
父类	GameStateBase	PlayerState
存在于	所有端（Always Relevant）	所有端（每个玩家一份，GameState.PlayerArray 在每端都维护）
复制变量	Phase（RepNotify）· MatchEndServerTime · AliveZombies	bReady（RepNotify）· Kills（RepNotify）· bIsAlive
事件分发器	OnPhaseChanged(Phase) · OnPlayersChanged()	OnInfoChanged(Self)
谁写	只有 GM_Coop	只有服务器（GM 或 PC 的 Server RPC）



图 9-8a GS_Coop 的网络规格

PS_Coop 父类 PlayerState · 存在于：所有端（每个玩家一份）

Class Defaults → Replication　Replicates: ✓　Always Relevant: ✓ PlayerState 默认　Owner：PlayerController（这个玩家的）

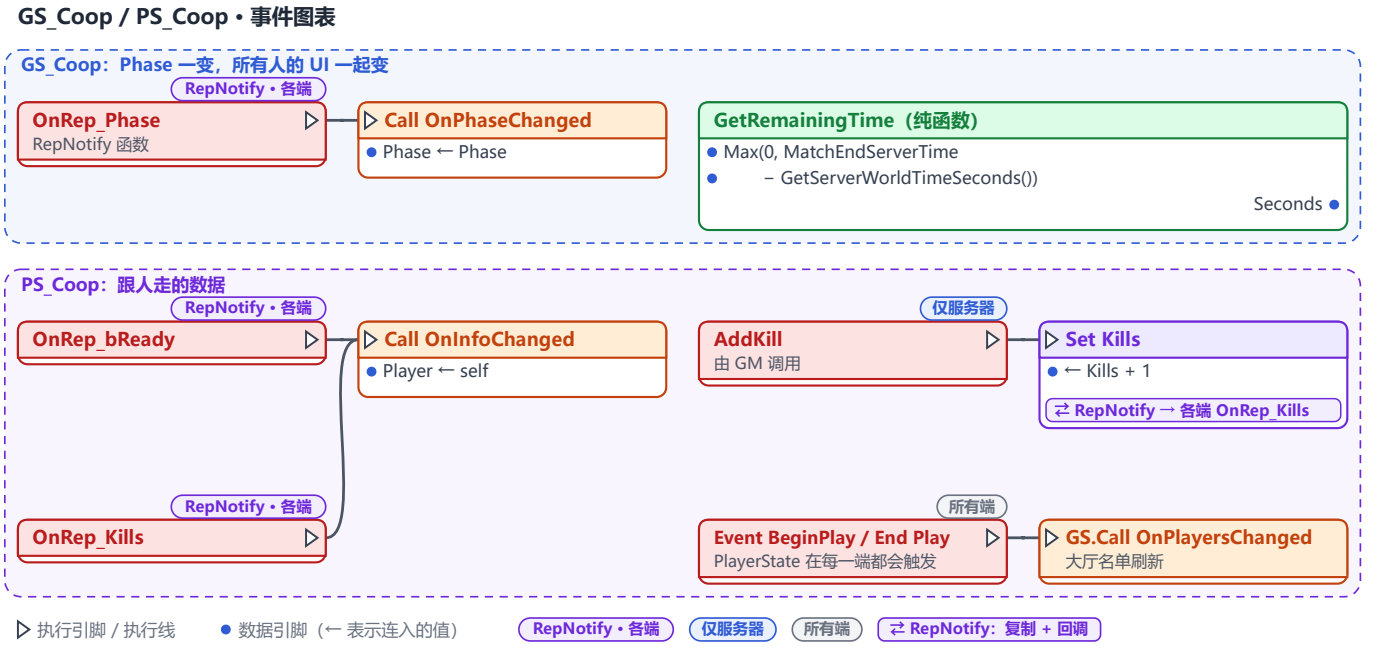
变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
bReady	Boolean	RepNotify	None（全部）	服务器（PC.Server_SetReady）	OnRep → OnInfoChanged → 大厅打勾
Kills	Integer	RepNotify	None（全部）	服务器（GM → AddKill）	OnRep → 记分板
bIsAlive	Boolean	Replicated	None（全部）	服务器（GM.OnPlayerDied）	僵尸选目标、HUD 队友状态
PlayerName（引擎）	String	Replicated	None（全部）	引擎	大厅、记分板显示

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
OnRep_bReady / OnRep_Kills	RepNotify 函数	RepNotify 函数	—	服务器(Set 时) + 收到的客户端	变量变化
AddKill	自定义事件	Not Replicated	—	仅服务器	GM.OnZombieKilled
Event BeginPlay / End Play	引擎事件	引擎事件	—	所有端（每个副本）	→ GS.OnPlayersChanged（名单刷新）

图 9-8b　PS_Coop 的网络规格。PlayerState 的 Owner 是 PlayerController，所以理论上可以在它上面定义 Server RPC；本书统一放在 PC 上，避免请求入口分散



本图网络摘要

▶ 执行端	RepNotify · 各端：OnRep_Phase、OnRep_bReady、OnRep_Kills；仅服务器：AddKill；所有端：Event BeginPlay / End Play
⇄ 写入的复制变量	Kills（RepNotify）
○ 只在本端的变量	无
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 9-8　GS_Coop / PS_Coop。三个 OnRep 都标“RepNotify · 各端”：客户端在收到新值时执行，主机在服务器 Set 的那一刻执行——所以主机的 UI 和客户端走同一条路径刷新，不需要额外通知

为什么复制“结束时刻”而不是“剩余秒数”

GetServerWorldTimeSeconds 由 GameState 周期性复制服务器时间（ServerWorldTimeSecondsUpdateFrequency，默认 0.1 s）得到差值估计，各端用同一条时间轴，显示平滑、只需复制一次结束时刻。它**没有扣除单程延迟**，客户端估计会偏慢约半个 RTT 并随抖动波动——用来显示倒计时足够，“时间到”的判定由服务器计时器决定；如果要拿它做命中回溯这种精度要求高的事，需要自己做带 RTT 补偿的时钟同步。

6. PC_Coop：本地界面与请求的出口

项	内容
父类	PlayerController
存在于	服务器（每个玩家一份）+ 它自己的客户端；**BeginPlay 在服务器上的远端 PC 也会执行**
职责	本机 UI 的创建与切换；暂停菜单；把 UI 的操作变成 Server RPC；接收观战等 Client RPC
关键变量（都不复制）	HUD、Lobby、Result、PauseMenu（Widget 引用）
RPC	Server_SetReady · Server_RequestRestart · Client_StartSpectating · Client_ShowHitMarker · Client_ShowDamageFeedback

PC_Coop 父类 PlayerController · 存在于：服务器（每个玩家一份）+ 它自己的客户端

Class Defaults → Replication Replicates: ✓ | Only Relevant to Owner: ✓ PC 默认：只复制给自己的客户端

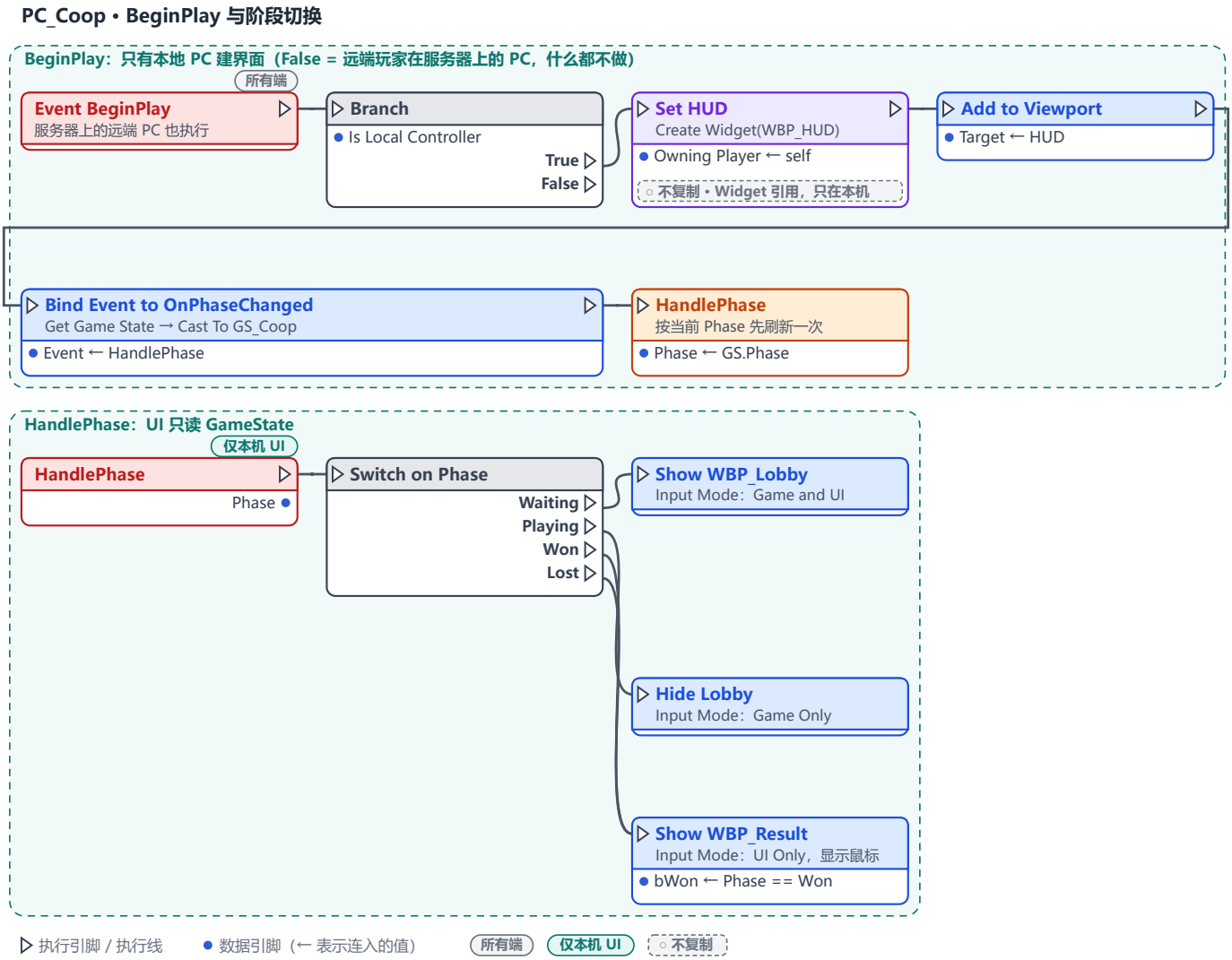
变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
HUD • Lobby • Result • Pause	Widget 引用	None 不复制	—	本机	只有本地 PC 创建；服务器上的远端 PC 里永远是空

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
Event BeginPlay	引擎事件	引擎事件	—	所有端（每个副本）	服务器上的远端 PC 也执行 → 先判断 Is Local Controller
HandlePhase	自定义事件	Not Replicated	—	本机 UI	GS.OnPhaseChanged
IA_Pause	输入事件	引擎事件	—	本地控制端	本机玩家按 Esc
Server_SetReady	自定义事件	Run on Server	✓	仅服务器	WBP_Lobby（Get Owning Player）
Server_RequestRestart	自定义事件	Run on Server	✓	仅服务器	WBP_Result
Client_StartSpectating	自定义事件	Run on owning Client	✓	仅拥有者客户端	GM（服务器）
Client_ShowHitMarker	自定义事件	Run on owning Client	✗	仅拥有者客户端	武器 Server_FireShot 命中后
Client_ShowDamageFeedback	自定义事件	Run on owning Client	✗	仅拥有者客户端	服务器上玩家受伤后

图 9-9a PC_Coop 的网络规格：所有“客户端 → 服务器”的 UI 请求从这里出（PC 一定属于这个客户端的连接）；所有“服务器 → 某个玩家”的通知也落在这里



本图网络摘要

▶ 执行端	所有端：Event BeginPlay；仅本机 UI：HandlePhase
⇄ 写入的复制变量	无
○ 只在本端的变量	HUD
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 9-9 PC_Coop：界面创建与阶段切换。客户端 PC 的 BeginPlay 发生在 World 开始游戏之后，GameState 已经到达，可以直接绑定；保险起见在 Cast 失败分支加一个短延迟重试

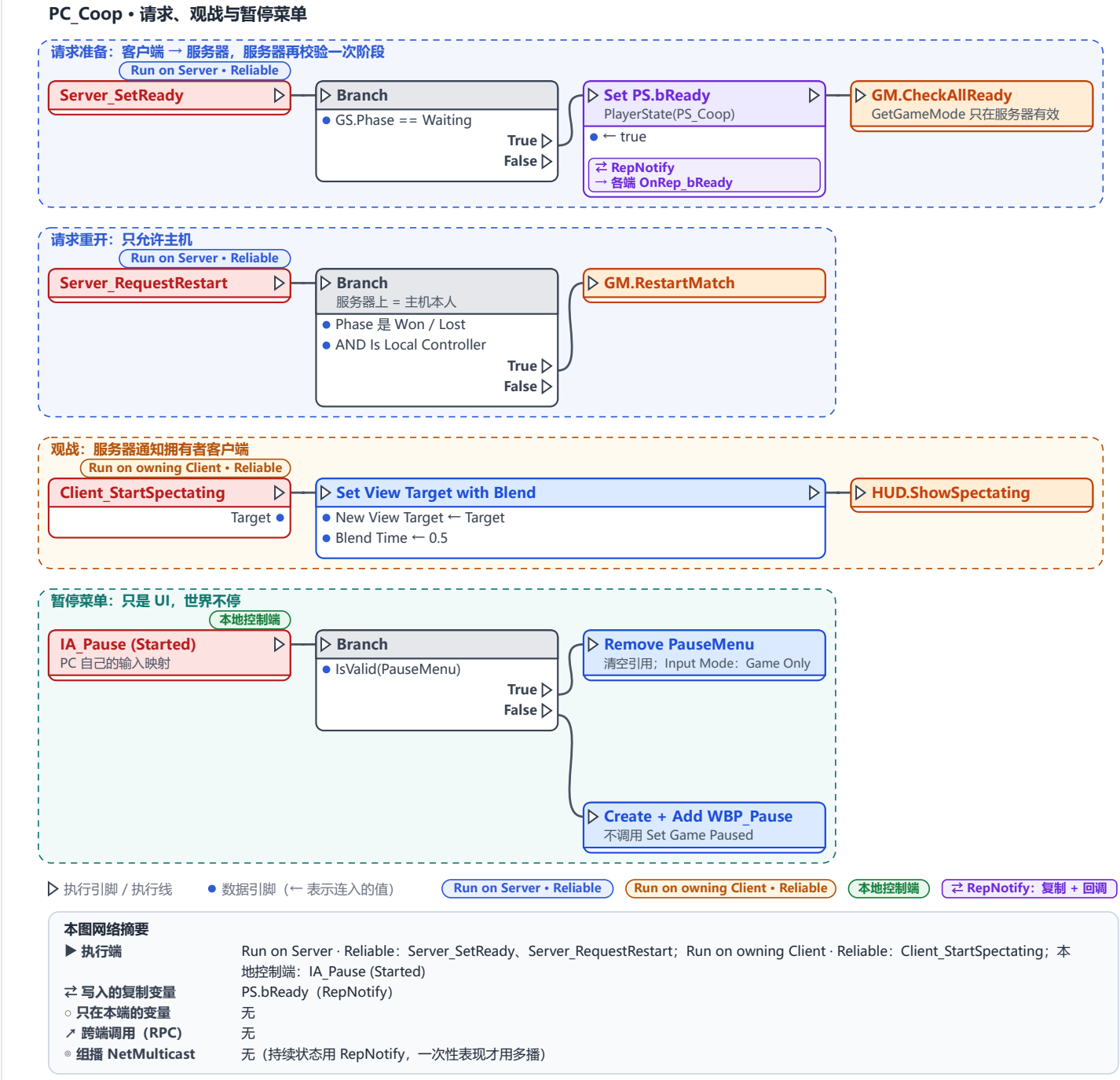


图 9-10 PC_Coop：请求、观战与暂停。所有 Server RPC 第一个节点就是校验；Client RPC 只做本机表现

7. 对照：原 Demo 的开局链路为什么“能跑”

原 Demo：从按任意键到开始（关卡蓝图 + GameMode + BP_Player）

```
关卡蓝图 BeginPlay（每一端）
  IsLocalController(GetPlayerController(0)) → 创建主菜单、随机机位、播 BGM
  Server_SetStart(false)                          // 关卡脚本不属于客户端连接：客户端调用被丢弃
  BP_Player(GetPlayerCharacter(0)).OnGameStart_Dispatch += Start_Logic
InputKey AnyKey（本地）→ DoOnce → DisableInput → BP_Player.Server_ReportReady()   // 角色上的 S
BP_Player.Server_ReportReady [Server] → GM_MyGame.AddReadyPlayer(GetController())
GM_MyGame.AddReadyPlayer → GameState.ReadyPlayersList.AddUnique(PC)          // 复制数组里存 PC
  人数到齐 → for PS in PlayerArray: Cast<BP_Player>(PS.GetPawn()).Multicast_StartGame()
BP_Player.Multicast_StartGame（每端、每个 Pawn 各一次）→ OnGameStart_Dispatch → 关卡蓝图 Start_Lo
Start_Logic（每一端）：切相机、建 InGame / Pause UI、刷怪点.开始游戏（内部 SwitchHasAuthority）、S
```

原 Demo 的做法	为什么“能跑”	哪里会坏	重做
准备 RPC 放在角色上	角色属于客户端连接 ✓	—	放到 PC（不依赖 Pawn 是否存在）
GameMode 上的 AddReadyPlayer 标成 Server RPC	在服务器上调用 = 普通函数	无意义的标志，误导读者	普通函数 CheckAllReady
“游戏是否开始”是 GameState 上的非复制布尔，由关卡蓝图多播设置	关卡脚本在客户端能收到多播	晚加入的人永远是 false，开不了火	GS.Phase 复制 + OnRep
开局表现靠每个 Pawn 一次 Multicast + 委托	本地角色恰好只绑定了一次	依赖“哪个 Pawn 绑了委托”，难以推理	各端 PC 监听 OnPhaseChanged
倒计时在 UMG 里 SetTimer 自减	两端各自能显示	两端结束时刻不同，各自判定胜利	复制结束时刻，服务器计时判定
任一玩家死亡：Multi_Death 在每端弹“失败”并暂停	—	活着的那个人也被判负	GM 判全灭，死者观战
重开：OpenLevel	单机能用	客户端点了就断开联机，主机点了踢掉客户端	servertravel
暂停：SetGamePaused	单机能用	客户端调用无效（SetPause 在客户端直接返回），主机调用冻结所有人	只开菜单，世界不停

本章速记

- GameInstance：会话与断线，本机专用，这里用 PC(0) 是对的。
- GM_Coop：HandleStartingNewPlayer（覆写要调父类）、CheckAllReady → StartMatch、SpawnZombie / OnZombieKilled、OnPlayerDied（全灭才输）、EndMatch（只改状态）、RestartMatch（servertravel）。
- GS / PS：只存复制状态、OnRep 里广播分发器；倒计时复制结束时刻。

- PC_Coop：BeginPlay 判断 IsLocalController 再建 UI；UI 操作 → Server RPC（先校验）；观战 → Client RPC。

▶ **自测：为什么 EndMatch 里不写“禁用所有玩家输入”？那玩家结算后还能开枪吗?**

▶ **自测：Server_RequestRestart 里为什么判断 IsLocalController 就能识别“主机”？**

CHAPTER 10

玩家层事件图表：BP_CoopCharacter（Player BP）/ AC_Health / AnimBP

Player BP 是原 Demo 里最“胖”的类：输入、相机、UI、冲刺、死亡、开局全在里面。重做后它只做三件事——把输入交给移动组件和武器、在服务器生成武器、在各端播自己的死亡表现；血量交给组件，界面交给 HUD。

本章目标

- 说出角色的组件结构，以及 1P / 3P 两套网格各自的可见性设置。
- 写出角色的 4 张事件图表：服务器初始化、本地初始化、输入、死亡表现。
- 写出 AC_Health 组件：只在服务器处理伤害，各端通过 OnRep 收到广播。
- 说清“冲刺 / 瞄准”在本机、服务器、别人眼里分别由什么驱动。

1. 类卡片与组件结构

项	内容
父类	ACoopCharacterBase（C++，只做一件事：把移动组件换成 UCoopMovementComponent）← ACharacter
存在于	所有端：拥有者客户端 AutonomousProxy，其他端 SimulatedProxy，服务器 Authority
类默认值	Use Controller Rotation Yaw = true，Pitch = false（原 Demo 是 true，整个胶囊会跟着低头抬头）；Replicates = true（Character 默认）
复制变量	CurrentWeapon（RepNotify）· bIsAiming（Skip Owner，给别人的 3P 姿势用）
事件分发器	OnWeaponChanged（各端）——HUD 据此重新订阅弹药
不再负责	血量（→ AC_Health）、UI 创建（→ PC_Coop）、开局（→ GM_Coop）、相机抖动 / 受击音效（→ PC 的 Client RPC 或 AC_Health.OnHit）

BP_CoopCharacter 父类 ACoopCharacterBase ← Character · 存在于：所有端（拥有者：AutonomousProxy；其他端：SimulatedProxy）

Class Defaults → Replication — Replicates: ✓ | Replicate Movement: ✓ Character 默认，由 CMC 负责 | Net Update Frequency: 100（默认）

变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
CurrentWeapon	Weapon 引用	RepNotify	None（全部）	服务器（Event Possessed）	OnRep_CurrentWeapon：各端把 3P / 1P 枪模挂到手上
bIsAiming	Boolean	Replicated	Skip Owner	服务器（移动组件解出瞄准标志后写）	别人的 3P 开镜姿势；拥有者自己用本地预测值
DefaultWeaponClass	Class	None 不复制	—	配置	Possessed 时生成
ReplicatedMovement（引擎）	FRepMovement	Replicated	SimOrPhysics	服务器（CMC）	COND_SimulatedOrPhysics：只发给模拟代理；拥有者靠移动预测，不收
RemoteViewPitch（引擎）	uint8	Replicated	Skip Owner	服务器	别人的俯仰（GetBaseAimRotation 读它）

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
Event Possessed	引擎事件	引擎事件	—	仅服务器	服务器 Possess 时
HandleDeath	自定义事件	Not Replicated	—	仅服务器	AC_Health.OnDeath（只在服务器绑定）
Event BeginPlay	引擎事件	引擎事件	—	所有端（每个副本）	每个副本各一次
ReceiveControllerChanged	引擎事件	引擎事件	—	服务器 + 拥有者客户端	Possess / Controller 复制到达
OnRep_CurrentWeapon	RepNotify 函数	RepNotify 函数	—	服务器(Set 时) + 收到的客户端	CurrentWeapon 变化
IA_Move / Look / Sprint / Aim / Fire / Reload	输入事件	引擎事件	—	本地控制端	本地玩家输入
PlayDeathVisual	自定义事件	Not Replicated	—	服务器(Set 时) + 收到的客户端	AC_Health.OnDeadVisual（在 OnRep 里广播）

图 10-1a BP_CoopCharacter 的网络规格。重做后角色上**没有任何 RPC、也没有组播**：冲刺 / 瞄准随移动包上行，开火 / 换弹的 RPC 在武器上，UI 请求在 PC 上；它只复制两个自己的变量

组件（层级 = 挂接关系）	关键设置	为什么
CapsuleComponent（根）	Pawn 通道 Block；Weapon 通道 Ignore	移动与碰撞的根；合作模式无友伤
Mesh（3P 全身，继承）	Owner No See = true；ABP_Coop3P	别人看到的你；自己看不到自己的身体
Weapon.Mesh3P	运行时挂到 hand_r 插槽	别人看到你手里的枪
FirstPersonCamera	Use Pawn Control Rotation = true	挂在胶囊眼高，跟随控制旋转
Mesh1P（手臂）	Only Owner See；Cast Shadow = false	只有自己看得到的贴脸手臂
Weapon.Mesh1P	运行时挂到 GripPoint 插槽	自己看到的枪
CharMoveComp	类型 = UCoopMovementComponent（C++）	冲刺 / 瞄准随移动包预测
AC_Health	Component Replicates = true；Max 100	玩家和僵尸共用一套伤害管线

图 10-1 BP_CoopCharacter 的组件结构。可见性规则：自己看 1P、别人看 3P；两把枪模由同一个武器 Actor 持有，分别挂到两套网格上

2. 服务器初始化：生成武器、订阅死亡

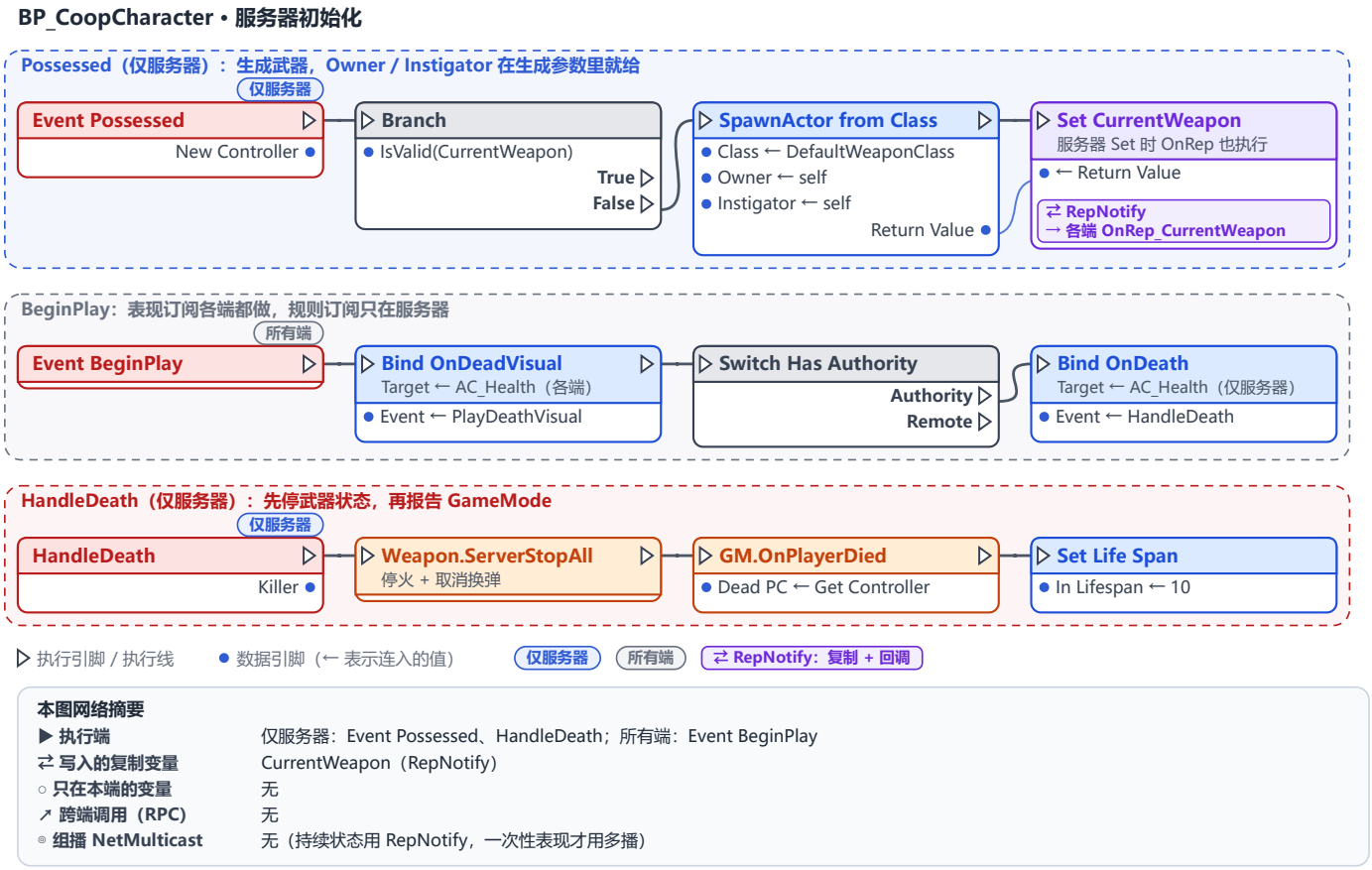


图 10-2 服务器初始化。a1 的 True 分支不接线：重生或重复 Possess 时武器已存在，不重复生成。原 Demo 先 SpawnActor(Owner=∅) 再 SetOwner，导致武器 BeginPlay 拿不到 Owner，又补了 for_server 函数

3. 本地初始化：输入映射、HUD、武器挂接

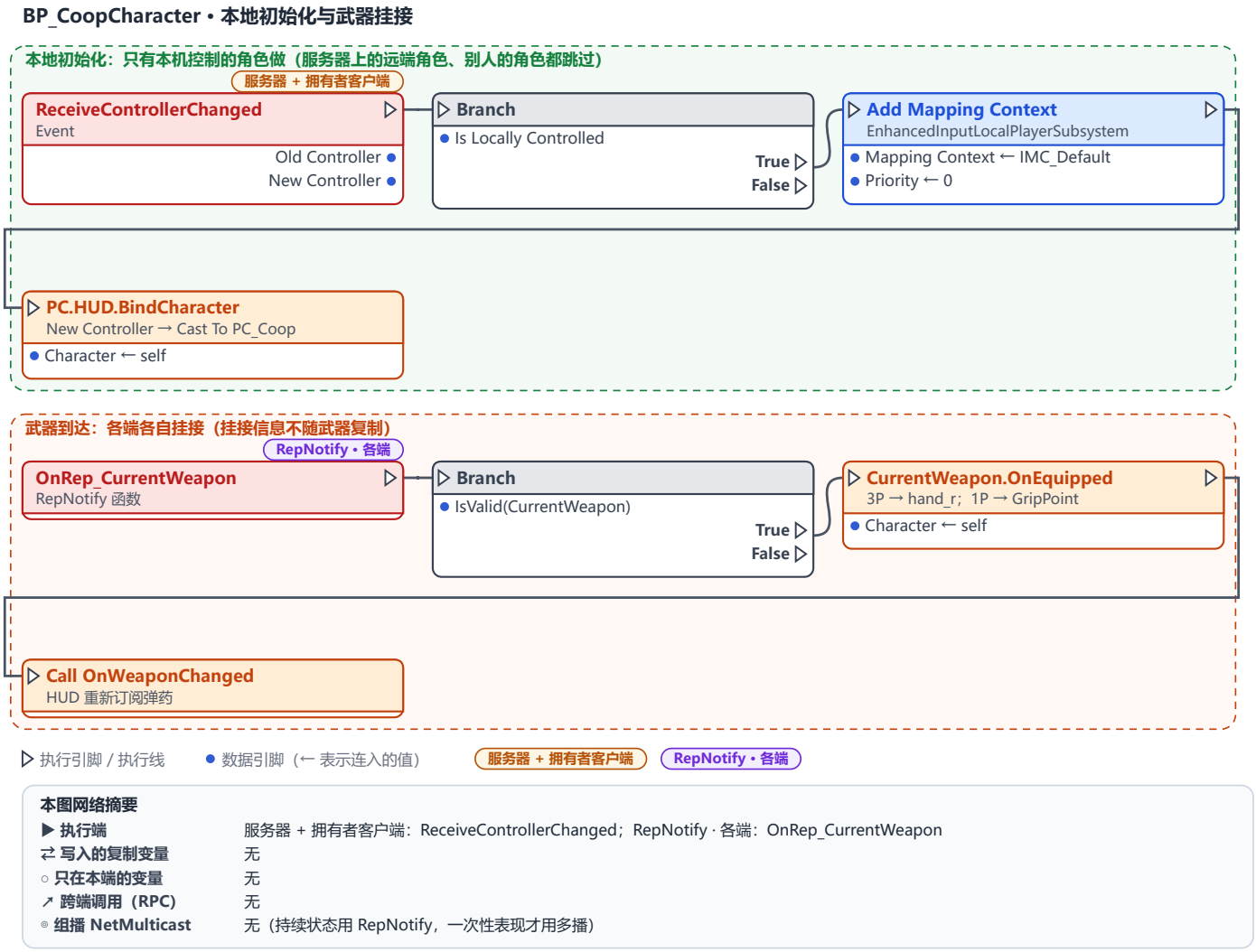
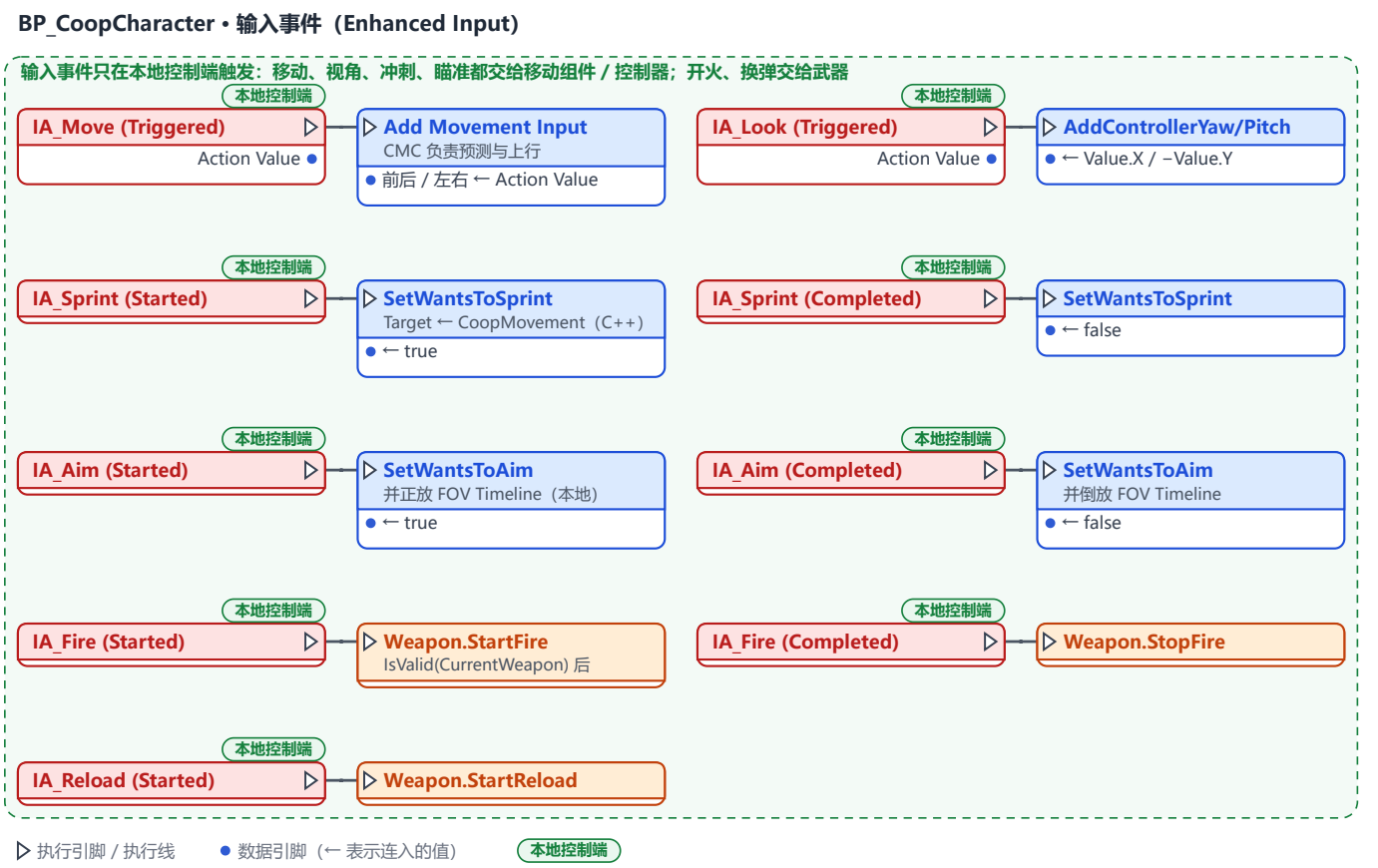


图 10-3 本地初始化。Receive Controller Changed 在服务器和拥有者客户端触发（客户端在 Controller 复制到达时），比 BeginPlay 更可靠；HUD 由 PC 创建，角色只负责把自己交给它

面试注意：为什么不在 BeginPlay 里加输入映射

BeginPlay 在每个副本上都会执行：主机上会执行两次（自己的角色 + 客户端玩家角色在服务器上的那份），客户端上也会执行两次。原 Demo 在 BeginPlay 里 `GetPlayerController(0) → Add Mapping Context`，每次加的都是本机玩家的映射，只是恰好重复添加无害。更大的问题是客户端 BeginPlay 时 Controller 可能还没到，`IsLocallyControlled` 为 false，判断会漏掉。Receive Controller Changed 正好在“控制者确定”的时刻触发。

4. 输入：全部在本地，一个 RPC 都不发



本图网络摘要

▶ **执行端**	本地控制端：IA_Move (Triggered)、IA_Look (Triggered)、IA_Sprint (Started)、IA_Sprint (Completed)、IA_Aim (Started)、IA_Aim (Completed)、IA_Fire (Started)、IA_Fire (Completed)、IA_Reload (Started)
⇄ **写入的复制变量**	无
○ **只在本端的变量**	无
↗ **跨端调用（RPC）**	无
◎ **组播 NetMulticast**	无（持续状态用 RepNotify，一次性表现才用多播）

图 10-4 输入。和原 Demo 对比：删掉了 Server_StartSprint / Server_StopSprint，删掉了 Tick 里两端各算 MaxWalkSpeed 的“速度计算”，删掉了写复制变量 MouseY；后坐力、FOV 都是本地表现

冲刺 / 瞄准在三个视角里分别由什么驱动

视角	移动速度	动画姿势	开火散布
本机（拥有者）	本地 bWantsToSprint / bWantsToAim → GetMaxSpeed（预测）	1P：本地值	本地值（预测）
服务器	每个移动包里的 CompressedFlags 解出同样的标志 → 同一个 GetMaxSpeed	—（Listen Server 主机看到的是 3P）	移动组件里的标志（权威，用于校验）
别人（模拟代理）	不预测，只平滑复制下来的位置和速度	3P：bIsAiming（Skip Owner 复制）+ 速度	—

UCoopMovementComponent：把标志同步给“别人看的”复制变量

```
void UCoopMovementComponent::SetWantsToAim(bool bAim) {            // 本地输入调用（BlueprintCa
    bWantsToAim = bAim;
    if (CharacterOwner && CharacterOwner->HasAuthority())          // 主机自己的角色：不走移动包
        Cast<ACoopCharacterBase>(CharacterOwner)->bIsAiming = bAim;
}
void UCoopMovementComponent::UpdateFromCompressedFlags(uint8 Flags) {   // 服务器：每个移动包
    Super::UpdateFromCompressedFlags(Flags);
    bWantsToSprint = (Flags & FSavedMove_Character::FLAG_Custom_0) != 0;
    bWantsToAim    = (Flags & FSavedMove_Character::FLAG_Custom_1) != 0;
    if (auto* C = Cast<ACoopCharacterBase>(CharacterOwner))
        C->bIsAiming = bWantsToAim;                                 // COND_SkipOwner：只给别
}
// SavedMove 子类与 GetMaxSpeed 见第 6 章。CompressedFlags 只有 FLAG_Custom_0~3 四个自定义位，
// 需要传更多数据（如精确的瞄准倍率）时改用 FCharacterNetworkMoveData 子类。
```

5. 死亡表现：各端执行，只有本机关输入

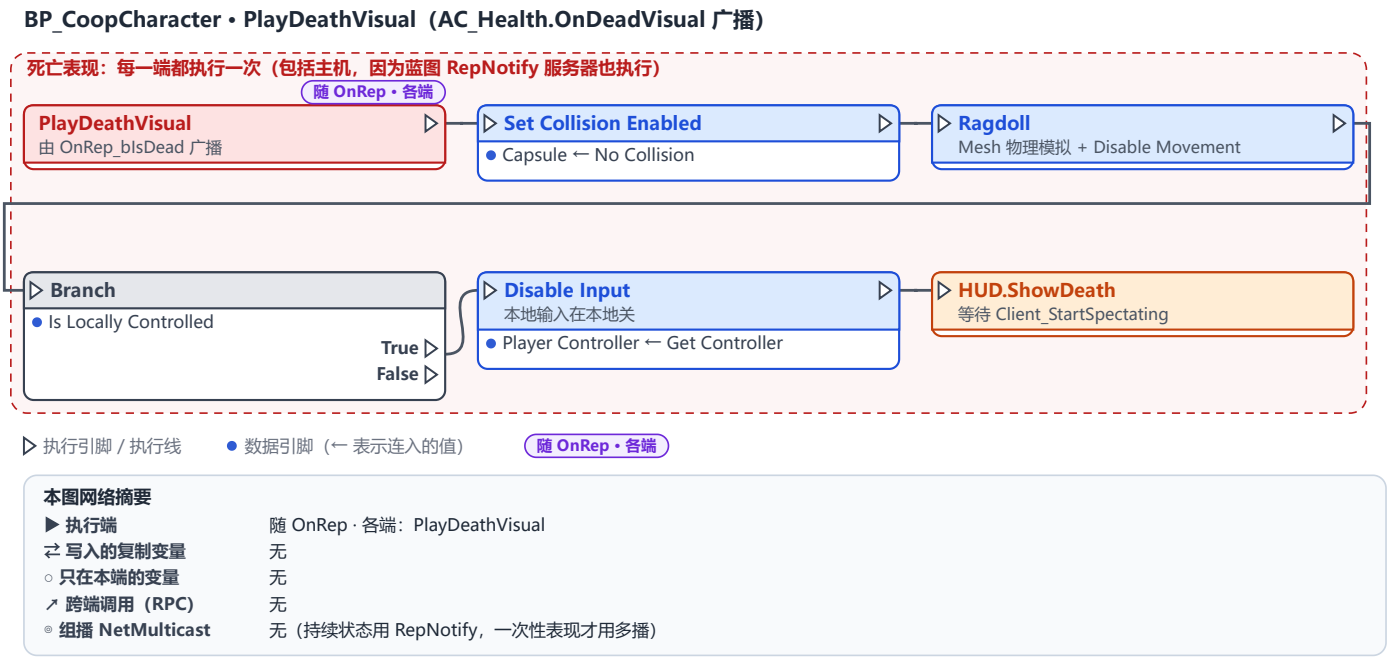


图 10-5　死亡表现。服务器那边不需要“禁用输入”：死者之后发来的所有 Server RPC 都会因为 bIsDead 被校验拒绝；客户端这边关掉本地输入，只是为了体验

面试注意：原 Demo 的“死亡禁用输入”到底发生了什么

原 Demo 在 `ReceiveAnyDamage`（只在服务器触发）里对死者自己调用 `DisableInput(GetPlayerController(0))`。当死的是**客户端玩家**时：目标 Pawn 是客户端玩家的角色，传入的却是主机的 PC——`APawn::DisableInput` 发现传入的不是自己的 Controller，只打一条 `LogPawn: Error: DisableInput can only be specified on a Pawn for its Controller`，什么也不做；而即使传对了，服务器上禁用输入也管不到客户端（输入在客户端处理）。结果是客户端玩家死后仍能移动、开火；同一段里的“停止射击”清的是服务器上并不存在的计时器（开火计时器跑在客户端）。只有主机自己死时 PC(0) 恰好是它的 Controller，表现才正常。真正让对局停下来的是 Multi_Death 里主机那份“3 秒后 SetGamePaused”——它把整个服务器连同还活着的队友一起冻结。

6. AC_Health：一个组件，两种角色共用

项	内容
父类	ActorComponent；细节面板勾选 **Component Replicates**（所属 Actor 也必须 Replicates）
复制变量	Health（RepNotify）· bIsDead（RepNotify）· LastHit：S_HitInfo {Counter, Point, bHeadshot}（RepNotify）
本地变量	MaxHealth（配置）· PrevHealth（算差值）
事件分发器	各端：OnHealthChanged(New, Delta)、OnHit(HitInfo)、OnDeadVisual()；仅服务器：OnDeath(Killer)
谁在用	BP_CoopCharacter、BP_Zombie；以后的油桶、可破坏物也能直接挂

AC_Health 父类 ActorComponent · 存在于：随所属 Actor（玩家、僵尸各挂一个）

Class Defaults → Replication — Component Replicates：✓ 必须勾 — 所属 Actor 的 Replicates：✓ 前提：Actor 不复制，组件也不会复制

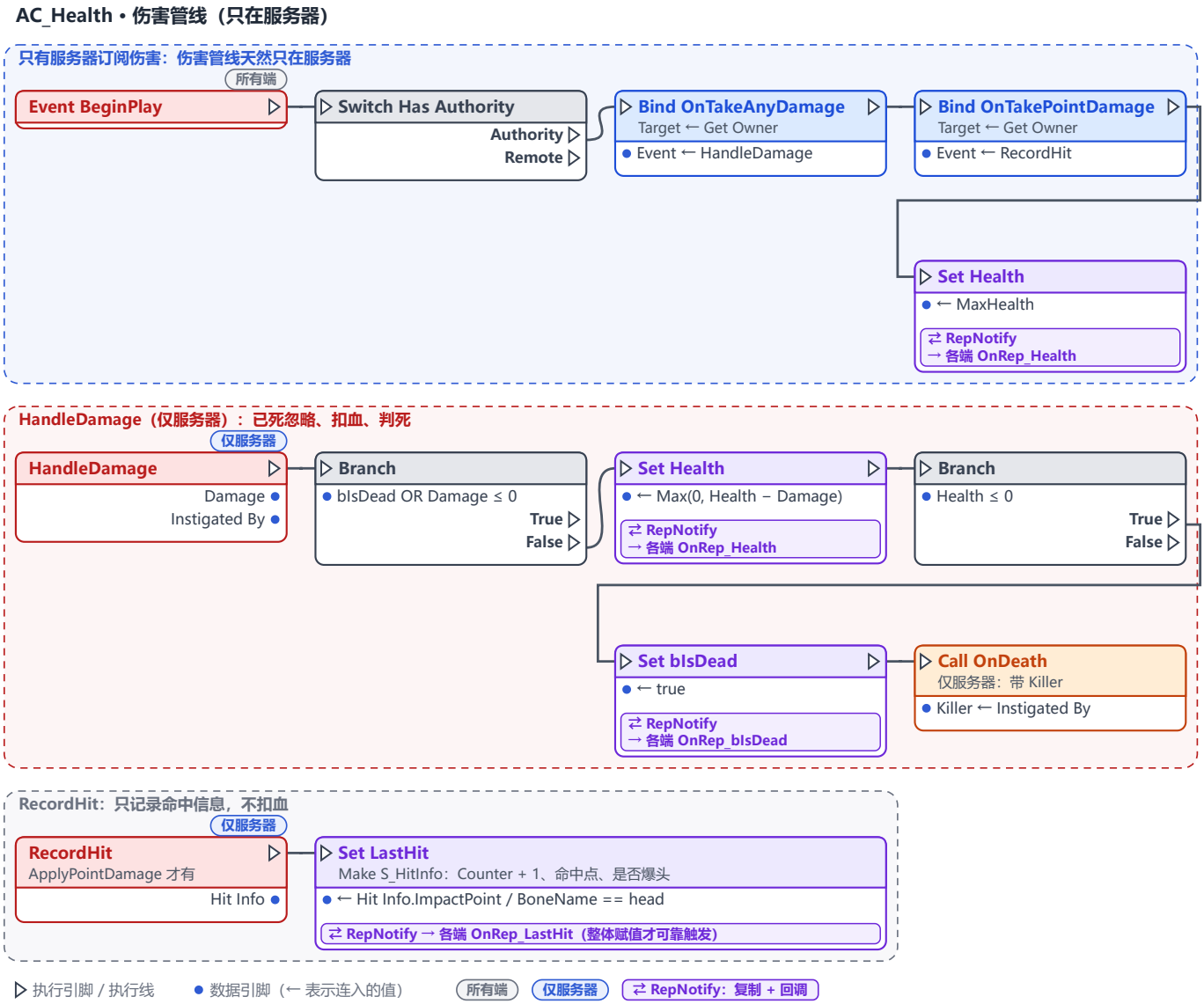
变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
Health	Float	RepNotify	None（全部）	服务器（HandleDamage）	OnRep → OnHealthChanged：血条、受伤后处理
bIsDead	Boolean	RepNotify	None（全部）	服务器（HandleDamage）	OnRep → OnDeadVisual：布娃娃、关碰撞、关本地输入
LastHit	S_HitInfo	RepNotify	None（全部）	服务器（RecordHit）	{Counter, Point, bHeadshot}；OnRep → OnHit：受击动作、血迹
MaxHealth	Float	None 不复制	—	配置	100
PrevHealth	Float	None 不复制	—	各端（在 OnRep 里）	算 Delta：只在本端有意义

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
Event BeginPlay	引擎事件	引擎事件	—	所有端（每个副本）	Authority 分支里才绑定伤害事件
HandleDamage	自定义事件	Not Replicated	—	仅服务器	Owner.OnTakeAnyDamage（只在服务器绑定）
RecordHit	自定义事件	Not Replicated	—	仅服务器	Owner.OnTakePointDamage
OnRep_Health / LastHit / bIsDead	RepNotify 函数	RepNotify 函数	—	服务器(Set 时) + 收到的客户端	变量变化
OnHealthChanged / OnHit / OnDeadVisual	事件分发器	事件分发器	—	服务器(Set 时) + 收到的客户端	在 OnRep 里广播
OnDeath	事件分发器	事件分发器	—	仅服务器	HandleDamage 判死 → 角色 / 僵尸 HandleDeath

图 10-6a AC_Health 的网络规格。规则（扣血、判死、OnDeath）只在服务器；表现（三个 OnRep 和它们广播的分发器）在所有端。没有任何 RPC



本图网络摘要

▶ 执行端	所有端：Event BeginPlay；仅服务器：HandleDamage、RecordHit
⇄ 写入的复制变量	Health（RepNotify）、bIsDead（RepNotify）、LastHit（RepNotify）
○ 只在本端的变量	无
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 10-6 AC_Health 伤害管线。ApplyPointDamage 会同时触发 AnyDamage 与 PointDamage：血量只在 AnyDamage 里扣（僵尸攻击用的 ApplyDamage 也走这里），PointDamage 只记命中点，避免扣两次

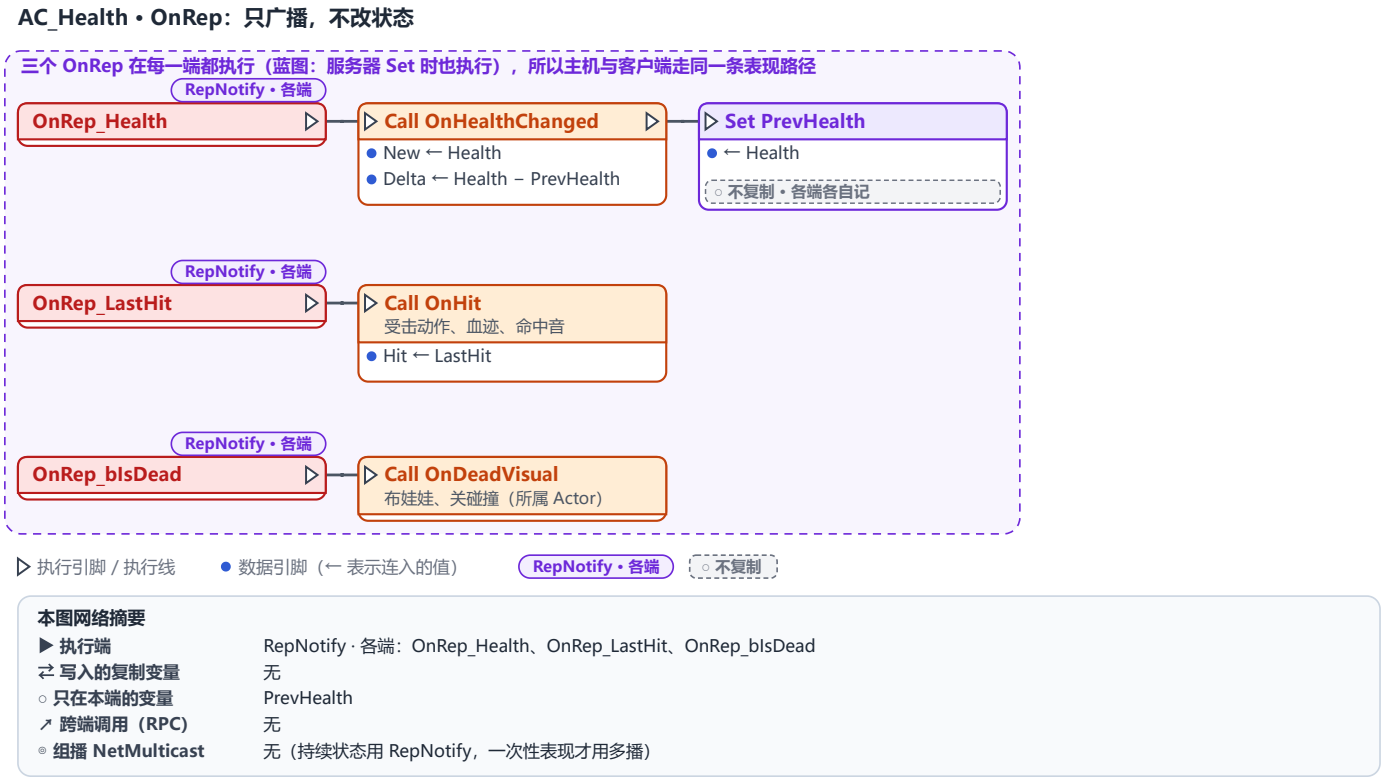


图 10-7 AC_Health 的表现广播。用计数器 LastHit.Counter 表达“又被打中一次”：同一网络帧被打两次也能被识别为变化

蓝图 RepNotify 在服务器也执行：怎么利用，怎么避坑

- **利用**：把“状态变了之后的表现”只写在 OnRep 里——服务器 Set 时它自己跑一次，客户端收到时再跑一次，主机和客户端的表现走同一份代码，不需要“服务器手动调一次”。
- **避坑 1**：不要在 OnRep 里再写这个复制变量或做规则判断（例如 OnRep_Health 里判死），否则服务器会重复执行规则；规则留在 HandleDamage。
- **避坑 2**：Dedicated Server 上 OnRep 里的纯表现会白跑，必要时先判断 `Is Dedicated Server`。
- **避坑 3**：“Set Members in Struct” 改结构体成员不一定触发服务器端的 RepNotify，结构体复制变量用 **Make → Set 整体赋值**。
- **C++ 对照**：`ReplicatedUsing` 的 OnRep 服务器不执行，需要在改值后手动调用一次。

7. AnimBP：只读、线程安全

ABP_Coop3P / ABP_Coop1P 父类 AnimInstance · 存在于：每个骨骼网格各一份，各端本地

Class Defaults → Replication　Replicates：✗ AnimInstance 不是 Actor，不能复制

变量（My Blueprint → Variables；Details → Replication / Replication Condition）

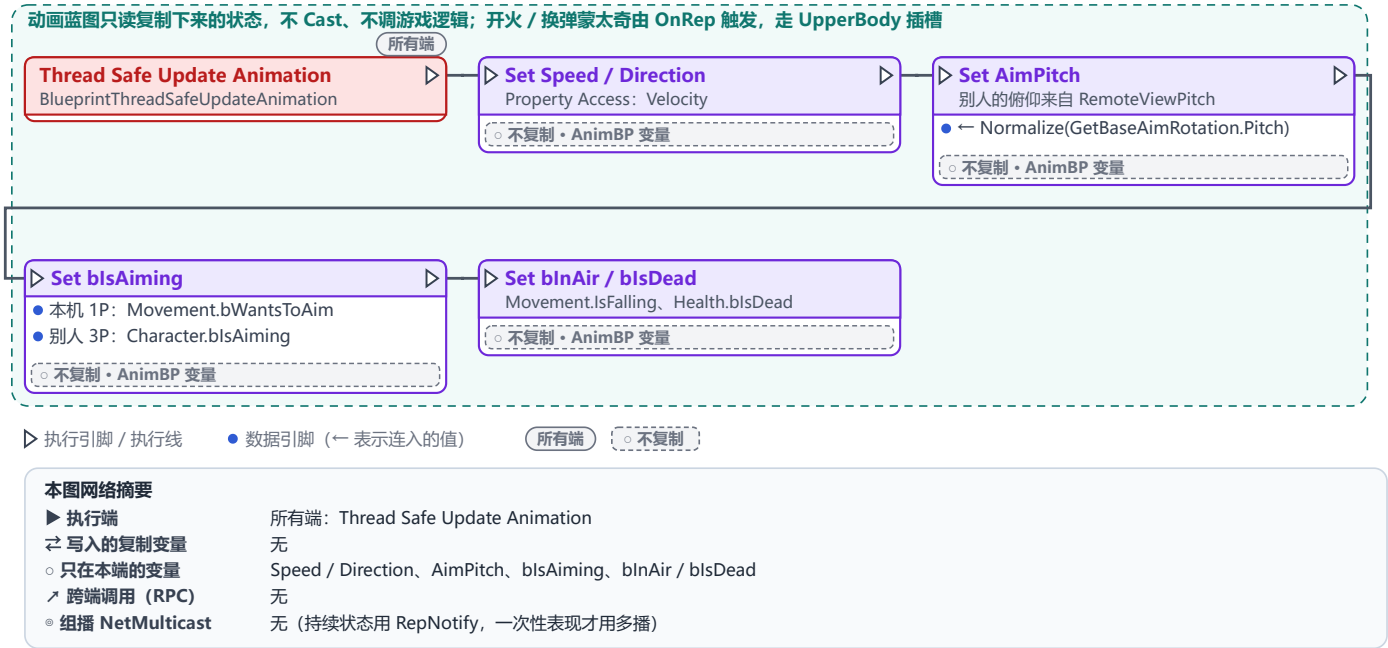
名称	类型	Replication	Condition	谁写	收到后 / 说明
Speed / Direction	Float	None 不复制	—	每端（读角色速度）	模拟代理的速度来自 ReplicatedMovement
AimPitch	Float	None 不复制	—	每端	别人：RemoteViewPitch；自己：控制旋转
bIsAiming	Boolean	None 不复制	—	每端	1P 读本地预测值；3P 读角色复制的 bIsAiming
bInAir / bIsDead	Boolean	None 不复制	—	每端	读 CMC 与 AC_Health 的复制状态

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
Thread Safe Update Animation	引擎事件	引擎事件	—	所有端（每个副本）	每帧（工作线程）

图 10-8a　动画蓝图的网络规格：它本身什么都不复制，只“读”已经复制下来的角色状态。动画要同步，就同步驱动动画的那几个状态

ABP_Coop3P • Thread Safe Update Animation



本图网络摘要

▶ 执行端	所有端：Thread Safe Update Animation
⇄ 写入的复制变量	无
○ 只在本端的变量	Speed / Direction、AimPitch、bIsAiming、bInAir / bIsDead
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 10-8　ABP_Coop3P。原 Demo 的 AnimBP 在 EventGraph（游戏线程）里 Cast 角色、读 MouseX / MouseY 复制变量做武器摇摆；重做后摇摆只在 1P 动画里用本地输入，3P 只需要俯仰与速度

8. 原 Demo 对照

环节	原 Demo	问题	重做
网格	只有 1P 手臂；胶囊随 Pitch 旋转	对方看到漂浮的手臂，模型随低头倾斜	1P + 3P，身体只跟 Yaw，AimOffset 处理俯仰
输入映射	BeginPlay 里 GetPlayerController(0)	每个副本都执行；客户端时序不可靠	Receive Controller Changed + IsLocallyControlled
移动速度	Tick 里两端各自 FInterpTo 写 MaxWalkSpeed	依赖只在一端存在的标志，持续被纠正（E6）	GetMaxSpeed 读预测标志
冲刺	Server RPC 改一个不复制的布尔	本机永远不冲刺、服务器在冲刺	SetWantsToSprint（进 SavedMove）
瞄准	只改本机 AnimBP 的变量	服务器不知道：速度、散布都不一致	SetWantsToAim + bIsAiming（Skip Owner）
视角输入	MouseY 是复制变量，在客户端写	客户端写复制变量无效且误导	只调 AddControllerPitchInput；别人用 RemoteViewPitch
血量	HP 复制、OnRep 为空；UI 每帧 Cast 读	无事件、无受击表现	AC_Health + OnHealthChanged
死亡	ReceiveAnyDamage 里一长串：DisableInput(PC0)、停止射击、关碰撞、惨叫、Server_Death → 多播失败界面	客户端死后仍能操作；活着的人也被判负	服务器：HandleDeath → GM；各端：PlayDeathVisual；本机关输入

受击反馈	Client_PlayHitFX（镜头抖动、音效、后处理）	✓ 用法正确	保留为 PC_Coop.Client_ShowDamageFeedback

本章速记

- Player BP 只做三件事：**输入转交**（移动组件 / 武器）、**服务器生成武器**（Possessed）、**各端死亡表现**（OnDeadVisual）。
- 本地初始化放 Receive Controller Changed + IsLocallyControlled；武器挂接放 OnRep_CurrentWeapon。
- 冲刺 / 瞄准：本机与服务器用移动包里的同一组标志算速度；别人看姿势用 Skip Owner 复制的 bIsAiming。
- AC_Health：服务器订阅 AnyDamage 扣血、PointDamage 记命中；三个 OnRep 只广播；OnDeath 只在服务器。
- “禁用输入”只能在本地做；服务器靠校验拒绝死者的请求。

▶ **自测：血量为什么做成组件而不是写在 Character 里?**

▶ **自测：为什么死者的“禁用输入”不放在服务器的 HandleDeath 里?**

CHAPTER 11

武器层事件图表：BP_WeaponBase

武器是全书技术密度最高的类：同一个 Actor 在本机要预测、在服务器要裁决、在别人那里要表现。按执行端把它拆成 5 张图——本地开火、服务器处理一发、开火表现、换弹、弹药对账——每张图只在一种“端”上跑。

本章目标

- 说出武器的变量表：哪些复制、什么条件、哪些只在本地。
- 逐节点讲清本地开火、服务器校验、表现播放、换弹状态机、弹药对账。
- 知道纯蓝图实现的三个细节：OnRep 没有旧值、结构体整体赋值、用 Set Timer by Event 代替 Delay。

1. 类卡片与变量表

项	内容
父类	Actor；Replicates = true，Replicate Movement = false，**Net Use Owner Relevancy = true**
组件	Root · Mesh3P（Owner No See）· Mesh1P（Only Owner See，Cast Shadow 关）· Muzzle（Arrow，挂在 Mesh1P 枪口插槽）
Owner / Instigator	持有它的 BP_CoopCharacter（生成参数里给）
数据	Data：DA_Weapon（伤害、射速、弹匣、换弹时长、散布、蒙太奇与特效的软引用）
对外接口	本地：StartFire / StopFire / StartReload / GetDisplayedAmmo；服务器：ServerStopAll；事件分发器：OnAmmoChanged

BP_WeaponBase 父类 Actor · 存在于：所有端（Owner = 持有它的角色）

Class Defaults → Replication　Replicates: ✓　Replicate Movement: ✗ 挂在手上，跟着角色走　Net Use Owner Relevancy: ✓ 相关性跟随持有者　Owner / Instigator: 持有者角色（生成参数里给）

变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
Ammo	S_AmmoState	RepNotify	Owner Only	服务器（Server_FireShot、FinishReload）	{InClip, Reserve, AckSeq, ReloadCount}；拥有者在 OnRep 里对账
BurstCounter	Integer	RepNotify	Skip Owner	服务器（每接受一发 +1）	其他端在 OnRep 里播 3P 开火
bPendingReload	Boolean	RepNotify	Skip Owner	服务器（开始 / 完成 / 取消换弹）	其他端在 OnRep 里播 / 停 3P 换弹
LocalShotSeq	Integer	None 不复制	—	本机（FireShot）	本机已发出的开火序号
bLocalReloading / LocalReloadBase	Boolean / Integer	None 不复制	—	本机	预测“正在换弹”，记下发起时的 ReloadCount
LastSeenBurst / bBegunPlay	Integer / Boolean	None 不复制	—	各端	蓝图 OnRep 没有旧值参数；过滤晚加入的首包
ShotBudget / LastBudgetTime	Float	None 不复制	—	服务器	射速令牌桶
bWantsToFire / RefireTimer / ReloadTimer	Bool / Timer	None 不复制	—	本机 / 服务器	开火循环（本机）、换弹计时（服务器）
Data	DA_Weapon	None 不复制	—	配置	伤害、射速、弹匣、换弹时长、散布

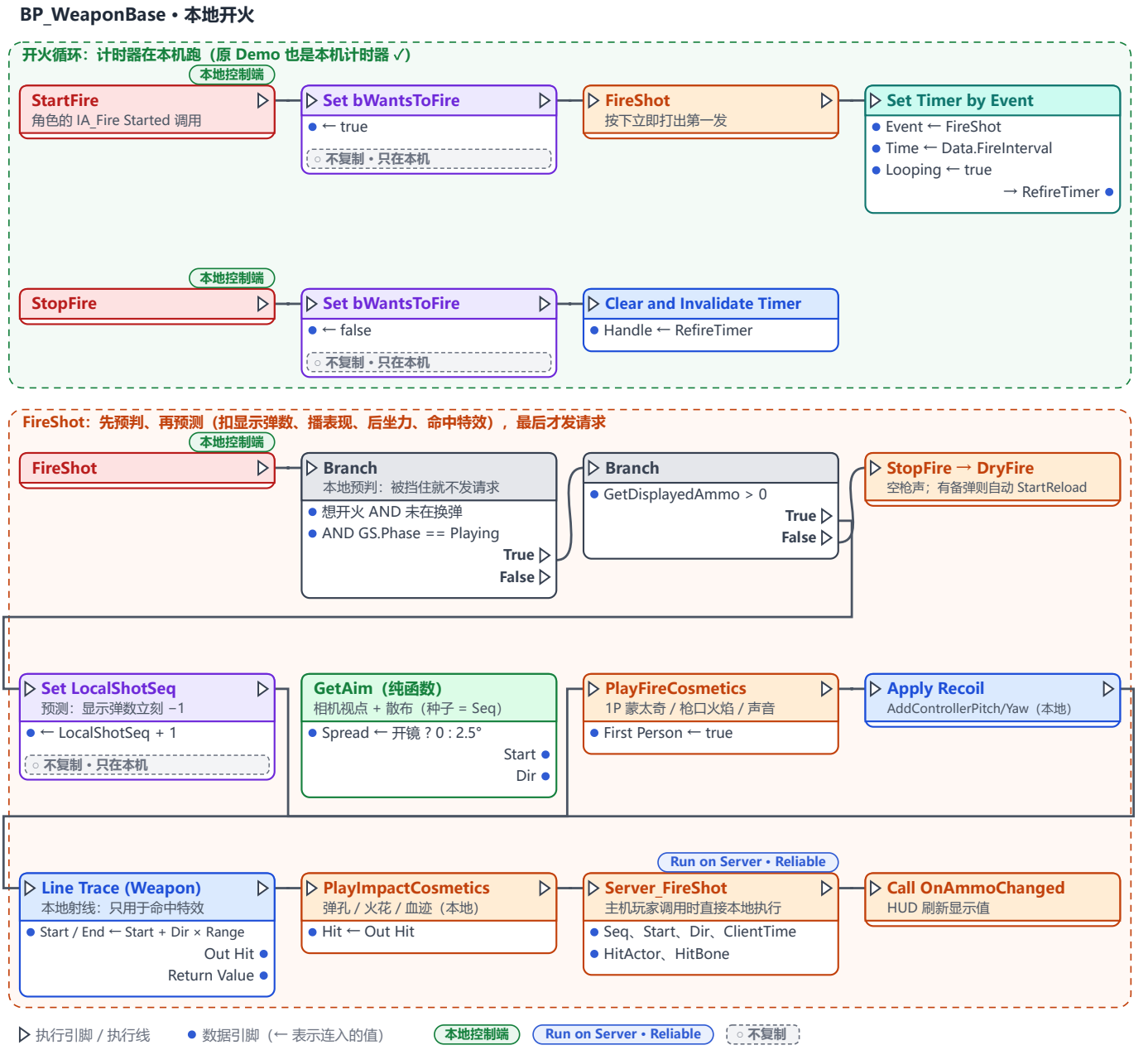
事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
StartFire / StopFire / FireShot	自定义事件	Not Replicated	—	本地控制端	角色输入、本机计时器
Server_FireShot	自定义事件	Run on Server	✓	仅服务器	FireShot（拥有者本机）
Server_StartReload	自定义事件	Run on Server	✓	仅服务器	StartReload（拥有者本机）
Client_CancelReload	自定义事件	Run on owning Client	✓	仅拥有者客户端	服务器拒绝换弹 / CancelReload
FinishReload / CancelReload / ServerStopAll	自定义事件	Not Replicated	—	仅服务器	换弹计时器 / 角色 HandleDeath
OnRep_Ammo	RepNotify 函数	RepNotify 函数	—	服务器(Set 时) + 收到的客户端	拥有者收到时；服务器 Set 时
OnRep_BurstCounter / OnRep_bPendingReload	RepNotify 函数	RepNotify 函数	—	服务器(Set 时) + 收到的客户端	其他客户端收到时；服务器 Set 时（要过滤拥有者）
PlayFireCosmetics / PlayReloadCosmetics	自定义事件	Not Replicated	—	本机（不联网）	本机预测（1P）或 OnRep（3P）
Event BeginPlay	引擎事件	引擎事件	—	所有端（每个副本）	每个副本：Set bBegunPlay

图 11-0　BP_WeaponBase 的网络规格（取代原来的变量表）。只复制 3 个变量：一个给拥有者（Owner Only），两个给别人（Skip Owner）；2 个 Server RPC + 1 个 Client RPC；**没有组播**——原 Demo 的 Multicast_Fire / Multicast_Change 分别被 BurstCounter / bPendingReload 取代

原 Demo 的 5 个弹药变量全部无条件复制给所有人；重做后只有一个 Owner Only 结构体，加上两个 Skip Owner 的“给别人看”的状态。它们在双人局里分别发给谁，见第 8 章表 8-4。

2. 本地开火：本机控制端（含主机玩家）



本图网络摘要

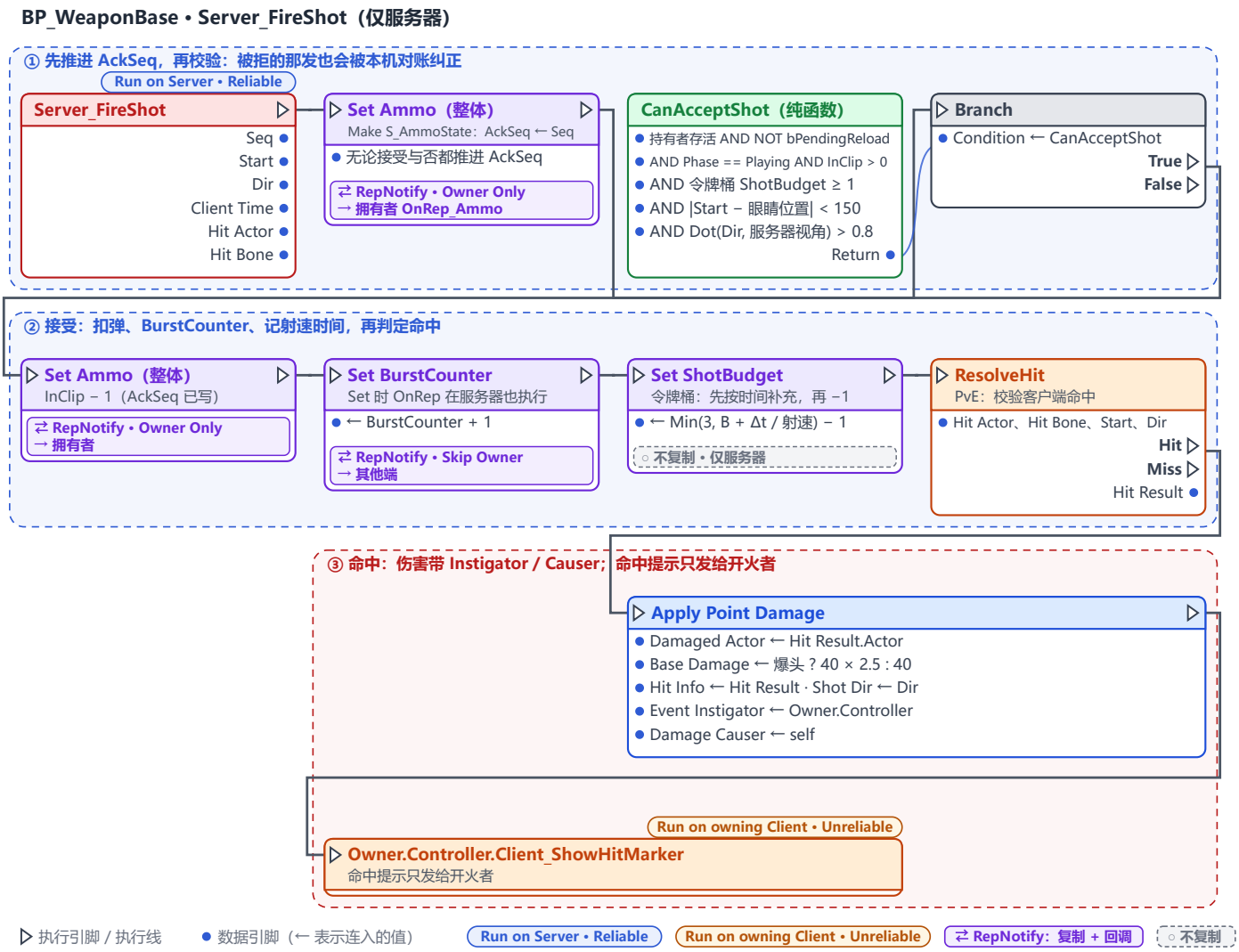
▶ 执行端	本地控制端：StartFire、StopFire、FireShot
⇄ 写入的复制变量	无
○ 只在本端的变量	bWantsToFire、LocalShotSeq
↗ 跨端调用（RPC）	Server_FireShot（Run on Server · Reliable）
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 11-1 本地开火。和原 Demo 的唯一共同点是“本机计时器驱动开火循环”；区别在于表现与后坐力本机立即发生，Server RPC 带上服务器校验需要的全部信息

GetDisplayedAmmo（纯函数）：预测值 = 服务器值 − 未确认发数

```
Unacked   = LocalShotSeq - Ammo.AckSeq          // 本机发出、服务器还没处理（接受或拒绝）的发数
Displayed = Max(0, Ammo.InClip - Unacked)
Return Displayed
// 蓝图里 Integer 是 int32，开局一局打不到 21 亿发，不必考虑回绕；C++ 用 uint16 时要注意回绕比较
```

3. 服务器处理一发：校验链



本图网络摘要

▶ 执行端	Run on Server · Reliable：Server_FireShot
⇄ 写入的复制变量	Ammo（RepNotify · Owner Only）、BurstCounter（RepNotify · Skip Owner）
○ 只在本端的变量	ShotBudget
↗ 跨端调用（RPC）	Owner.Controller.Client_ShowHitMarker（Run on owning Client · Unreliable）
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 11-2　Server_FireShot。所有“能不能开这一枪”的判断都在服务器重新做一遍——客户端的预判只是为了少发无用请求，不能作为依据。主机怎么看到客户端开火？v5 的 Set BurstCounter 在服务器执行时，蓝图会立刻调用 OnRep_BurstCounter——主机进程直接播 3P 表现，不需要额外代码（C++ 版不会自动调用，见第 4 章的手动分支）

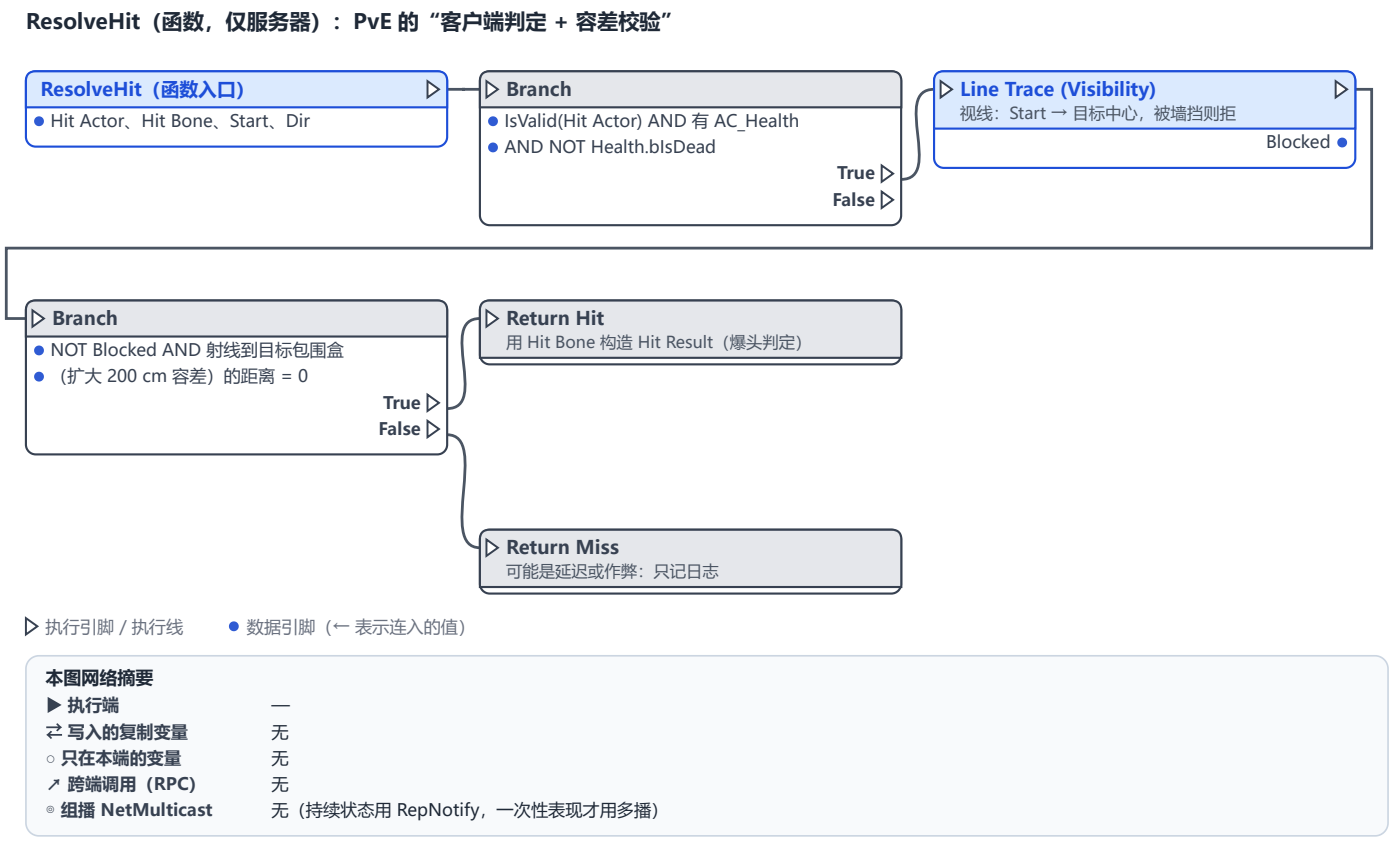
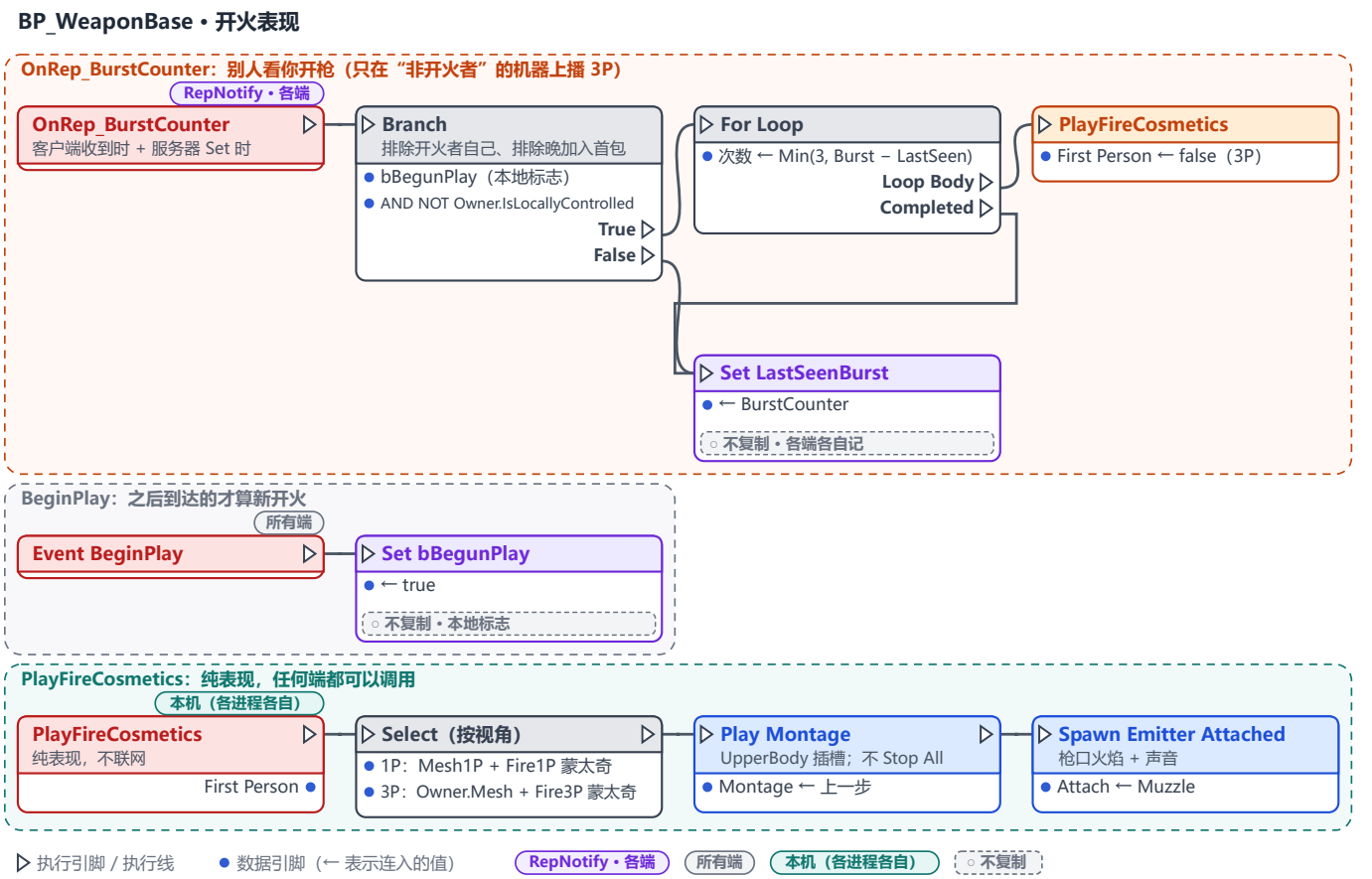


图 11-3 ResolveHit。这是 ShooterGame 的做法（ClientSideHitLeeway = 200 cm）：信任客户端看到的命中，但服务器检查“这一枪物理上说得通吗”。PvP 应换成第 6 章的服务器回溯（C++）

4. 开火表现：每一端各播各的



本图网络摘要

▶ 执行端	RepNotify · 各端：OnRep_BurstCounter；所有端：Event BeginPlay；本机（各进程各自）：PlayFireCosmetics
⇄ 写入的复制变量	无
○ 只在本端的变量	LastSeenBurst、bBegunPlay
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 11-4　开火表现。c1 的两个条件缺一不可：**NOT IsLocallyControlled**——蓝图里服务器 Set BurstCounter 时 OnRep 也会执行，主机自己开枪时如果不挡，会在 1P 预测之外再播一次 3P；**bBegunPlay**——晚加入时首包里带着历史计数，OnRep 在 BeginPlay 之前触发，只记数不播放。原 Demo 的 Multicast_Fire 用 bStopAllMontages = true，空放的那次会打断换弹动画；这里开火蒙太奇只在上半身插槽播放

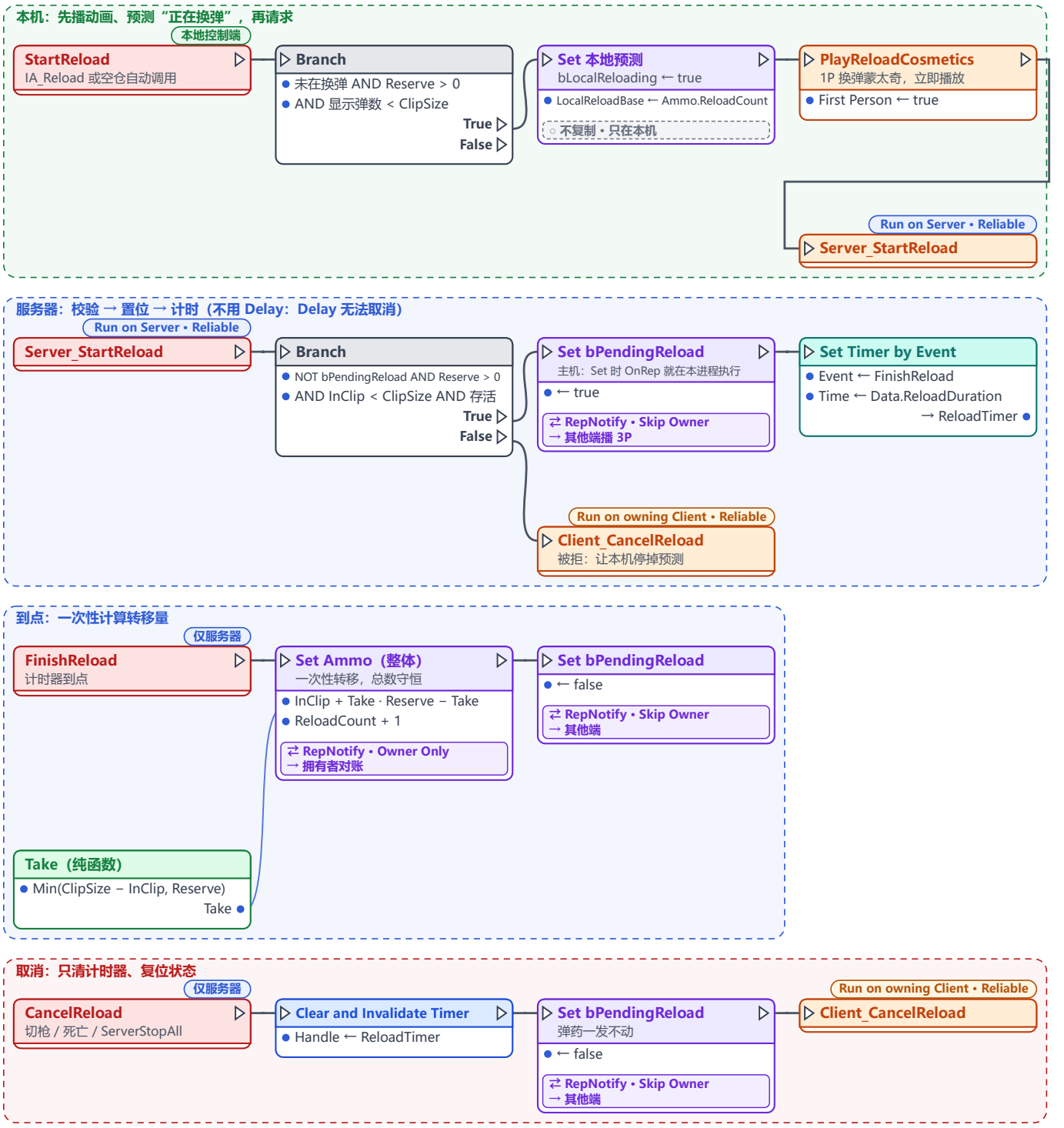
OnRep_BurstCounter 到底在哪几台机器执行：蓝图 vs C++

谁开的枪	开火者的机器	主机进程（服务器）	另一台机器
主机玩家（主机的武器，拥有者 = 主机）	同一个进程：蓝图 Set 时 OnRep 立即执行 → 被 `NOT IsLocallyControlled` 挡掉（1P 已预测）；C++ 不执行		**客户端：收到复制 → OnRep → 播 3P**
客户端玩家（客户端的武器，拥有者 = 客户端）	**Skip Owner：不发送（1P 已预测）**	**蓝图：Set 时 OnRep 执行 → 主机看到 3P；C++：不执行，要在 Server_FireShot 里手动播（第 4 章）**	—（双人局没有第三台）

一句话：**蓝图的 RepNotify 是“谁 Set 谁也跑一次”，C++ 的 OnRep 是“只有收到复制的客户端才跑”**。同一套设计从 C++ 搬到蓝图（或反过来）时，Listen Server 主机这一端最容易多播一次或少播一次。bPendingReload 的 OnRep 同理，也要过滤拥有者。

5. 换弹：本机预测 + 服务器计时状态机

BP_WeaponBase • 换弹



▷ 执行引脚 / 执行线　● 数据引脚（← 表示连入的值）　本地控制端　Run on Server • Reliable　Run on owning Client • Reliable　仅服务器

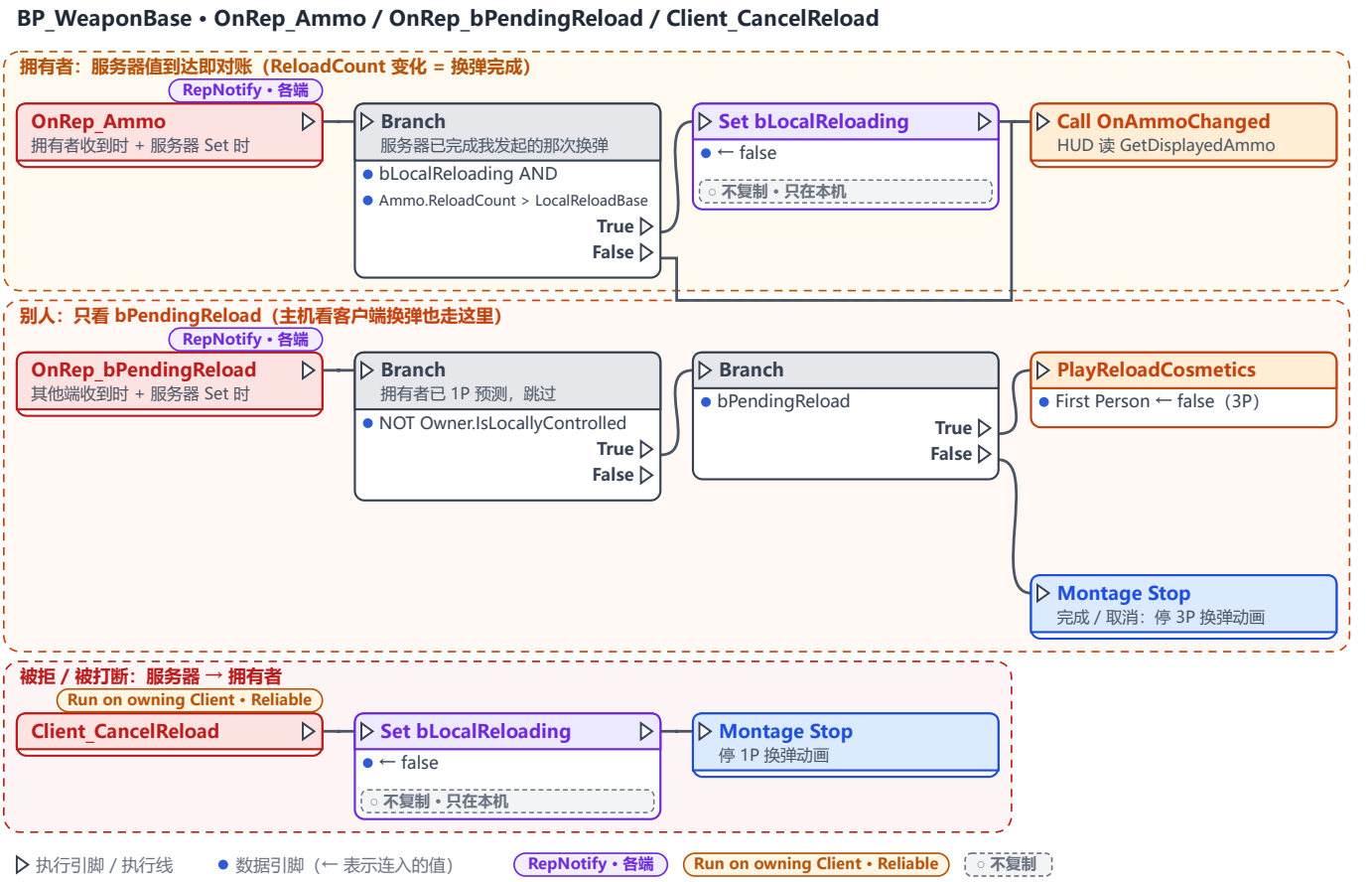
⇄ RepNotify：复制 + 回调　○ 不复制

本图网络摘要

▶ 执行端	本地控制端：StartReload；Run on Server · Reliable：Server_StartReload；仅服务器：FinishReload、CancelReload
⇄ 写入的复制变量	bPendingReload（RepNotify · Skip Owner）、Ammo（RepNotify · Owner Only）
○ 只在本端的变量	bLocalReloading / LocalReloadBase
↗ 跨端调用（RPC）	Server_StartReload（Run on Server · Reliable）、Client_CancelReload（Run on owning Client · Reliable）
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 11-5 换弹状态机的蓝图实现。对照原 Demo：一开始就把当前弹数清零、调用漏连 Return 的函数、用 Delay 等待（无法取消），三处都改掉了

6. 弹药对账与别人看到的换弹



本图网络摘要

▶ 执行端	RepNotify · 各端：OnRep_Ammo、OnRep_bPendingReload；Run on owning Client · Reliable：Client_CancelReload
⇄ 写入的复制变量	无
○ 只在本端的变量	bLocalReloading
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 11-6 对账与表现。用 ReloadCount 而不是“InClip 变大”判断换弹完成：开火和换弹的复制可能合并在同一次更新里到达，只看弹数会误判。OnRep_Ammo 在服务器上对“客户端的武器”也会执行一次，但那份的 bLocalReloading 永远是 false、HUD 也没绑定它，所以无害

7. 一个隐蔽的坑：射速校验与“扎堆到达”



图 11-7 为什么射速不能按“到达间隔”校验。Reliable 丢包重传后多发请求会扎堆到达（E2：RTT 200 ms、丢包 10% 时 25.8% 的请求与别的请求同帧执行），朴素校验会误拒正常玩家；令牌桶允许短时突发，但长期平均不超过射速

8. 原 Demo 对照（逐节点）

原节点	原逻辑	问题	重做对应
射击中 #96	读复制的 当前子弹数量 ≤ 0 → 空枪声；否则 Server_Fire	读的是晚一个 RTT 的值 → 多发一次请求（E1 空放）	FireShot 读 GetDisplayedAmmo
Server_Fire #100	Multicast_Fire → Server_SpawnBullet	先多播后校验；本机也等多播	本机先播；服务器接受后 BurstCounter+1
Server_SpawnBullet #95	CheckCanFire → SpawnActor(BP_Bullet, Owner=∅) → 扣弹 → 后坐力	复制子弹每端算伤害；Instigator 空；后坐力在服务器无效	命中扫描 + ResolveHit + ApplyPointDamage；后坐力本地
子弹发射位置旋转 #121	枪口插槽 + 散布（读 AnimBP.是否瞄准）	服务器上 是否瞄准 永远 false → 客户端开镜也有散布	相机视点 + 预测标志；Seq 作种子
Server_Change #105	先清零当前弹数 → 多播 → Delay → 剩余子弹计算()	吞子弹（E4）、无法取消、打断丢弹	图 11-5
剩余子弹计算 #120	SwitchHasAuthority；false 分支未连 Return	返回 0	Take 纯函数 + 一次性 Set
OnRep_* ×5	全部为空	复制了但没人用	3 个 OnRep，各有用途
BeginPlay / for_server	Cast GetOwner 缓存 As BP Player、打印名字	BeginPlay 时 Owner 还是空	OnEquipped(Character) 传入

面试注意：武器这一章最常被追问的三个细节

- **“AckSeq 为什么在校验之前写？”**——不管接受还是拒绝，这一发都“处理完了”。先写 AckSeq，被拒的那发就从“未确认”里消失，显示值自动 +1 回来；如果只在接受时写，本机会一直少显示一发。
- **“服务器怎么复现客户端的散布？”**——散布用 Seq 做随机种子（Random Stream），客户端和服务器用同一个种子得到同一个方向；服务器只需要校验方向和自己的视角接近，不需要信任客户端随便给的方向。
- **“射速怎么校验才不误伤？”**——不能比较两条请求的**到达**间隔：Reliable 丢包重传后会扎堆到达（图 11-7）。用令牌桶（允许 2~3 发的短时突发，长期平均不超过射速），或比较客户端时间戳的间隔并与服务器时间做合理性检查。
- **“Delay 和 Set Timer by Event 有什么区别？”**——Delay 是 Latent 节点，挂在事件图表上无法从外部取消，同一事件再次进入还会被忽略；计时器有句柄，可以清除、查询剩余时间，换弹被打断时必须能取消。

本章速记

- 武器只同步 3 个东西：**Ammo（OwnerOnly，带 AckSeq / ReloadCount）**、**BurstCounter（SkipOwner）**、**bPendingReload（SkipOwner）**。
- 本地：预判 → Seq+1 → 播 1P → 后坐力 → 本地射线播特效 → Server_FireShot。
- 服务器：先写 AckSeq → 校验（存活 / 换弹 / 阶段 / 弹药 / 射速 / 起点 / 方向）→ 扣弹 + BurstCounter → ResolveHit → ApplyPointDamage(Instigator, Causer)。
- 换弹：本机预测动画 → Server_StartReload 校验 → 计时器 → 一次性 Take → ReloadCount+1；取消只清计时器。
- 蓝图细节：OnRep 无旧值（自存 LastSeenBurst）、结构体整体赋值、用计时器不用 Delay。

▶ **自测：换弹期间玩家按住开火，本机和服务器分别发生什么？**

▶ **自测：为什么武器要勾 Net Use Owner Relevancy?**

CHAPTER 12

敌人层事件图表：BP_Zombie / AIC_Zombie / 行为树

僵尸是“服务器拥有一切”的典型：决策、寻路、攻击判定、死亡都在服务器；客户端只拿到一个会动的身体，和几个用来播表现的复制变量。原 Demo 的僵尸 AI 基本放对了地方，问题集中在“随机”和“死亡”上。

本章目标

- 说清僵尸的“灵魂”（AIController + 行为树）只在服务器、“身体”在所有端。
- 用“服务器决定种子、各端算结果”的方式同步随机外观；用计数器同步攻击动作。
- 写出攻击判定（服务器时间窗口）和死亡（服务器规则 + 各端表现）。

1. 类卡片

项	BP_Zombie	AIC_Zombie
父类	Character	AIController
存在于	所有端（客户端是 SimulatedProxy）	**仅服务器**；客户端上僵尸的 Get Controller 为 None
关键设置	Auto Possess AI = Placed in World or Spawned；AI Controller Class = AIC_Zombie；Net Update Frequency = 20；CMC：RVO Avoidance；Mesh 用 Physics Asset 判爆头	Blackboard = BB_Zombie；行为树 = BT_Zombie
组件	Capsule · Mesh · CharMoveComp · **AC_Health**（与玩家同一个组件）	（PathFollowing、BrainComponent 引擎自带）
复制变量	VariantSeed（Initial Only，RepNotify）· AttackInfo {Counter, Index}（RepNotify）	—
仅服务器变量	bAttacking · Data（DA_Zombie：速度档位、攻击力、攻击距离、动作表）	—

BP_Zombie 父类 Character · 存在于：所有端（客户端是 SimulatedProxy；没有任何客户端拥有它）

Class Defaults → Replication — Replicates: ✓ | Replicate Movement: ✓ | Net Update Frequency: 20 | Auto Possess AI: Placed in World or Spawned

变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
VariantSeed	Integer	RepNotify	Initial Only	服务器（BeginPlay）	各端用同一个种子算速度档位、动画集
AttackInfo	S_AttackInfo	RepNotify	None（全部）	服务器（StartAttack）	{Counter, Index}：各端播同一个攻击动作
bAttacking	Boolean	None 不复制	—	服务器	攻击状态只给规则用，不复制
Data	DA_Zombie	None 不复制	—	配置	速度、攻击力、攻击距离、动作表
（组件）AC_Health	Health 等	RepNotify	None（全部）	服务器	Health / LastHit / bIsDead，同玩家（图 10-6a）

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
Event BeginPlay	引擎事件	引擎事件	—	所有端（每个副本）	每个副本
OnRep_VariantSeed / OnRep_AttackInfo	RepNotify 函数	RepNotify 函数	—	服务器(Set 时) + 收到的客户端	客户端收到时；服务器 Set 时
StartAttack / DoAttackTrace / EndAttack	自定义事件	Not Replicated	—	仅服务器	BTT_Attack、计时器
HandleDeath	自定义事件	Not Replicated	—	仅服务器	AC_Health.OnDeath
PlayDeath / PlayHitReact	自定义事件	Not Replicated	—	服务器(Set 时) + 收到的客户端	AC_Health 的 OnRep 广播
Multicast_DeathCry	自定义事件	Multicast	✗	服务器 + 所有客户端	HandleDeath（服务器）
Multicast_Growl	自定义事件	Multicast	✗	服务器 + 所有客户端	BTS_FindTarget 换目标时（服务器）

图 12-0a BP_Zombie 的网络规格。全书仅有的两个组播都在这里，而且都是 Unreliable 的一次性音效；状态（外观、攻击、血量、死亡）全部是 RepNotify

AIC_Zombie 父类 AIController · 存在于：仅服务器

Class Defaults → Replication — Replicates: ✗ AIController 默认不复制 | 客户端上存在: ✗ 客户端上僵尸的 Get Controller = None

变量（My Blueprint → Variables；Details → Replication / Replication Condition）

名称	类型	Replication	Condition	谁写	收到后 / 说明
黑板键 TargetActor、bInRange	Blackboard	None 不复制	—	服务器（BTS_FindTarget）	黑板、行为树、感知都只在服务器；客户端想显示“僵尸盯着谁”只能另加复制变量

事件 / 函数（自定义事件 Details → Graph → Replicates • Reliable）

名称	类型	Replicates	Reliable	在哪执行	谁触发
Event On Possess	引擎事件	引擎事件	—	仅服务器	僵尸生成后自动 Possess

图 12-0b AIC_Zombie 的网络规格：整个“灵魂”只在服务器，客户端只看到身体的移动和复制下来的状态

2. BeginPlay 与外观变体：服务器只决定一个种子

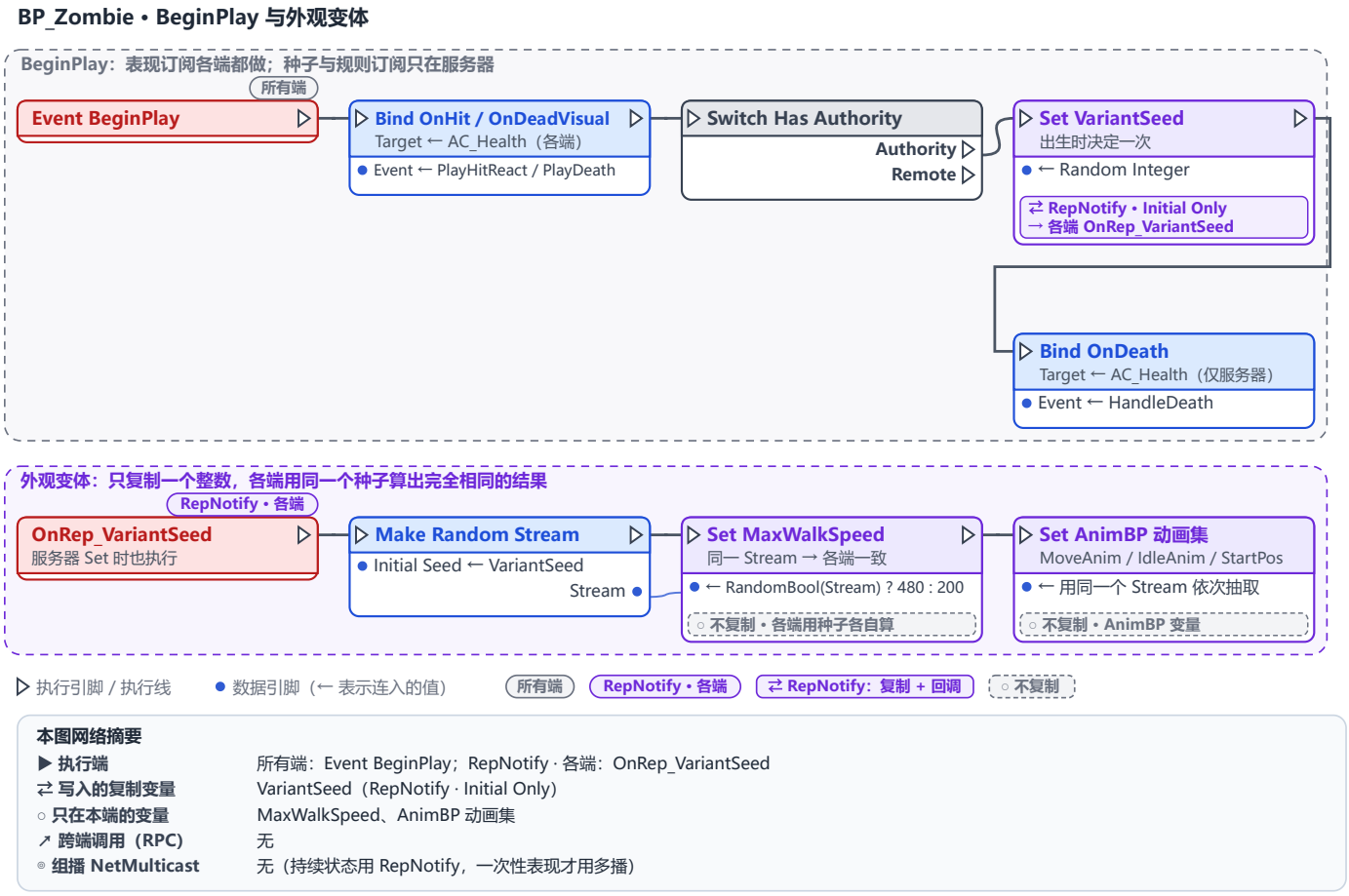
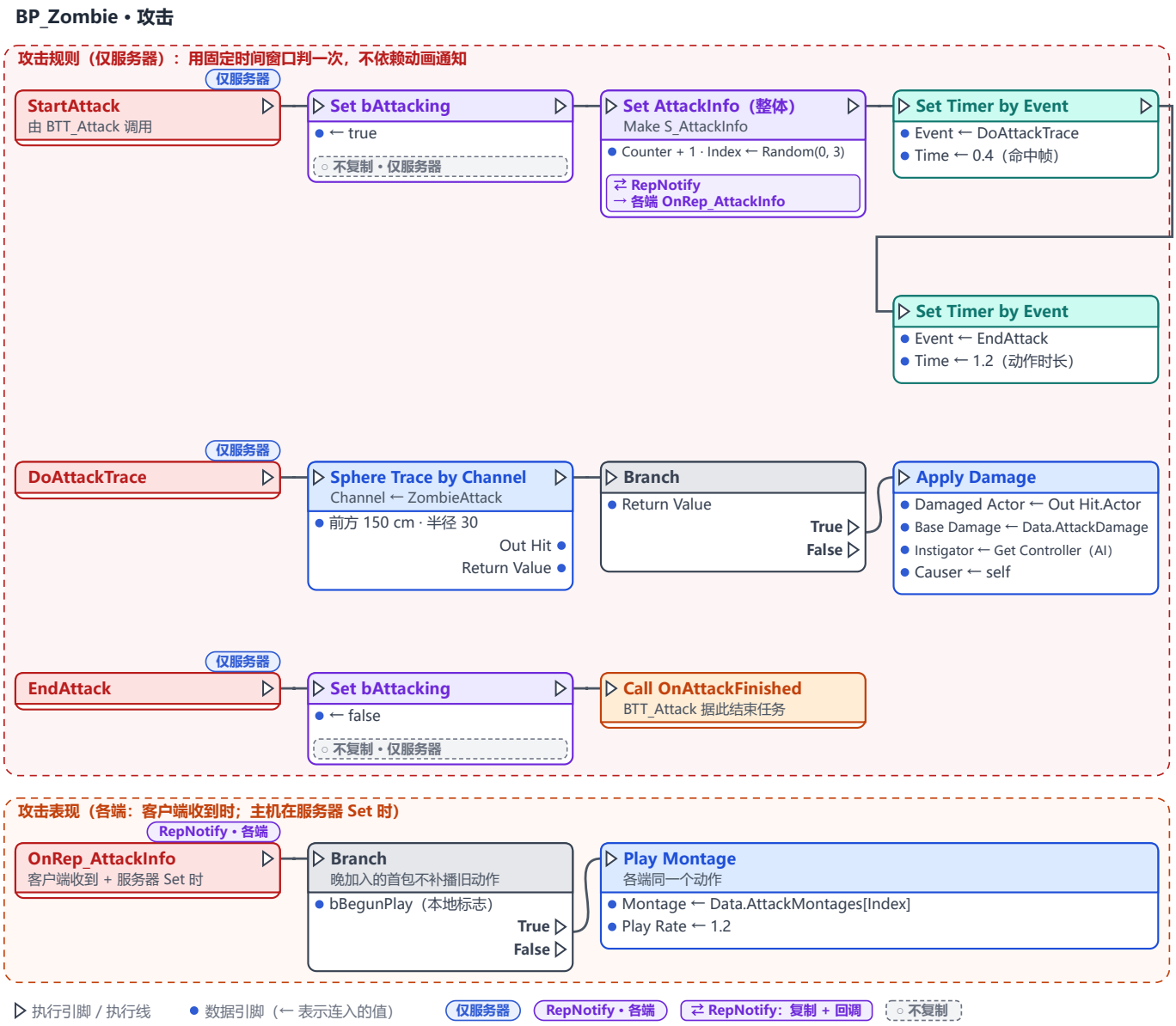


图 12-1 外观变体。原 Demo 的“初始化速度和动画”放在 SwitchHasAuthority 之后，只在服务器设置了 AnimBP 的动画变量——客户端上所有僵尸都用默认的 IP_Run_Front，480 速度的僵尸在客户端是“滑步跑”。VariantSeed 用 Initial Only：出生后不再变化，省去后续比较

“同步随机”的通用做法

随机结果要在各端一致时，不要各端各自随机，也不要把结果逐项复制：**服务器生成一个种子并复制，各端用同一个 Random Stream 按相同顺序抽取。**外观、动作序号、散布方向（武器用 Seq 作种子）都是这个模式。复制量是一个整数，而且天然支持晚加入。

3. 攻击：服务器时间窗口判定，各端播同一个动作



本图网络摘要

▶ 执行端	仅服务器：StartAttack、DoAttackTrace、EndAttack；RepNotify · 各端：OnRep_AttackInfo
⇄ 写入的复制变量	AttackInfo（RepNotify）
○ 只在本端的变量	bAttacking
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 12-2 攻击。原 Demo 用 AnimNotify 开启一个“间隔 = 当帧 DeltaTime”的循环计时器做骨骼扫描，DoOnce 防重复；在 Dedicated Server 上动画可能不更新、通知不触发。重做后判定只依赖服务器计时，动画只负责好看

面试注意：原 Demo 的“正在攻击”为什么不对

`正在攻击` 是复制变量：服务器在“攻击范围检测”里置 true，然后在 `Multicast_PlayAttackAnim` 的蒙太奇 `On Completed` 里置 false——这个多播在每一端都执行，于是每一端都在写一个复制变量；服务器上的值依赖服务器自己播放蒙太奇的完成回调，Dedicated Server 下蒙太奇未必按时完成，僵尸可能永远卡在“正在攻击”。重做后 bAttacking 只是服务器内部状态，由计时器复位；给客户端的只有“第几次攻击、用哪个动作”。

4. 死亡：规则在服务器，表现在各端

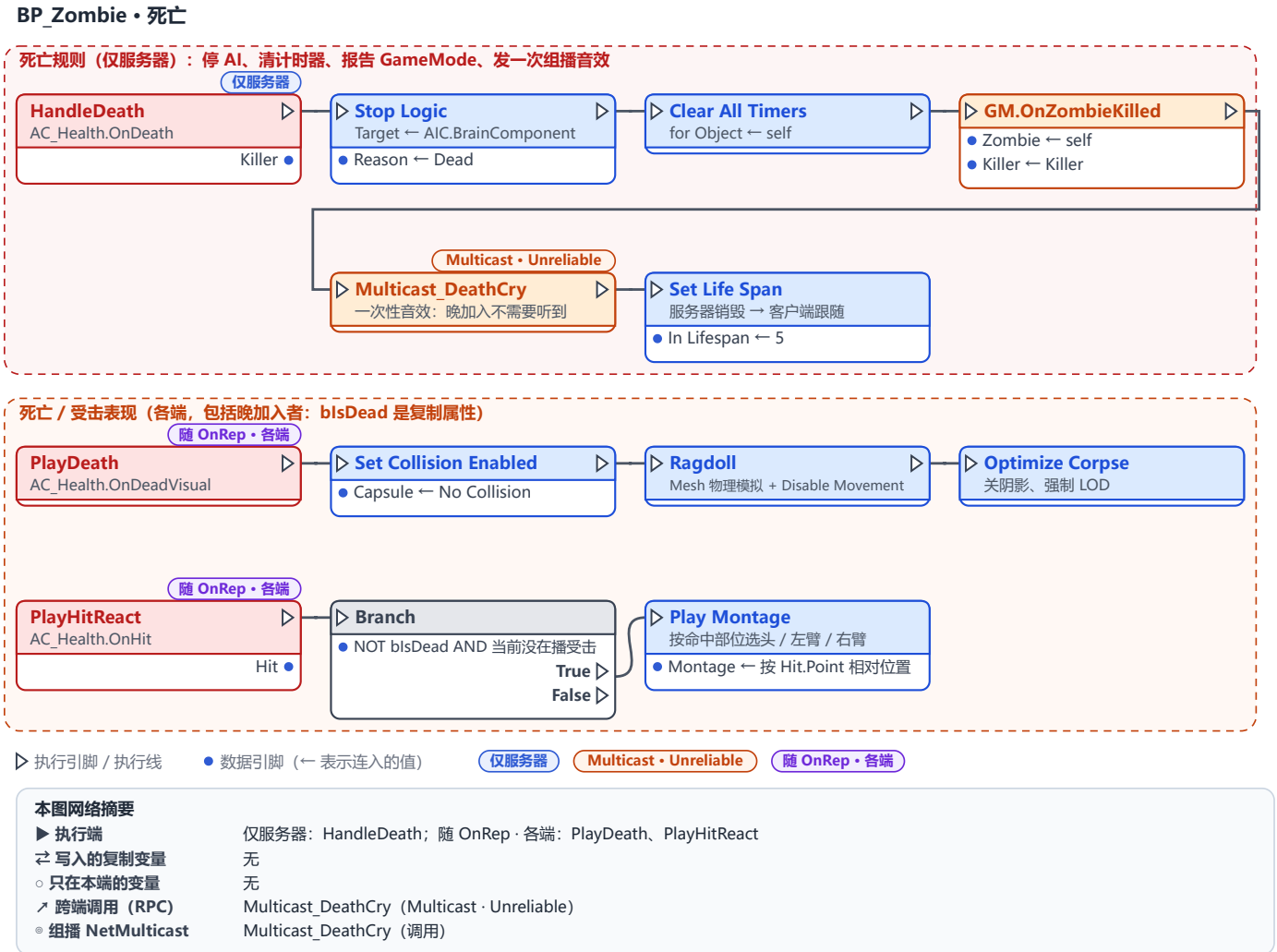
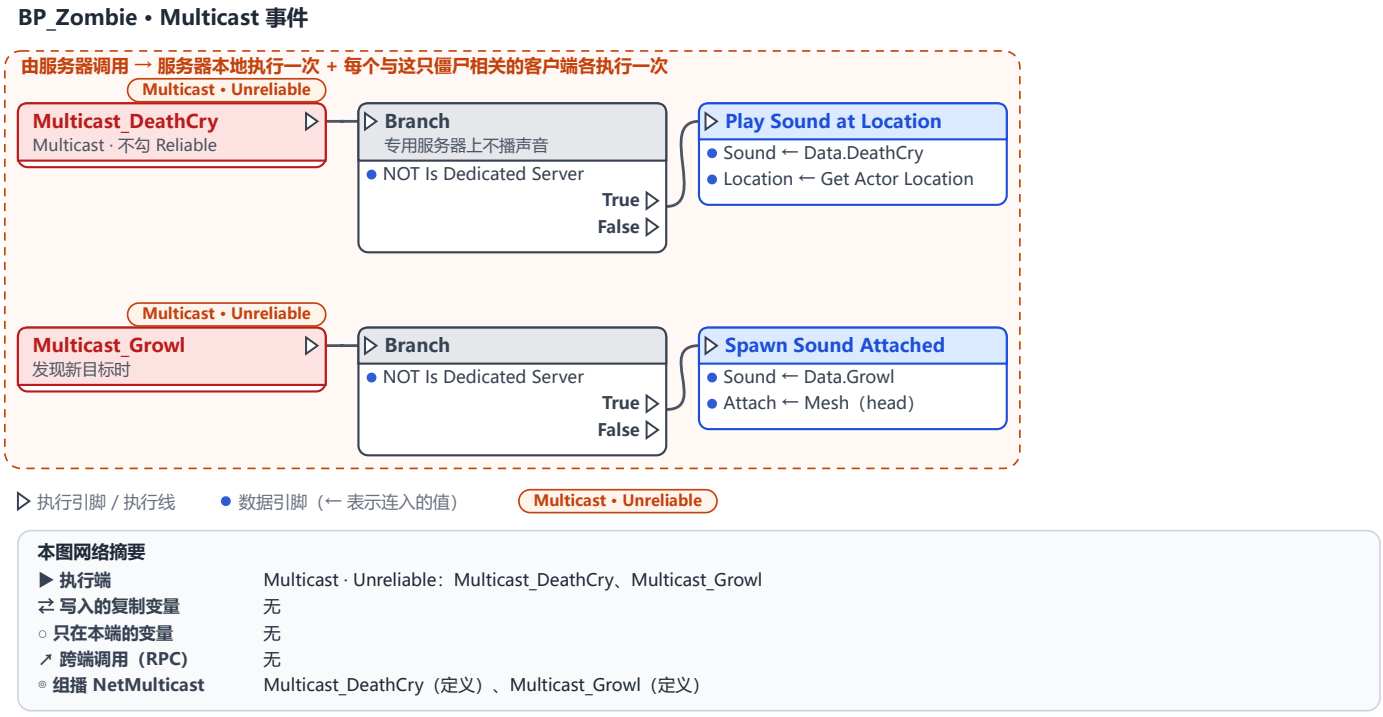


图 12-3 死亡与受击。原 Demo 的 Multicast_Death 里清的是一个不存在的计时器“追逐玩家”，找目标和攻击检测的计时器都还在跑；补刷调用直接跳过了 SwitchHasAuthority，在客户端生成只有自己看得到的僵尸

4.5 全书仅有的两个组播：一次性世界音效



本图网络摘要

▶ 执行端	Multicast · Unreliable：Multicast_DeathCry、Multicast_Growl
⇄ 写入的复制变量	无
○ 只在本端的变量	无
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	Multicast_DeathCry（定义）、Multicast_Growl（定义）

图 12-3a　两个 Unreliable 组播。调用方都在服务器：HandleDeath 调 Multicast_DeathCry，BTS_FindTarget 换目标时调 Multicast_Growl（图 12-6）。Listen Server 下主机进程会执行一次（主机听到），客户端收到一次

什么时候用组播，什么时候用 RepNotify

问自己	是 → 用 RepNotify（状态）	是 → 用 Unreliable 组播（事件）
晚加入的人需要看到吗?	需要：尸体、外观、比赛阶段	不需要：惨叫、低吼、爆炸火花
它是“持续的”还是“一瞬间的”？	持续：死了一直是死的	一瞬间：声音播完就没了
丢了一次会怎样?	不能丢：复制属性会被下一次更新纠正	无所谓：少听到一次低吼
开火者 / 拥有者自己已经预测过了吗?	用 Skip Owner 条件排除	组播没有“跳过拥有者”的选项，要在事件里自己判断

为什么死亡惨叫不放在 OnRep_bIsDead 里？因为晚加入的玩家在收到初始复制时也会执行 OnRep_bIsDead——尸体（状态）应该出现，惨叫（事件）不应该再响一次。把“状态的表现”和“事件的表现”分开，是 RepNotify 和组播分工的核心。也可以在 OnRep 里用 bBegunPlay 过滤（武器的 BurstCounter 就是这样做的），两种都对，关键是想清楚晚加入时会发生什么。

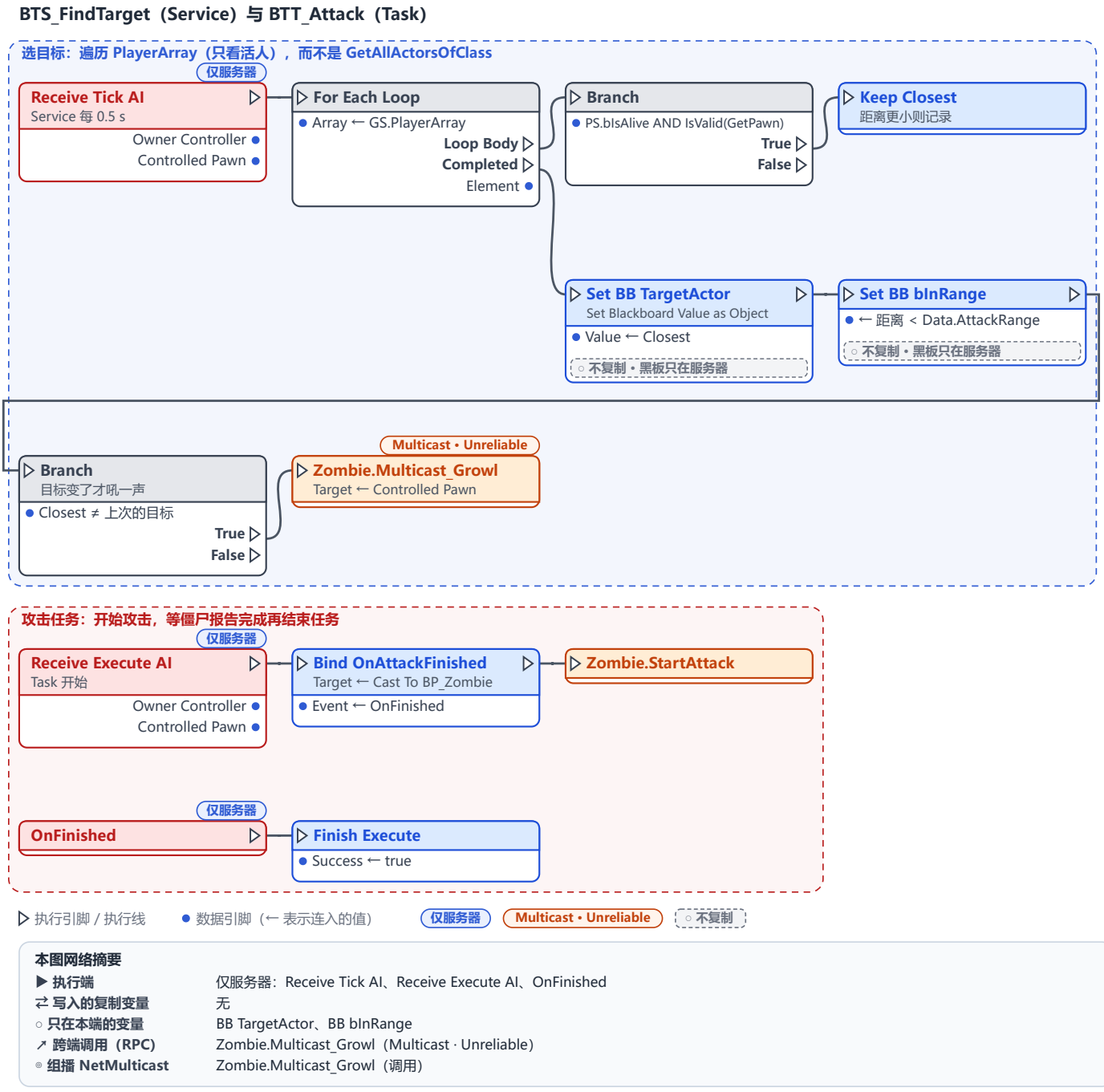
5. AIController 与行为树



图 12-4 AIC_Zombie 只做一件事：被 Possess 时启动行为树。AIController 只在服务器存在，所以整棵树天然只在服务器运行

BT_Zombie 节点	作用	备注
Root	黑板：BB_Zombie（TargetActor、bInRange）	—
Selector	从上到下执行第一个条件满足的分支	Service：BTS_FindTarget，0.5 s
Sequence：攻击	装饰器：TargetActor 已设置 且 bInRange	Observer Aborts：Lower Priority
Rotate to face BB entry	朝向目标	引擎自带任务
BTT_Attack	调 StartAttack，等 OnAttackFinished	自写任务（图 12-6）
Sequence：追击	装饰器：TargetActor 已设置	—
Move To	目标 TargetActor，可接受半径 100	目标移动时持续追踪
Wait	没有目标：原地待机 1 s	—

图 12-5 BT_Zombie。原 Demo 用每 0.5 s 一次的计时器 + SimpleMoveToActor + 每 0.1 s 一次的攻击范围检测，逻辑等价但散落在蓝图里；行为树把“选目标 / 追 / 打”分开，并能在调试器里直接看到当前分支



本图网络摘要

▶ 执行端	仅服务器：Receive Tick AI、Receive Execute AI、OnFinished
⇄ 写入的复制变量	无
○ 只在本端的变量	BB TargetActor、BB bInRange
↗ 跨端调用（RPC）	Zombie.Multicast_Growl（Multicast · Unreliable）
◎ 组播 NetMulticast	Zombie.Multicast_Growl（调用）

图 12-6 Service 与 Task。原 Demo 的 FindClosestTarget 用 GetAllActorsOfClass(BP_Player)，死掉的玩家仍会被当成目标；重做后通过 PlayerState.bIsAlive 过滤。Service 跑在服务器的 AIController 里，所以它调用的 Multicast_Growl 是“服务器调用组播”——服务器和所有相关客户端都会执行

6. BP_SpawnPoint 与刷怪

项	内容
父类	Actor；**Replicates = false**（只是位置标记，各端随关卡加载都有一份，只有服务器会读）
变量	Weight（权重）· Radius（随机可达点半径，默认 200）
谁用	GM_Coop.StartMatch 收集一次，SpawnZombie 每次随机选一个（图 9-5、9-6）
原 Demo	BP_ZombieSpawn 自己负责刷怪，“开始游戏”在每端调用（内部有 SwitchHasAuthority），“重新生成”由每端的僵尸死亡调用且绕过了权限判断

本章速记

- 僵尸：身体在所有端，灵魂（AIC + 行为树）只在服务器；客户端上 Get Controller 为空。
- 同步随机：服务器生成 VariantSeed（Initial Only），各端用同一个 Random Stream 抽取。
- 攻击：服务器计时器窗口判一次 → ApplyDamage(Instigator = AI Controller)；表现用 AttackInfo 计数器。
- 死亡：服务器 StopLogic、清计时器、报告 GameMode、SetLifeSpan；各端由 bIsDead 的 OnRep 播布娃娃。
- 组播只用于“晚加入不需要、丢了无所谓”的一次性表现：Multicast_DeathCry、Multicast_Growl（Unreliable，由服务器调用）。
- 选目标遍历 PlayerArray 并过滤 bIsAlive。

▶ **自测：两名玩家同时打出僵尸的最后一发，会不会死两次、补刷两只?**

▶ **自测：为什么攻击判定不用 AnimNotify?**

CHAPTER 13

UI 层事件图表：主菜单 / 大厅 / HUD / 结算 / 暂停

UI 只有两条规则：**只读不算**——显示的一切都来自复制下来的状态或本地预测值；**请求走 PC**——按钮只调用拥有者 PlayerController 的 Server RPC 或 GameInstance 的本地函数。

本章目标

- 写出 5 个界面的事件图表，每个都不包含任何规则判断。
- 用事件分发器代替 UMG 属性绑定函数（每帧执行）。
- 说清“只有主机能重开”“暂停不暂停世界”在 UI 上怎么体现。

界面	何时出现	读什么	写什么 / 请求什么	原 Demo
WBP_MainMenu	L_Menu 地图	GI.LastError、会话搜索结果	GI.HostGame / FindGames / JoinGame	没有联机入口；关卡蓝图里“按任意键开始”
WBP_Lobby	Phase = Waiting	GS.PlayerArray → 各 PS 的名字、bReady	PC.Server_SetReady	“等待其他玩家确认…”文本，无名单
WBP_HUD	Phase = Playing	AC_Health、武器显示弹数、GS 剩余时间与僵尸数	—	属性绑定每帧 Cast GetPlayerPawn(0)；倒计时自己算
WBP_Result	Phase = Won / Lost	GS.Phase、各 PS 的 Kills	PC.Server_RequestRestart（仅主机）、GI.LeaveGame	OpenLevel 重开、QuitGame
WBP_Pause	按 Esc	—	关闭自身 / GI.LeaveGame	SetGamePaused（客户端无效、主机冻结全员）

0. UI 的网络规格：读哪些复制数据、发哪些 RPC

Widget **只存在于本机**：不复制、没有 Owner 链、不能发 RPC，服务器上的远端 PC 也不会创建它。所以 UI 和网络只有三种接触方式：**读**复制到本机的变量；**订阅**在 OnRep 里广播的分发器；**经由拥有者 PC** 发 Run on Server。本章图里节点底部的青色“⇠ 读复制值”标注，就是 UI 读取的复制数据。

界面	读的复制数据（复制设置）	订阅的通知	发出的请求
WBP_MainMenu	无（GI.LastError 是本机变量）	GI 的搜索结果回调	无 RPC：主持 / 加入是本机 Online Subsystem 调用
WBP_Lobby	PS_Coop.bReady（RepNotify）、PlayerName（引擎复制）	GS.OnPlayersChanged；PS.OnInfoChanged（在 OnRep_bReady 里广播）	**PC.Server_SetReady • Reliable**
WBP_HUD	AC_Health.Health（RepNotify）；Weapon.Ammo（RepNotify · Owner Only）+ 本地预测；GS.MatchEndServerTime、AliveZombies（Replicated）	OnHealthChanged、OnAmmoChanged；PC 收到 Client_ShowHitMarker / Client_ShowDamageFeedback 后转给 HUD	无
WBP_Result	GS.Phase（RepNotify）、PS_Coop.Kills（RepNotify）	GS.OnPhaseChanged（由 PC.HandlePhase 创建本界面）	**PC.Server_RequestRestar • Reliable**（按钮仅主机可见）
WBP_Pause	无	无	无（不调用 Set Game Paused）

表 13-1　UI 的网络规格。HUD 读的 Ammo 是 Owner Only：只有持枪者自己的客户端有这份数据——这正是 HUD 只显示“自己”的弹药所需要的

1. 主菜单

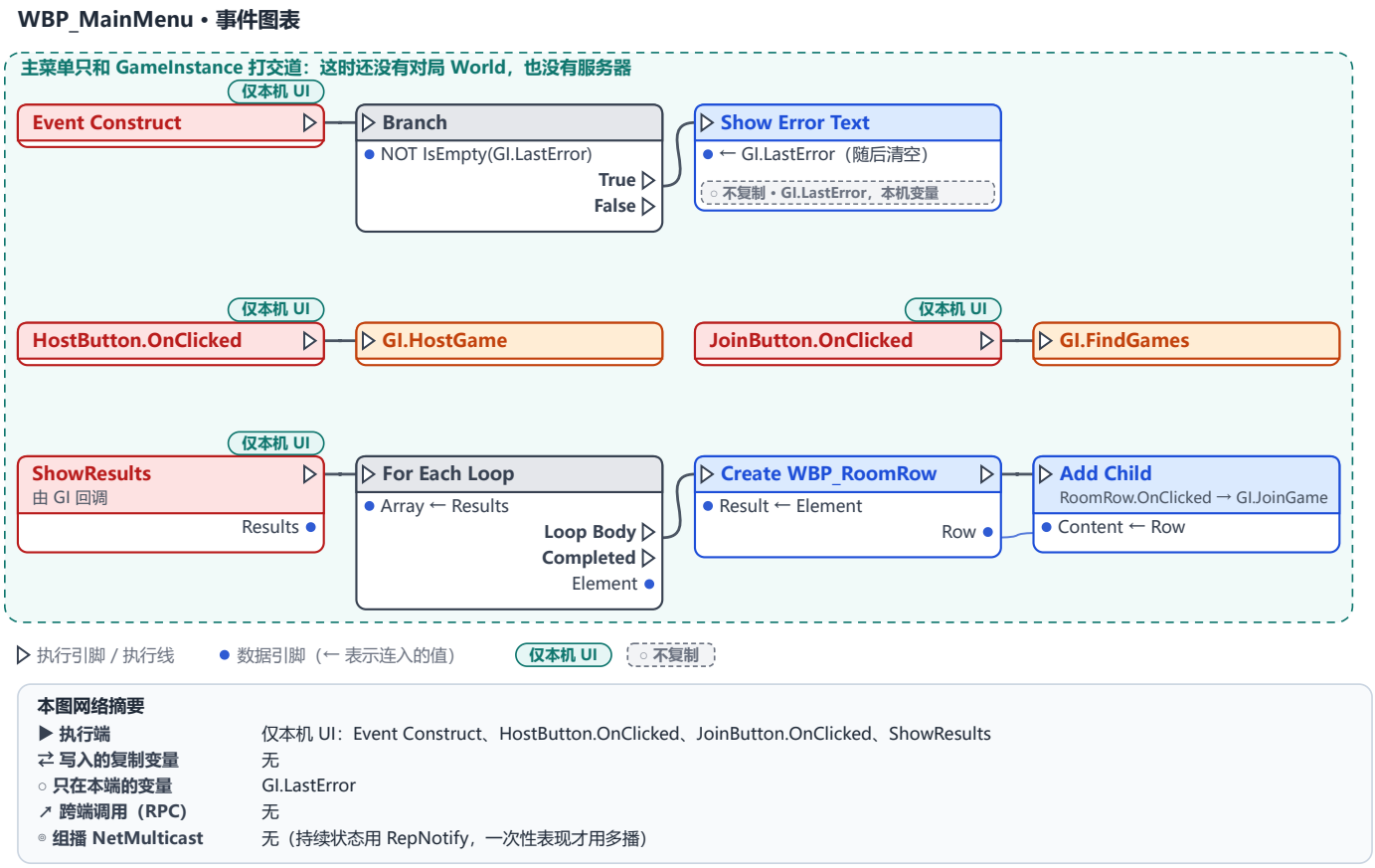


图 13-1　主菜单。上一局断线的原因由 GameInstance 带过来（它跨地图存活），菜单打开时显示一次

2. 大厅

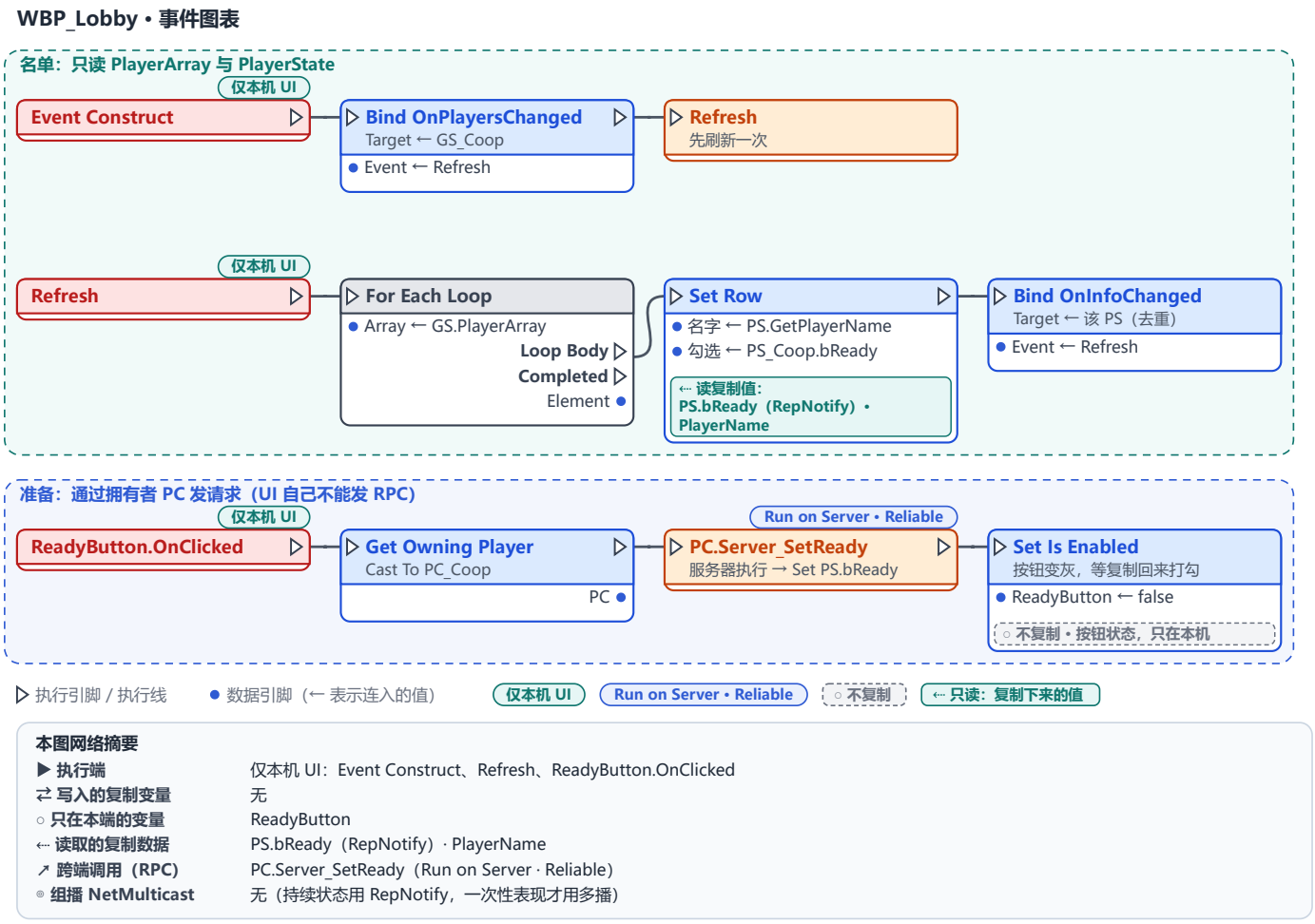
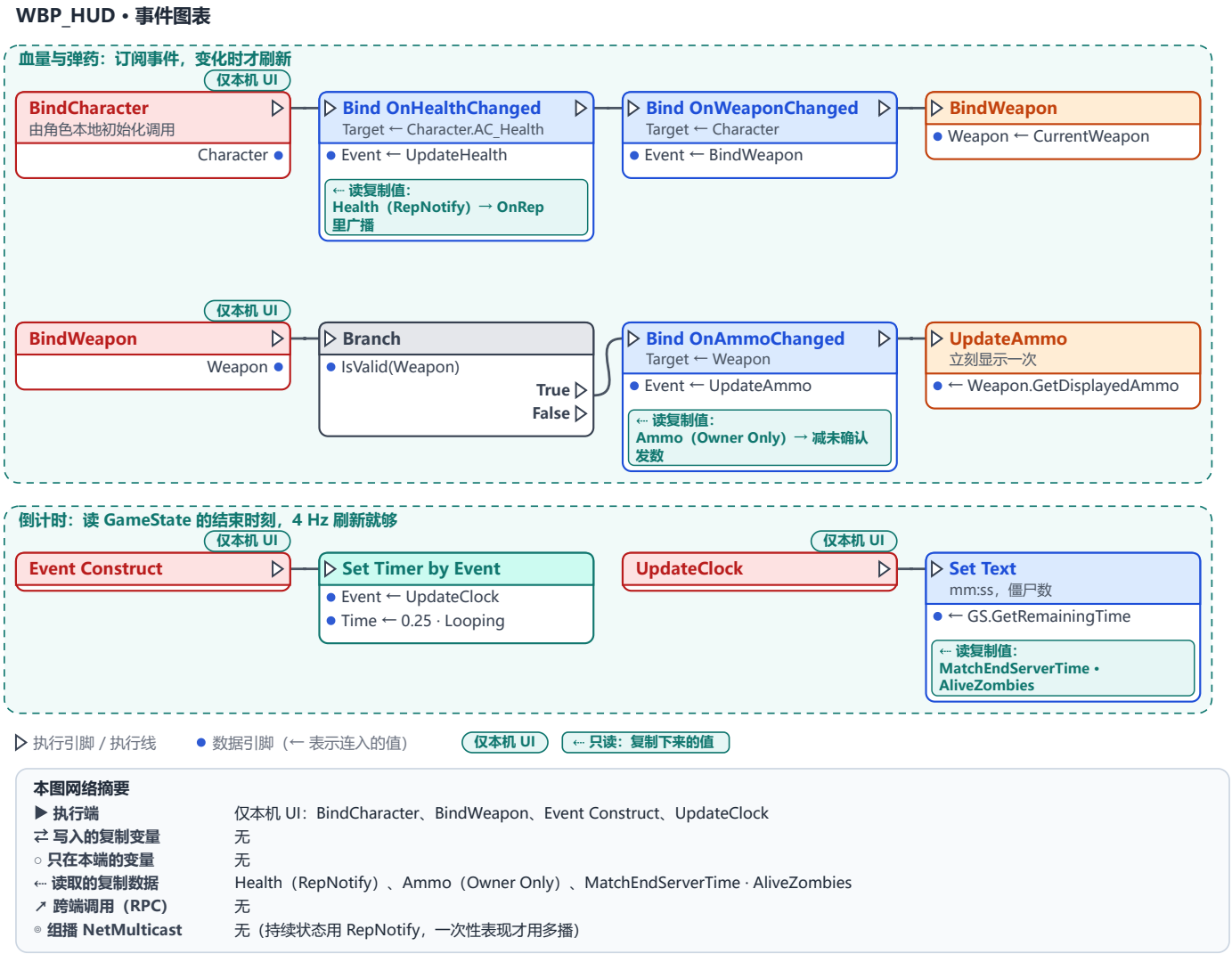


图 13-2 大厅。OnPlayersChanged 由 PS_Coop 在 BeginPlay / EndPlay 时调用 GS 的分发器触发（PlayerState 在每一端都会 BeginPlay，所以有人加入、离开时各端名单都会刷新）。Widget 不是 Actor、没有网络连接，不能直接发 RPC

3. HUD：事件驱动，不用属性绑定



本图网络摘要

▶ 执行端	仅本机 UI：BindCharacter、BindWeapon、Event Construct、UpdateClock
⇄ 写入的复制变量	无
○ 只在本端的变量	无
⇠ 读取的复制数据	Health（RepNotify）、Ammo（Owner Only）、MatchEndServerTime · AliveZombies
↗ 跨端调用（RPC）	无
◎ 组播 NetMulticast	无（持续状态用 RepNotify，一次性表现才用多播）

图 13-3 HUD。原 Demo 的血条、弹药、颜色都是 UMG 属性绑定函数：每一帧 Cast GetPlayerPawn(0)、再读 WeaponRef 的复制变量；武器还没复制到时会访问空引用。重做后 HUD 只在角色把自己交给它、或武器变化时重新订阅

HUD 为什么由角色“交给”它，而不是 HUD 自己去找

HUD 由 PC 在 BeginPlay 创建，此时角色可能还没到（客户端），之后还可能更换（重生、观战），所以 HUD 不能在创建时“找一次角色”就完事。本书让本地控制的角色在 Receive Controller Changed 时调用 `HUD.BindCharacter(self)`：角色只调用 PC 暴露的一个接口，并不知道 HUD 内部怎么画。想再解耦一层，可以让 PC 订阅自己的 `On Possessed Pawn Changed`（服务器和客户端都会广播），由 PC 把新 Pawn 转交给 HUD——这样角色完全不需要知道 HUD 的存在。

4. 结算与暂停

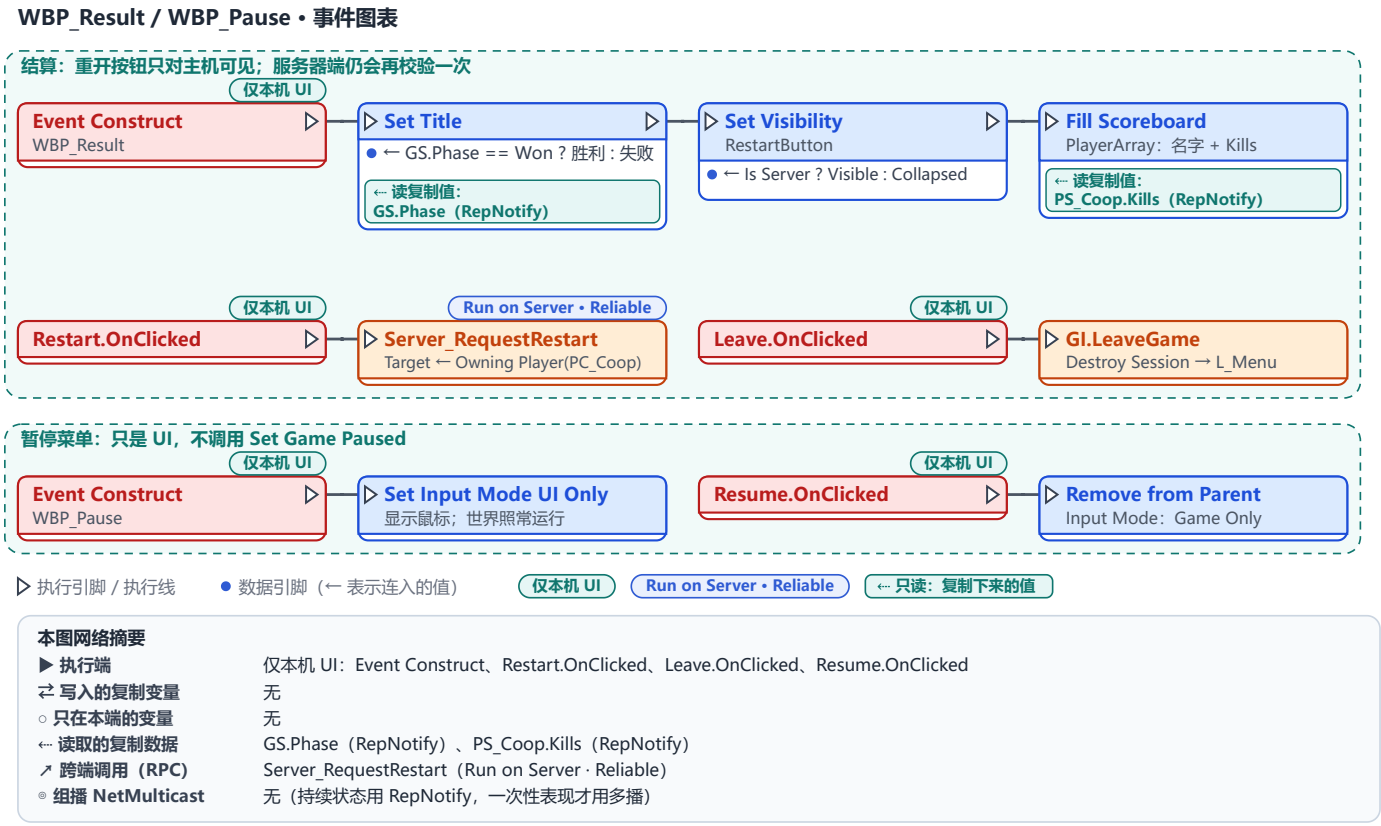


图 13-4 结算与暂停。Is Server 在 Listen Server 的主机上为 true；按钮隐藏只是体验，权限检查在 PC 的 Server_RequestRestart 里（图 9-10），不能只靠 UI

面试注意：UI 的三句“标准答案”

- **“UI 在哪一端？”**——只在本机，永远不复制；服务器不能“给客户端创建界面”，只能改状态或发 Client RPC，让客户端自己创建。
- **“UI 能直接发 RPC 吗？”**——不能，Widget 不是 Actor、没有 Owner 链；通过 Get Owning Player 拿到本地 PC，调用它的 Server RPC。
- **“为什么不用属性绑定？”**——绑定函数每帧执行，联机下还经常读到尚未复制的空引用；事件驱动只在变化时刷新，也更容易推理。

本章速记

- UI = 只读 + 通过 PC / GI 发请求；不做规则判断、不存权威状态。
- UI 接触网络只有三种方式：读复制下来的变量、订阅 OnRep 里广播的分发器、经拥有者 PC 发 Run on Server（全书 UI 只发 2 个：Server_SetReady、Server_RequestRestart）。

- 主菜单 ↔ GameInstance；大厅 ↔ PlayerArray / PlayerState；HUD ↔ AC_Health / 武器 / GameState；结算 ↔ Phase / Kills。
- 重开按钮只给主机看（Is Server），权限仍由服务器校验；暂停菜单不暂停世界。

CHAPTER 14

验收：测试清单、调试工具与演示方式

“合格”最后要落在可复现的测试上。本章给出每次改网络代码都要跑的清单、按症状查问题的工具表，以及面试前怎么把这个 Demo 录成一段有说服力的演示。

---

1. 四种运行模式，各测什么

模式	PIE 设置	能暴露的问题
Listen Server	Number of Players = 2，Net Mode = Play As Listen Server	主机特殊性：主机玩家既 HasAuthority 又 IsLocallyControlled；SkipOwner 的 OnRep 不在服务器触发
Dedicated Server	Net Mode = Play As Client（后台启动专用服务器）	依赖“主机玩家存在”的逻辑（原 Demo 的开局链路）；服务器不渲染：动画通知、骨骼命中盒
独立进程	取消 Run Under One Process	同进程共享的单例、静态变量掩盖的问题；更接近真实的加载时序
Standalone 双开 / 打包	两个独立游戏进程，局域网会话	会话（主持 / 查找 / 加入）、断线回菜单、ServerTravel 带人换图

2. 测试清单

#	设置	步骤	预期	原 Demo
1	Listen Server	两人各自开火、换弹、冲刺、开镜	两边都能看到对方的 3P 开火 / 换弹 / 开镜姿势	**只能看到漂浮手臂**
2	Dedicated Server + 2 客户端	打完一整局	开局、刷怪、爆头、结算全部正常	**开局依赖主机**
3	网络模拟 Bad（或 200 ms / 10%）	连射打空、换弹、冲刺来回切	开火零延迟；弹药单调；`p.NetShowCorrections 1` 无纠正	**E1 / E5 / E6**
4	同上	对横移的冲刺僵尸连射	命中率与 0 延迟时接近	**E3**
5	Allow Late Joining	开局 30 s 后第二人加入	直接观战；看到正确的阶段、倒计时、已死僵尸的尸体	**游戏是否开始 = false**
6	Listen Server	客户端玩家先死；再测主机先死	死者不能操作、进入观战；活着的人继续；全灭才失败	**死者仍能操作；主机 3 s 后冻结全员**
7	任意	弹匣 3 / 备弹 7 时换弹；换弹中按开火；换弹中被打死	换完 10 / 0；开火无效；死亡时弹药不变	**E4：0 / 0**
8	任意	两人同时打同一只僵尸的最后一滴血	只死一次、只补刷一只、击杀算给先到服务器的一发	**客户端本地多刷**
9	Listen Server	两人并排看同一只僵尸	跑姿、待机、攻击动作一致	**各端各自随机**
10	Standalone 双开	客户端按 Esc；客户端点重开；主机点重开	暂停只影响自己；客户端看不到重开按钮；主机重开带上客户端	**暂停无效 / OpenLevel 断线**
11	Standalone 双开	对局中关闭主机	客户端回到主菜单并提示“主机已断开”	**无处理**

12	任意	过滤日志 LogNet、LogNetPlayerMovement、LogPawn、LogScript	无 No owning connection、无 Accessed None、无持续纠正、无 DisableInput 错误	**均有**

3. 按症状查问题

症状	先看	工具 / 命令
客户端调用没反应	RPC 是否被丢弃（Actor 不属于这个客户端）	日志 `No owning connection for actor`；检查 Owner 链
某个变量客户端不更新	Actor / 组件是否 Replicates；变量是否标了复制；条件是否把这个客户端排除了	Networking Insights 的属性 / RPC 视图；在 OnRep 里打 Print 确认是否到达
主机正常、客户端不正常	是否用了 GetPlayerController(0)、HasAuthority 代替 IsLocallyControlled	全局搜索；在节点前打 Print（会带 Server / Client 前缀）
角色被拉回 / 抖动	两端移动参数是否一致	`p.NetShowCorrections 1`、`p.NetCorrectionLifetime 4`、LogNetPlayerMovement
AI 不动 / 卡住	行为树当前分支、黑板值	Gameplay Debugger（按 `'`）、Behavior Tree 调试视图、Visual Logger
延迟高、带宽高	哪个 Actor / 属性 / RPC 占了带宽	`stat net`、启动参数 `-NetTrace=1 -trace=net` 录 Networking Insights
帧卡（不是网络）	游戏线程 / 渲染线程耗时	`stat unit`、`stat game`、Unreal Insights（CPU）

一个省时间的小习惯：给 Print String 加上“我是谁”

PIE 多人时 Print String 自带 `Server:` / `Client 1:` 前缀，但自定义日志没有。写一个函数库函数 `NetTag(Actor)`，返回 “NetMode + LocalRole + 是否本地控制”，所有调试输出都带上它，排查“这段代码到底在哪一端跑”会快很多。

4. 面试前：把 Demo 录成一段有说服力的演示

段落	时长	画面	旁白要点
1. 架构	20 s	Content/Coop 目录 + 类清单表	“规则在 GameMode，状态在 GameState / PlayerState，武器挂在 Owner 链上，血量是组件”
2. 正常联机	30 s	两个窗口并排，主机 + 客户端，打一波僵尸	两边看到同一只僵尸的同一个动作、同一个死亡
3. 弱网	40 s	开启 Bad 网络模拟，左上角 `stat net`	开火零延迟、弹药不跳、冲刺不拉回（开着 p.NetShowCorrections）
4. 边界	30 s	客户端先死 → 观战；晚加入	“死亡和阶段都是复制状态，晚加入也看得到”
5. 前后对比	30 s	原 Demo 同条件录屏 + E1 / E6 图	“原来开火要等 237 ms、冲刺每秒被纠正 9.2 次，现在都是 0” （数字来自本书的模拟；录屏时用自己测到的值）

README 模板（放在仓库首页）

```
# Coop Zombie Shooter（UE 5.5 · Listen Server · 2 人合作）
## 我做了什么
- 服务器权威：伤害 / 死亡 / 弹药 / 比赛阶段 / 刷怪只在服务器；客户端只发请求、只播表现
- 客户端预测：开火表现、弹药（序号对账）、换弹动画、冲刺 / 瞄准（自定义 CharacterMovement）
- 弱网：200 ms / 10% 丢包下开火零延迟、弹药不回跳、移动无纠正；PvE 命中采用客户端判定 + 服务器容差校验
## 类结构
GM_Coop（规则）· GS_Coop（全局状态）· PS_Coop（玩家状态）· PC_Coop（UI 与请求）· BP_CoopCharacter
## 同步清单
16 个复制变量 / 8 个 RPC + 2 个 Unreliable 组播；每个变量的复制条件与接收方、每个事件的执行端（见 Do
## 测试
Listen Server / Dedicated Server / 独立进程 / 网络模拟 Bad / 晚加入 / 死亡顺序（见 Docs/TestPlan.
## 演示视频
（链接）
## 已知限制与下一步
- 只支持 2 人、局域网会话；未做主机迁移
- PvP 需改为服务器回溯；GAS 化
```

本章速记

- 四种模式：Listen Server（主机特殊性）、Dedicated Server（无主机玩家、无渲染）、独立进程、Standalone 双开（会话）。
- 12 条测试：弱网三件套、晚加入、死亡顺序、换弹边界、同时击杀、随机一致、暂停 / 重开、断线、日志干净。
- 演示：架构 → 正常联机 → 弱网 → 边界 → 前后对比，每段都有一个可量化的结论。

PART FOUR

第四部分　面试防守

面试官很少只问一句。这一部分按“主问题 → 追问 1 → 追问 2 → 追问 3”的链条准备，每条链的最后给出防守要点：哪些话一定要说、哪些话不要说。

CHAPTER 15

追问链：十二条面试官最可能走的路

每条链都从一个“开放问题”开始，越往下越具体。准备的原则：第一层给结构，第二层给机制，第三层给数字或代码，第四层承认边界并说出下一步。

回答追问的“三段式”

结论（一句话）→ **机制**（为什么，引擎里谁负责）→ **证据**（我在 Demo 里怎么做的 / 测到了什么）。被问到不确定的细节时，说“我的理解是……，我会用 X 方法验证”，比硬编一个答案安全得多。

链 1　“介绍一下这个 Demo 的架构。”

按“规则 / 状态 / 人 / 灵魂 / 身体 / 工具”讲：GM_Coop 只在服务器，负责登录、开局、刷怪、胜负；GS_Coop 复制比赛阶段和结束时刻；PS_Coop 记准备和击杀；PC_Coop 管本地 UI 和请求；Character 是身体，持有武器；武器挂在 Owner 链上；血量是 AC_Health 组件，玩家和僵尸共用；僵尸的 AI 在服务器的 AIController + 行为树里。（第 7、8 章）

追问 1：分数为什么放 PlayerState，不放 Character?

Character 是一条命，死了就销毁、重生是新的 Actor；PlayerState 跟人走，整局都在，而且总是复制给所有人（GameState.PlayerArray 每端都有），记分板直接读它。

追问 2：那为什么不能在 GameState 里存一个 PlayerController 数组?

PlayerController 只存在于服务器和它自己的客户端；复制到别的客户端时，引用指向一个不存在的对象，变成空。原 Demo 的 ReadyPlayersList 就是这样。要列举玩家，用 PlayerArray 里的 PlayerState。

追问 3：改成 4 人、Dedicated Server，要动哪些地方?

规则层基本不用改（本来就不依赖“主机玩家”）：MinPlayers 改 4、会话人数改 4；“只有主机能重开”的判断要换成“所有人投票”或由服务器自动重开；Listen Server 下手动播远端表

现的分支在 DS 上不会走到；僵尸的骨骼命中要打开服务器端的骨骼刷新或改用简化碰撞体；再按人数调 NetUpdateFrequency。

防守要点

一定要说“谁在哪一端”；不要只背类名。主动提一句“原 Demo 的关卡蓝图承担了开局逻辑，是典型的上帝对象，我把它拆到了 GameMode 和 GameState”。

链 2 “从开火到击中僵尸，说一遍。”

三段：本机预测表现并发请求 → 服务器校验、扣弹、判定、伤害 → 各端从复制状态播表现。（第 4、11 章）

追问 1：客户端判定命中，不怕外挂吗?

怕，所以服务器要校验：射速（令牌桶）、弹药、起点离眼睛的距离、方向与服务器视角的夹角、目标包围盒加容差后是否被射线穿过、视线是否被墙挡。这是 ShooterGame 的做法，适合 PvE 合作；PvP 我会改成服务器回溯，不信任客户端的命中结果。

追问 2：服务器回溯具体怎么做？回溯多少?

服务器每个网络帧记录候选目标的胶囊（或关键骨骼 hitbox）位置；收到开火时按客户端看到的时刻（客户端上报的服务器时间估计，减去插值延迟）在相邻两帧之间插值，把目标临时移过去做射线再移回来。最大回溯一般 150~250 ms，超过就不补偿。E3 里 RTT 300 ms 超过 200 ms 上限时，回溯命中率掉回 11%。

追问 3：回溯的代价是什么?

被打的人会“躲到墙后还被打中”（E3：480 cm/s、RTT 200 ms 时平均 79 cm 的受害者位移）；高延迟玩家占便宜；服务器每帧存历史、每发做回溯有 CPU 和内存开销。所以要设上限，并且只回溯射线附近的候选目标。

防守要点

别说“客户端打中了就调 RPC 扣血”；一定要带上“服务器校验”。被问到 PvP 时主动区分 PvE / PvP 的取舍，这是面试官想听的判断力。

链 3 “角色、控制器、武器怎么绑定？”

服务器 Possess：Pawn 的 Owner = PC；武器在服务器生成，生成参数里 Owner = Pawn、Instigator = Pawn；CurrentWeapon 复制，在 OnRep 里各端挂到 3P 和 1P 网格。（第 3、10 章）

追问 1：Owner 和 Instigator 有什么区别?

Owner 是网络归属：决定谁能发 Server RPC、OwnerOnly 发给谁、相关性；Instigator 是责任归属：这件事是谁造成的，用于伤害统计、友伤、仇恨。武器两者都是持有者，但子弹、榴弹这类抛射物的 Owner 可以是武器，Instigator 仍是开火的 Pawn。

追问 2：为什么不先 SpawnActor 再 SetOwner?

SpawnActor 内部就会执行 BeginPlay，此时 Owner 还是空的，武器里依赖 Owner 的初始化全部失效——原 Demo 就是这样，作者又补了一个 for_server 函数。生成参数里给 Owner，或者用 SpawnActorDeferred / Begin Deferred Actor Spawn 在 FinishSpawning 前设置。

追问 3：切换武器时旧武器怎么处理?

两种：单武器 Demo 直接在服务器 Destroy 旧武器；多武器背包则保留 Actor，隐藏网格、关闭 Tick，必要时设为休眠（Dormant）减少复制开销，切回来时 FlushNetDormancy。无论哪种，切枪前先 CancelReload、StopFire，避免旧武器的计时器继续改弹药。

防守要点

说出“挂接（AttachParent）和 Owner 是两回事，挂接信息不随武器复制”，这是能区分“背过”和“做过”的细节。

链 4　“弱网卡顿怎么优化？”

先复现定位（NetEmulation、stat net、p.NetShowCorrections、Networking Insights），再分四类：反馈慢→预测；拉回→移动预测一致；打不中→延迟补偿；堆积 / 带宽→Reliable 使用、更新频率、相关性。（第 6 章）

追问 1：CharacterMovement 的预测是怎么工作的?

拥有者每帧本地移动并保存 SavedMove，通过 ServerMove 把输入发给服务器；服务器用同样输入重放，误差平方超过 MAXPOSITIONERRORSQUARED（默认 3）就发 ClientAdjustPosition；客户端回到服务器位置，重放之后还没被确认的 SavedMove。纠正发送有最小间隔（NetworkMinTimeBetweenClientAdjustments，默认 0.1 s）。

追问 2：为什么冲刺一定要进 SavedMove？Server RPC 不行吗？

RPC 和移动包走的是不同的消息，到达时机不同：切换冲刺的那一刻，服务器重放某几帧时用的速度和客户端当时用的不一样，就会纠正。E6 的“半修复”在不丢包时 0 纠正，10% 丢包时每秒仍有 0.56 次；放进 SavedMove 后，服务器重放每一帧都用那一帧的标志，时序天然一致。

追问 3：纯蓝图项目怎么办？

纯蓝图扩展不了 SavedMove。次优解是本机和服务器同时切换、两端用同一个函数算速度，接受偶发纠正；或者引入约 60 行 C++ 的移动组件子类——这是我在 Demo 里唯一写的 C++。UE 5.4 起还有实验性的 Mover 插件，我了解但没有在项目里用。

防守要点

“弱网优化”最忌讳一上来就说“提高 Tick 频率、压缩数据”。先分类、再对症；说出“预测只能消除自己的延迟，消除不了别人看到你的延迟”（E1），会显得你真的想清楚了。

链 5 “子弹同步和换弹怎么做的？”

弹药 OwnerOnly 复制、本机预测 + 序号对账；别人看开枪靠 SkipOwner 的 BurstCounter；换弹是服务器计时状态机，一次性 Take，取消不动弹药。（第 5、11 章）

追问 1：AckSeq 解决了什么问题？

OnRep 收到的服务器弹数是“处理完某一发时”的值，本机这时往往已经多打了几发；直接覆盖会让 UI 往回跳（E5：每次连射平均回跳 1 次）。服务器把已处理到的序号一起发回来，本机显示 = 服务器弹数 − 还没被处理的发数，就不跳了；被拒的发数也会因此自动加回来。

追问 2：换弹到一半被打死怎么办？

死亡时服务器调用 CancelReload：清计时器、复位 bPendingReload，弹药一发不动——因为转移只在计时器到点时一次性发生。原 Demo 一开始就把当前弹数清零，被打断会凭空丢子弹。

追问 3：换弹时长从哪来？为什么不挂 AnimNotify？

从 DA_Weapon 的 ReloadDuration（与蒙太奇时长对齐）来。服务器上动画不一定按时更新（Dedicated Server 不刷新不可见网格、URO 降频），蒙太奇被打断时通知也不来；规则用服务器计时，动画只做表现。

防守要点

主动说出原 Demo 的换弹 bug（漏连 Return 返回 0），并说“是我把蓝图逐节点移植成测试代码才发现的”——这比任何概念都有说服力。

链 6 “Reliable 和 Unreliable 怎么选？”

按“丢了怎么办”选：必须到达、低频的用 Reliable（开火请求、换弹请求、准备）；表现类、高频的用 Unreliable 或复制计数器。（第 2、6 章）

追问 1：开火请求用 Reliable，丢包时不会卡吗？

会排队：E2 中 RTT 200 ms、丢包 10% 时 P99 处理延迟 554 ms。但因为本机已经预测了表现，排队只推迟伤害生效，不影响手感；换成 Unreliable 则约 10% 的命中会直接丢失，“打中了不扣血”更难接受。

追问 2：扎堆到达带来了什么新问题？

射速校验如果比较到达间隔，会把重传后挤在一起的正常请求误判为超射速（图 11-7）。我改成令牌桶：允许 2~3 发的短时突发，长期平均不超过射速。

追问 3：射速很高（比如 20 发/秒）怎么办？

把同一帧内的多发合并成一个数组参数发送，减少 RPC 数量；或者改成“开始开火 / 停止开火 + 服务器按射速自己模拟”，客户端只上报每发的方向（ShooterGame 的 BurstCounter 思路）。另外不要在 Tick 里发 Reliable，可靠缓冲区满了会被断开连接。

防守要点

说出“Reliable 是有序的”——队头阻塞就是从这里来的；很多人只知道“可靠 = 不丢”。

链 7 “晚加入的玩家怎么看到正确的状态？”

持续状态全部是复制属性：比赛阶段、结束时刻、僵尸的 bIsDead、武器的 bPendingReload、外观种子；新连接建立后服务器会把当前值复制过去，OnRep 里播对应的表现。（第 2、12 章）

追问 1：为什么 RPC 不行？

RPC 只对当时在线且相关的客户端执行一次，晚加入的人永远收不到。原 Demo 的死亡、开局都靠多播，晚加入者看到的僵尸永远站着，“游戏是否开始”永远是 false。

追问 2：Initial Only 用在哪？

只在 Actor 首次复制时发送、之后不再比较的属性，比如僵尸的外观种子 VariantSeed：出生后不会变，省掉每次复制时的比较开销。

追问 3：尸体用 bIsDead 复制还是 TearOff?

两种都可以。bIsDead 复制简单，尸体由服务器 SetLifeSpan 统一销毁；TearOff 让服务器停止复制这个 Actor，各端接管自己的那份（布娃娃各算各的、不再同步），适合大量尸体、节省带宽，但 TearOff 之后服务器就不能再改它了。Demo 里我用 bIsDead + 5 秒销毁。

防守要点

一句话总结：**状态用属性，事件用 RPC 或计数器。**

链 8　“Listen Server 的主机有什么特殊？”

主机既是服务器又是玩家：它的角色 HasAuthority 和 IsLocallyControlled 同时为 true；它调用 Server RPC 时直接在本地执行；SkipOwner 属性的 OnRep 不会在服务器上触发，主机看客户端开火要在服务器上手动播远端表现。（第 2、4 章）

追问 1：主机是不是有延迟优势?

有，主机的 RTT 为 0，命中判定和移动都最“准”。合作 PvE 可以接受；竞技 PvP 一般用 Dedicated Server。

追问 2：主机掉线怎么办?

UE 不内置主机迁移。合作游戏通常的处理是：客户端收到 NetworkError / 连接关闭，GameInstance 回到主菜单并提示“主机已断开”。要做迁移需要自己同步全部状态到候选主机并重建会话，成本很高，Demo 不做。

追问 3：主机能作弊吗?

能，主机就是服务器，规则在它手里。所以 Listen Server 只适合朋友之间的合作；需要公平性时用 Dedicated Server，并把校验逻辑都放在服务器上（本 Demo 的校验在 DS 上照样生效）。

防守要点

别说“Listen Server 就是有一个玩家当服务器”就停下，要说出它对代码的影响：权限判断和本地控制判断必须分开写。

链 9　“为什么不用 GAS？”

Demo 的目标是把复制、RPC、预测这些底层机制讲清楚，用 GAS 会把这些封装起来；功能也只有一把枪、一种伤害，引入 GAS 的成本（AttributeSet、Ability、Effect、Cue、预测键）不划算。

追问 1：那你了解 GAS 的预测吗?

Ability 在客户端激活时生成一个 Prediction Key，随激活请求发给服务器；客户端在这个键下预测执行的 GameplayEffect、Tag、Cue，在服务器确认或拒绝后保留或回滚。我的 AckSeq 其实是这个思想的简化版：用序号把“本地预测的东西”和“服务器确认的东西”对上。

追问 2：如果做正式项目呢?

会用：开火、换弹做成 Ability，伤害是 GameplayEffect，表现是 GameplayCue，弹药放在物品上的 Tag Stack，参考 Lyra。底层的 Owner、复制条件、移动预测这些知识在 GAS 里同样需要。

防守要点

不要贬低 GAS，也不要假装用过；说清“为什么这个规模不用”和“到什么规模会用”。

链 10　“僵尸变成 100 只，怎么优化？”

分网络、AI、动画三块。

追问 1：网络上?

降低僵尸 NetUpdateFrequency（10~20），远处的用距离相关性裁掉；死亡用 TearOff；只复制必要状态（外观种子 Initial Only、攻击计数器）；人更多时考虑 Replication Graph 或 Iris 做空间分区和优先级。

追问 2：AI 和动画上?

AI：行为树 Service 降频、远处僵尸切到简化逻辑（只追不打），用 Significance Manager 分级；寻路用 Crowd / RVO 但限制数量。动画：URO（更新频率随距离降低）、远处 LOD 减骨、Visibility Based Anim Tick 设为不可见时不刷新——但服务器上需要判定命中的僵尸要保留骨骼刷新或改用简化碰撞体。

追问 3：怎么知道优化有没有用?

先量化：stat net 看带宽、Networking Insights 看哪个 Actor 占字节最多、Unreal Insights 看服务器帧时间；每改一项对比一次。

防守要点

先说“先测再改”，再分块列手段；不要一口气背十个名词。

链 11 “书里这些数字是怎么来的？可信吗？”

两类证据：蓝图连线是从 .uasset 解析出来的事实（每个节点都有编号可查）；性能 / 时序数字来自我写的离散事件模拟，它按 UE 的默认参数（30 Hz 网络帧、Reliable 有序重传、纠正阈值和最小间隔）复现相关行为。（附录 A）

追问 1：模拟和真实引擎有什么差距?

没有模拟带宽限制、包合并、优先级调度、移动平滑；抖动是均匀分布；所以只用来比较方案的相对好坏，绝对值以引擎里实测为准。

追问 2：那在引擎里怎么测?

PIE 网络模拟 + 录屏逐帧看；p.NetShowCorrections 数纠正次数；Networking Insights 看 RPC 的发出与到达时间；自动化测试可以用 Gauntlet 跑多进程。

防守要点

主动说出局限，比被问出来好。

链 12 “这个 Demo 现在还有什么不足？”

诚实列三条：① 只支持局域网 2 人，没有主机迁移；② PvE 的命中采用客户端判定 + 容差，不适合 PvP；③ 冲刺 / 瞄准的移动预测用了少量 C++，纯蓝图版本做不到零纠正。

追问：下一步你会做什么?

按优先级：加 Dedicated Server 构建和自动化多进程测试；把开火 / 换弹迁到 GAS；做服务器回溯并支持简单的 PvP 模式；用 Networking Insights 做一次带宽基线。

防守要点

“没有不足”是最差的答案。说不足的同时给出下一步，说明你知道“合格之后”是什么。

危险说法对照表

不要这么说	问题	换成
“联机就是把变量都勾上复制”	复制单向、只发最新值、条件不同；事件不能用状态表达	“状态用复制属性，事件用 RPC 或计数器，表现各端本地做”
“用 Multicast 让所有人同步”	客户端调用只在本地执行；晚加入收不到；开火者也要等	“持续状态用属性 + OnRep，一次性表现才用 Unreliable 多播”
“HasAuthority 判断是不是自己”	Listen Server 主机两个都是 true	“改状态看 HasAuthority，本地表现看 IsLocallyControlled”
“服务器上禁用玩家输入”	输入在客户端处理；Pawn 的 DisableInput 只接受自己的 Controller	“客户端本地关输入，服务器在每个请求里校验”
“GameInstance 存全局数据大家共享”	每个进程一份，不复制	“跨地图的本地数据放 GameInstance，共享状态放 GameState”
“网络卡就提高 Tick / 压缩数据”	没有分类，抓不住主要矛盾	“先分反馈慢、拉回、打不中、堆积四类，再对症”
“Reliable 不会丢，所以都用 Reliable”	有序 → 队头阻塞；缓冲溢出会断线	“按丢了怎么办来选”
“我的 Demo 没什么问题”	面试官一定能找到问题	“原版有 X 类问题，我是这样发现和修复的”

被问住时怎么办

说出你知道的上一层机制，再给出验证路径：“这个细节我没有在引擎源码里确认过。我的理解是 X，因为 Y；如果要确认，我会在 Z 函数里下断点 / 用 Networking Insights 看消息到达顺序。”面试官更在意你的推理过程和验证方法，而不是记住了多少 API。

CHAPTER 16

56 道面试追问与速记卡

按“基础 → 绑定 → 开火命中 → 弹药换弹 → 弱网 → 项目本身 → Gameplay 框架与蓝图实现 → 执行端与复制标注”排列。每题先给能直接说出口的答案，括号里是本书对应位置。

一、网络基础（1~9）

▶ **1. UE 的网络模型是什么？Listen Server 和 Dedicated Server 区别？**

▶ **2. GameMode、GameState、PlayerState、PlayerController 分别在哪几端存在？**

▶ **3. Role 有哪几种？主机玩家的角色在主机上是什么 Role？**

▶ **4. 能不能在一个 Actor 上调用 Server RPC，取决于什么？**

▶ **5. 客户端调用 Multicast 会怎样？服务器调用 Client RPC 但 Actor 没有拥有者会怎样？**

▶ **6. Reliable 和 Unreliable 怎么选？Reliable 滥用会怎样？**

▶ **7. 属性复制有哪些特点？为什么“事件”不适合用普通复制属性表达？**

▶ **8. C++ 的 OnRep 和蓝图的 RepNotify 有什么区别？**

▶ **9. 常用的复制条件有哪些？各举一个例子。**

二、角色、控制器、武器（10~15）

▶ **10. 讲一下 Possess 的过程。**

▶ **11. 武器应该在哪生成？Owner 和 Instigator 分别设成什么、有什么用？**

▶ **12. 武器挂到角色手上，挂接信息会自动复制吗？**

▶ **13. 第一人称射击联机后，为什么要两套模型？**

▶ **14. Enhanced Input 的映射应该在哪里添加？为什么不放在 BeginPlay？**

▶ **15. 在服务器上调用 GetPlayerController(0) 拿到的是什么？**

三、开火到命中（16~23）

▶ **16. 从按下开火到僵尸扣血，完整说一遍。**

▶ **17. 为什么开火表现要本机先播？只用 Multicast 有什么问题？**

▶ **18. 服务器需要校验客户端开火的哪些内容？**

▶ **19. 命中扫描和抛射物各适合什么？抛射物怎么做到本机不延迟？**

▶ **20. 原 Demo 的伤害为什么会在客户端也算一次？**

▶ **21. 僵尸的死亡应该怎么同步？**

▶ **22. Dedicated Server 上爆头判定可能出什么问题？**

▶ **23. Listen Server 主机看客户端开火，SkipOwner 的 BurstCounter 有什么坑？**

四、弹药与换弹（24~29）

▶ 24. 弹药应该怎么复制?

▶ 25. 预测扣弹后 OnRep 直接覆盖本地值，会出现什么？怎么解决?

▶ 26. 换弹的流程？期间开火、切枪、死亡怎么处理?

▶ 27. 原 Demo 的换弹 bug 是什么？怎么发现的?

▶ 28. 换弹加子弹应该挂在蒙太奇的 AnimNotify 上吗?

▶ 29. 另一端怎么看到“他在开枪”？为什么不用复制的 bIsFiring?

五、弱网与性能（30~37）

▶ 30. 弱网“卡”有哪几种？分别用什么工具定位?

▶ 31. 讲一下 CharacterMovement 的客户端预测和纠正。

▶ 32. 冲刺为什么会被拉回？正确做法?

▶ 33. 什么是延迟补偿？有哪些做法?

▶ 34. 延迟补偿的副作用是什么?

▶ 35. 远端角色 / 僵尸移动一顿一顿的，怎么查?

▶ 36. 有哪些降低带宽的手段?

▶ 37. 为什么即使 RTT 为 0，原 Demo 的开火反馈也有约 20 ms?

六、关于这个项目本身（38~40）

▶ 38. 这个 Demo 最大的问题是什么？你会先改哪三处?

▶ 39. 你是怎么验证这些问题的?

▶ 40. 如果做成正式项目，你会怎么组织网络代码?

七、Gameplay 框架与蓝图实现（41~50）

▶ 41. GameInstance、World、Level 是什么关系？切地图时谁会被销毁?

▶ 42. Outer、Owner、AttachParent、Instigator 分别管什么?

▶ 43. 服务器上一个 PlayerController 的 IsLocalController 为 false 说明什么?

▶ 44. 一个玩家 Pawn 的生命周期里，哪些蓝图事件只在服务器、哪些在拥有者、哪些在所有端?

▶ 45. 覆写 GameMode 的 HandleStartingNewPlayer 要注意什么?

▶ 46. UMG 能直接调用 Server RPC 吗?

▶ 47. 蓝图的 RepNotify 和 C++ 的 OnRep 在使用上有哪些差别?

▶ 48. 为什么换弹用 Set Timer by Event 而不是 Delay?

▶ 49. 僵尸的随机外观怎么在各端保持一致?

▶ **50. 你认为一个“合格”的联机 Demo 的标准是什么?**

八、执行端、复制与组播（51~56）

▶ **51. 一个事件在哪端执行，你怎么判断?**

▶ **52. Listen Server 的主机会“收到”复制吗？主机是怎么看到客户端玩家开火表现的?**

▶ **53. Owner Only、Skip Owner、Initial Only 分别发给谁？本 Demo 哪些变量用了?**

▶ **54. 什么时候用组播，什么时候用 RepNotify？原 Demo 的 8 个组播你怎么处理的?**

▶ **55. 在蓝图里让一个变量复制要勾哪几处？为什么有人勾了 RepNotify 却在客户端收不到?**

▶ **56. 客户端调用 Multicast 会怎样？客户端在不属于自己的 Actor 上调用 Server RPC 会怎样?**

速记卡

四个问题

这个对象在哪几端？这段代码在哪端执行？谁能调这个 RPC？这个变量发给谁?

一条总规则

状态用复制属性，瞬时事件用 RPC 或计数器，表现各端本地做，结果只认服务器。

两个判断

改状态看 HasAuthority；播本地表现看 IsLocallyControlled。主机玩家两个都是 true。

Owner 链

Weapon → Pawn → PC → Connection。决定 Server RPC 可达、OwnerOnly 发给谁。

开火三段

本机预测表现 + 发请求；服务器校验、扣弹、判定、伤害；各端 OnRep / 计数器播表现。

弹药

OwnerOnly 结构体带 AckSeq；显示 = InClip − 未确认发数；别人用 SkipOwner 的

BurstCounter。

换弹

服务器计时器状态机；到点 take = min(空位, 备弹)；取消不动弹药；本机预测动画。

移动

影响速度的状态进 SavedMove；两端用 GetMaxSpeed 同一规则算；不另发 RPC。

命中

PvE：客户端判定 + 容差校验；PvP：服务器回溯，限最大回溯时间。

Reliable

低频必达才用；表现类 Unreliable；Tick 里发 Reliable 必扣分。

整局流程

规则 GameMode、全局 GameState、个人 PlayerState；倒计时复制结束时刻；重开 ServerTravel。

框架职责

GameInstance 进程 · GameMode 规则 · GameState 全局 · PlayerState 人 · PC 灵魂与 UI · Pawn 身体 · Component 能力。

五种关系

Outer 管内存、Owner 管网络、AttachParent 管变换、Instigator 管责任、Possess 管控制。

蓝图三坑

OnRep 无旧值；结构体整体赋值；需要取消的等待用计时器不用 Delay。

测试

Dedicated Server 模式、独立进程、Bad 网络、晚加入、谁先死、同时击杀。

执行端

RPC 看类型；普通事件跟调用者；OnRep = 服务器（蓝图 Set 时）+ 收到的客户端；GameMode / AI 只在服务器，Widget 只在本机。

复制三层 + 三条件

Actor Replicates → Component Replicates → 变量 Replicated / RepNotify。OwnerOnly 只给自己，SkipOwner 给别人，InitialOnly 只发一次。

组播四条件

服务器调用、一瞬间、晚加入不需要、丢了无所谓 → Unreliable 组播；否则用 RepNotify。原 8 个：7 个换 RepNotify、1 个删。

APPENDIX · A

原 Demo 问题全清单、审计方法与实测总表

每条问题都能在 audit/*.txt 中按节点编号查到；“推断”类给出在编辑器里验证的方法。

A.1 问题全清单（23 条）

#	级别	位置	问题	修复
1	P0	BP_Weapon.剩余子弹计算	false 分支未连 Return，返回 0，换弹吞弹匣	一次性 take 备弹)
2	P0	BP_Bullet.ReceiveHit / 给丧尸伤害	每端都 ApplyDamage	伤害前 Swit Authority；
3	P0	BP_Zombie.Multicast_Death → BP_ZombieSpawn.重新生成	补刷在每端执行，且跳过了 SwitchHasAuthority	服务器通知 补刷
4	P0	BP_Player.ReceiveAnyDamage	在服务器上 DisableInput(GetPlayerController(0)) / PlaySound2D：客户端玩家死时 DisableInput 只报 LogPawn 错误不生效，死者仍能操作；惨叫在主机响	服务器只改 验请求；拥 OnRep 里关
5	P0	BP_Player.冲刺 / 瞄准 / 速度计算	IsRunning 仅服务器、是否瞄准 仅本机；Tick 里各自写 MaxWalkSpeed	标志进 Save GetMaxSpe
6	P0	UWB_InGame.计算剩余时间	倒计时与胜利在每端 UI 自算	GameState 刻，GameM
7	P0	BP_Player.Multi_Death	任一玩家死亡，每端都弹“失败”并暂停	GameMod 者观战
8	P0	UWB_Pause / UWB_WinOrFail	SetGamePaused 客户端无效、主机冻结全员；OpenLevel 重开断线	暂停只开 U ServerTrave
9	P1	BP_Weapon.Server_Fire	本机表现等多播（≈ RTT + 网络帧）	本机预测播
10	P1	BP_Weapon.Server_Fire	Multicast_Fire 在 CheckCanFire 之前：空放、打断换弹	服务器接受 BurstCount
11	P1	BP_Weapon.Server_Fire	逐发 Reliable，丢包时队头阻塞	表现不依赖 合并

12	P1	BP_Bullet	每发一个复制 Actor：远端突现、消息量大	命中扫描 +
13	P1	BP_Weapon.后坐力 / 子弹发射位置旋转	后坐力在服务器调用 AddControllerInput（客户端无效）；散布读服务器 AnimBP 的 是否瞄准	后坐力本地移动预测或报
14	P1	BP_Zombie.Multicast_Death	ClearTimer("追逐玩家") 名字不存在；FindClosestTarget / 攻击范围检测 仍在跑	ClearAllTim + StopLog
15	P1	关卡蓝图 Server_SetStart	客户端调用被丢弃；游戏是否开始 非复制，晚加入者永远 false	GameState
16	P1	关卡蓝图 BeginPlay / BP_Player BeginPlay	绑定 GetPlayerCharacter(0)、Cast GetGameState 依赖复制时序	在 OnRep_ PawnClient OnRep_Ph
17	P2	BP_Player 组件	只有 1P 手臂；bUseControllerRotationPitch 让胶囊俯仰	1P + 3P 网 Yaw
18	P2	BP_MyGameState.ReadyPlayersList / GM.AddReadyPlayer	复制 PC 引用（客户端为空）；GameMode 上标 Server RPC	PlayerState
19	P2	BP_Bullet / BP_Zombie ApplyDamage	Instigator / Causer 为空	填 Controll
20	P2	BP_Weapon 复制变量；BP_Player.MouseY	弹药无条件复制给所有人；MouseY 在客户端写复制变量	OwnerOnly
21	P1	BP_Zombie.初始化速度和动画	速度与动画变体只在服务器随机并设置（SwitchHasAuthority 之后），AnimBP 变量不复制	服务器复制（Initial Or 同一 Rando
22	P2	BP_Zombie.Multicast_PlayAttackAnim / Multicast_HitReact	多播里 RandomIntegerInRange 选蒙太奇：各端各自随机，动作不一致；On Completed 里各端写复制变量 正在攻击	服务器选 In AttackInfo bAttacking 计时器复位

23	P3	多处	for_server 补丁、空 OnRep、未连接的 servertravel / 计算子弹数量 节点、重复变量 As BP Player_0	清理；生成

A.2 没有 UE 编辑器，怎么读蓝图

仓库里的蓝图是 Git LFS 管理的 .uasset。为了只看逻辑、不下载几百 MB 的美术资源，做法是：

只取逻辑资产（不整体拉取 LFS）

```
$ git clone --filter=blob:none --no-checkout --depth 1 https://github.com/dxHWH/-demo.git repo
$ git sparse-checkout init --no-cone
#   只检出 /Config/、/Content/Blueprints/、/Content/UI/UWB_*、主地图
$ GIT_LFS_SKIP_SMUDGE=1 git checkout HEAD          # 得到的是 LFS 指针（几十字节的文本）
#   逐个按指针里的 size 从 media.githubusercontent.com 下载实体，校验大小一致后替换
$ python tools/bpdump.py repo/Content/Blueprints/Weapon/BP_Weapon.uasset audit/BP_Weapon.txt
```

`tools/uasset.py` 实现了 UE 5.5 编辑器包的最小读取：包头、名称表、导入 / 导出表、带类型名的属性流（5.4 起的完整类型名标签）、以及 `UEdGraphNode` 在属性之后序列化的引脚数组（引脚名、方向、类型、默认值、连线）。`tools/bpdump.py` 在此之上：

- 从 Blueprint 对象读出父类、变量（复制标志、RepNotify、复制条件）；从 CDO 和组件模板读出 bReplicates、碰撞、网格等设置；
- 从 CustomEvent 的 FunctionFlags 读出 RPC 类型（Net / Reliable / Server / Client / Multicast）；
- 从每个事件入口沿执行引脚追踪，把数据引脚还原成嵌套表达式，输出缩进伪代码；标出未连接的执行节点、被禁用的节点。

这种静态审计的边界

能确定“节点怎么连、在哪个事件里、标了什么 RPC”，不能确定运行时的时序（例如 BeginPlay 时 GameState 是否已到达）。所以书里把“UE 运行规则 + 连线”推出的结论标为 **推断**，并在 A.1 给出在 PIE 里验证的方法；需要定量的结论用 `demos/coop_net_demo.cpp` 模拟。

A.3 实测总表（demos/run_output.txt）

实验	内容	关键数字	用在
E1	开火链路时序：原方案 / 预测 + 覆盖 / 预测 + 对账，RTT 0~200	RTT 200：本机反馈 237 → 0 ms；另一端 235 ms 不变；子弹突现 3.3 m	第 4 章
E2	逐发 Reliable 在丢包下的处理延迟	RTT 200 / 10%：P99 554 ms，最大 1028 ms；Unreliable 会丢 10.2%	第 4、6 章
E3	延迟补偿命中率（目标横移、随机折返）	480 cm/s、RTT 200：不补偿 10.9%，回溯 99.4%，容差 100.0%	第 6 章
E4	换弹逻辑逐节点移植 vs 修正版	3/7 → 0/0（丢 10）；整局 30/40 vs 40/40	第 5 章
E5	弹药 UI 的三种显示方式	直接覆盖回跳 1.00 次 / 连射，对账 0.00 次	第 5 章
E6	CMC 纠正：MaxWalkSpeed 两端不一致	冲刺只在服务器：9.2 次/秒、平均 21.4 cm；半修复 10% 丢包 0.56 次/秒；正确 0	第 6 章
E7	每秒消息量估算	3767 vs 333 B/s（量级）	第 6 章

模拟的假设

服务器网络 Tick 30 Hz，属性与 Unreliable 多播在下一个网络帧发出；Reliable 有序，丢包后约 1 RTT + 1 帧重传；客户端 60 Hz；纠正阈值 √3 cm、最小间隔 0.1 s（大误差 0.05 s）；E1 / E5 的抖动为 20 / 30 ms 均匀分布，乱序到达的旧属性包丢弃；E3 的模拟代理按“最新快照 + 速度 × 收到后经过的时间”外推，服务器回溯在相邻两个网络帧之间线性插值，上限 200 ms。E6 把原蓝图的 FInterpTo 速度渐变简化为直接切换。所有实验固定随机种子，带参数运行（如 `coop_net_demo.exe E3`）只跑指定实验。